OptiPy

Generated by Doxygen 1.16.1

1 Topic Index 1
1.1 Topics . . . 1

2 Concept Index 3
2.1 Concepts . . . 3

3 Hierarchical Index 5
3.1 Class Hierarchy . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 File Index 9
5.1 File List . . . 9

6 Topic Documentation 11
6.1 Flow Shop Scheduling . . . 11
6.1.1 Detailed Description . . . 11
6.2 Flow Shop Results . . . 11
6.2.1 Detailed Description . . . 11
6.3 Optimization Algorithms . . . 11
6.3.1 Detailed Description . . . 12
6.3.2 Differential Evolution Optimizer . . . 12
6.3.2.1 Detailed Description . . . 12
6.3.2.2 DE Crossover Strategies . . . 12
6.3.2.3 DE Mutation Strategies . . . 13
6.3.2.4 DE Optimization Results . . . 14
6.3.3 Concept Definitions . . . 14
6.4 Optimization Problems . . . 14
6.4.1 Detailed Description . . . 14

7 Concept Documentation 15
7.1 Evaluable Concept Reference . . . 15
7.1.1 Concept definition . . . 15
7.1.2 Detailed Description . . . 15

8 Class Documentation 19
8.1 AckleyOne Class Reference . . . 19
8.1.1 Detailed Description . . . 20
8.1.2 Member Function Documentation . . . 20
8.1.2.1 evaluate() . . . 20
8.2 AckleyTwo Class Reference . . . 21
8.2.1 Detailed Description . . . 22
8.2.2 Member Function Documentation . . . 22
8.2.2.1 evaluate() . . . 22
8.3 Best1 Class Reference . . . 22

8.3.1 Detailed Description . . . 23
8.3.2 Member Function Documentation . . . 26
8.3.2.1 mutate() . . . 26
8.4 Best2 Class Reference . . . 27
8.4.1 Member Function Documentation . . . 28
8.4.1.1 mutate() . . . 28
8.5 BinCrossover Class Reference . . . 28
8.5.1 Detailed Description . . . 29
8.5.2 Member Function Documentation . . . 32
8.5.2.1 crossover() . . . 32
8.6 Blind Class Reference . . . 33
8.6.1 Detailed Description . . . 34
8.6.2 Constructor & Destructor Documentation . . . 34
8.6.2.1 Blind() . . . 34
8.6.3 Member Function Documentation . . . 34
8.6.3.1 optimize() . . . 34
8.7 Crossover Class Reference . . . 35
8.7.1 Detailed Description . . . 35
8.7.2 Member Function Documentation . . . 36
8.7.2.1 crossover() . . . 36
8.8 DeJong Class Reference . . . 38
8.8.1 Detailed Description . . . 38
8.9 DeJongOne Class Reference . . . 38
8.9.1 Member Function Documentation . . . 39
8.9.1.1 evaluate() . . . 39
8.10 DifferentialEvolution Class Reference . . . 40
8.10.1 Detailed Description . . . 40
8.10.2 Member Function Documentation . . . 41
8.10.2.1 createCrossover() . . . 41
8.10.2.2 createMutation() . . . 41
8.10.2.3 crossover() . . . 42
8.10.2.4 optimize() . . . 42
8.11 EggHolder Class Reference . . . 44
8.11.1 Detailed Description . . . 45
8.11.2 Member Function Documentation . . . 45
8.11.2.1 evaluate() . . . 45
8.12 ExpCrossover Class Reference . . . 45
8.12.1 Detailed Description . . . 46
8.12.2 Member Function Documentation . . . 50
8.12.2.1 crossover() . . . 50
8.13 ExperimentConfig Struct Reference . . . 51
8.13.1 Detailed Description . . . 51
8.14 FlowShop Class Reference . . . 51

8.14.1 Detailed Description . . . 53
8.14.2 Constructor & Destructor Documentation . . . 54
8.14.2.1 FlowShop() [1/3] . . . 54
8.14.2.2 FlowShop() [2/3] . . . 54
8.14.2.3 FlowShop() [3/3] . . . 54
8.14.3 Member Function Documentation . . . 55
8.14.3.1 data() [1/2] . . . 55
8.14.3.2 data() [2/2] . . . 55
8.14.3.3 due_dates_data() [1/2] . . . 55
8.14.3.4 due_dates_data() [2/2] . . . 56
8.14.3.5 evaluateSolution() . . . 56
8.14.3.6 getInitialSolutions() . . . 57
8.14.3.7 getLowerBounds() . . . 57
8.14.3.8 getUpperBounds() . . . 58
8.14.3.9 num_jobs() . . . 58
8.14.3.10 num_machines() . . . 58
8.14.3.11 runDE() . . . 58
8.14.3.12 runNEH() . . . 60
8.14.3.13 set_due_dates() . . . 60
8.14.3.14 set_jobs() . . . 61
8.15 FlowShopResult Struct Reference . . . 61
8.15.1 Detailed Description . . . 62
8.15.2 Member Data Documentation . . . 64
8.15.2.1 completionTimes . . . 64
8.15.2.2 fitnesses . . . 65
8.15.2.3 makespan . . . 66
8.15.2.4 sequence . . . 67
8.15.2.5 tardiness . . . 67
8.16 Griewangk Class Reference . . . 68
8.16.1 Detailed Description . . . 69
8.16.2 Member Function Documentation . . . 69
8.16.2.1 evaluate() . . . 69
8.17 LocalSearch Class Reference . . . 70
8.17.1 Detailed Description . . . 71
8.17.2 Constructor & Destructor Documentation . . . 71
8.17.2.1 LocalSearch() . . . 71
8.17.3 Member Function Documentation . . . 71
8.17.3.1 optimize() . . . 71
8.18 MersenneTwister Class Reference . . . 72
8.18.1 Detailed Description . . . 72
8.18.2 Constructor & Destructor Documentation . . . 72
8.18.2.1 MersenneTwister() . . . 72
8.18.2.2 ~MersenneTwister() . . . 72

8.18.3 Member Function Documentation . . . 73
8.18.3.1 genrand_int31() . . . 73
8.18.3.2 genrand_int32() . . . 73
8.18.3.3 genrand_real1() . . . 73
8.18.3.4 genrand_real2() . . . 73
8.18.3.5 genrand_real3() . . . 74
8.18.3.6 genrand_res53() . . . 74
8.18.3.7 init_by_array() . . . 74
8.18.3.8 init_genrand() . . . 74
8.18.3.9 print() . . . 75
8.19 Mutation Class Reference . . . 75
8.19.1 Detailed Description . . . 76
8.19.2 Member Function Documentation . . . 77
8.19.2.1 getSubset() . . . 77
8.19.2.2 mutate() . . . 78
8.20 Optimizer Class Reference . . . 81
8.20.1 Detailed Description . . . 82
8.20.2 Constructor & Destructor Documentation . . . 82
8.20.2.1 Optimizer() . . . 82
8.20.3 Member Function Documentation . . . 82
8.20.3.1 getBestFitness() . . . 82
8.20.3.2 getBestFitnesses() . . . 83
8.20.3.3 getBestSolution() . . . 83
8.20.3.4 getMaxIterations() . . . 83
8.20.3.5 getProblem() . . . 83
8.20.3.6 getSolutionBuilder() . . . 83
8.20.3.7 getSolutions() . . . 83
8.20.3.8 optimize() . . . 84
8.21 OptimizerFactory Class Reference . . . 84
8.21.1 Detailed Description . . . 84
8.21.2 Member Function Documentation . . . 84
8.21.2.1 initOptimizer() . . . 84
8.22 OptResult Struct Reference . . . 85
8.22.1 Detailed Description . . . 85
8.22.2 Member Data Documentation . . . 87
8.22.2.1 bestFitness . . . 87
8.22.2.2 bestFitnesses . . . 88
8.22.2.3 bestSolution . . . 90
8.23 ParticleSwarm Class Reference . . . 91
8.23.1 Detailed Description . . . 92
8.23.2 Member Function Documentation . . . 93
8.23.2.1 optimize() . . . 93
8.24 Problem Class Reference . . . 93

8.24.1 Detailed Description . . . 94
8.24.2 Constructor & Destructor Documentation . . . 94
8.24.2.1 Problem() . . . 94
8.24.3 Member Function Documentation . . . 95
8.24.3.1 evaluate() . . . 95
8.24.3.2 getLowerBounds() . . . 95
8.24.3.3 getName() . . . 95
8.24.3.4 getUpperBounds() . . . 96
8.25 ProblemFactory Class Reference . . . 96
8.25.1 Detailed Description . . . 96
8.25.2 Member Function Documentation . . . 96
8.25.2.1 create() . . . 96
8.26 ProblemResult Class Reference . . . 97
8.27 Rand1 Class Reference . . . 97
8.27.1 Detailed Description . . . 98
8.27.2 Member Function Documentation . . . 101
8.27.2.1 mutate() . . . 101
8.28 Rand2 Class Reference . . . 102
8.28.1 Detailed Description . . . 103
8.28.2 Member Function Documentation . . . 107
8.28.2.1 mutate() . . . 107
8.29 RandBest1 Class Reference . . . 107
8.29.1 Detailed Description . . . 108
8.29.2 Constructor & Destructor Documentation . . . 113
8.29.2.1 RandBest1() . . . 113
8.29.3 Member Function Documentation . . . 114
8.29.3.1 mutate() . . . 114
8.30 Rastrigin Class Reference . . . 115
8.30.1 Detailed Description . . . 116
8.30.2 Member Function Documentation . . . 116
8.30.2.1 evaluate() . . . 116
8.31 Rosenbrock Class Reference . . . 117
8.31.1 Detailed Description . . . 118
8.31.2 Member Function Documentation . . . 118
8.31.2.1 evaluate() . . . 118
8.32 Schwefel Class Reference . . . 118
8.32.1 Detailed Description . . . 119
8.32.2 Member Function Documentation . . . 119
8.32.2.1 evaluate() . . . 119
8.33 SineEnvelope Class Reference . . . 120
8.33.1 Detailed Description . . . 121
8.33.2 Member Function Documentation . . . 121
8.33.2.1 evaluate() . . . 121

8.34 SolutionBuilder Class Reference . . . 121
8.34.1 Detailed Description . . . 122
8.34.2 Constructor & Destructor Documentation . . . 122
8.34.2.1 SolutionBuilder() . . . 122
8.34.3 Member Function Documentation . . . 122
8.34.3.1 checkBounds() . . . 122
8.34.3.2 getNeighbors() . . . 123
8.34.3.3 getRand() . . . 124
8.34.3.4 getSubset() . . . 124
8.35 StretchedV Class Reference . . . 125
8.35.1 Detailed Description . . . 126
8.35.2 Member Function Documentation . . . 126
8.35.2.1 evaluate() . . . 126

9 File Documentation 127
9.1 src/opti_py/cpp/bindings.cpp File Reference . . . 127
9.1.1 Detailed Description . . . 127
9.2 debug.h . . . 128
9.3 src/opti_py/cpp/include/opti_py/ExperimentConfig.h File Reference . . . 128
9.3.1 Detailed Description . . . 128
9.4 ExperimentConfig.h . . . 128
9.5 mt.h . . . 129
9.6 src/opti_py/cpp/include/opti_py/FlowShop/FlowShop.h File Reference . . . 129
9.6.1 Detailed Description . . . 130
9.7 FlowShop.h . . . 130
9.8 src/opti_py/cpp/include/opti_py/FlowShop/FlowShopResult.h File Reference . . . 133
9.8.1 Detailed Description . . . 133
9.9 FlowShopResult.h . . . 134
9.10 src/opti_py/cpp/include/opti_py/Optimizer/Blind.h File Reference . . . 134
9.10.1 Detailed Description . . . 134
9.11 Blind.h . . . 135
9.12 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/AllCrossovers.h File Reference . . . 135
9.12.1 Detailed Description . . . 135
9.13 AllCrossovers.h . . . 136
9.14 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/BinCrossover.h File Reference . . . 136
9.14.1 Detailed Description . . . 136
9.15 BinCrossover.h . . . 137
9.16 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/Crossover.h File Reference . . . 137
9.16.1 Detailed Description . . . 137
9.17 Crossover.h . . . 138

9.18 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h File Reference . . . 138
9.18.1 Detailed Description . . . 138
9.19 ExpCrossover.h . . . 139
9.20 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/DifferentialEvolution.h File Reference . . . 139
9.20.1 Detailed Description . . . 139
9.21 DifferentialEvolution.h . . . 140
9.22 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/AllMutations.h File Reference . . . 141
9.22.1 Detailed Description . . . 141
9.23 AllMutations.h . . . 142
9.24 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best1.h File Reference . . . 142
9.24.1 Detailed Description . . . 142
9.25 Best1.h . . . 143
9.26 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best2.h File Reference . . . 143
9.26.1 Detailed Description . . . 143
9.27 Best2.h . . . 144
9.28 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Mutation.h File Reference . . . 144
9.28.1 Detailed Description . . . 145
9.29 Mutation.h . . . 145
9.30 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand1.h File Reference . . . 146
9.30.1 Detailed Description . . . 146
9.31 Rand1.h . . . 146
9.32 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand2.h File Reference . . . 147
9.32.1 Detailed Description . . . 147
9.33 Rand2.h . . . 147
9.34 src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/RandBest1.h File Reference . . . 148
9.34.1 Detailed Description . . . 148
9.35 RandBest1.h . . . 149
9.36 src/opti_py/cpp/include/opti_py/Optimizer/Evaluable.h File Reference . . . 149
9.36.1 Detailed Description . . . 150
9.37 Evaluable.h . . . 150
9.38 src/opti_py/cpp/include/opti_py/Optimizer/LocalSearch.h File Reference . . . 150
9.38.1 Detailed Description . . . 150
9.39 LocalSearch.h . . . 151
9.40 src/opti_py/cpp/include/opti_py/Optimizer/Optimizer.h File Reference . . . 151
9.40.1 Detailed Description . . . 151
9.41 Optimizer.h . . . 152

9.42 src/opti_py/cpp/include/opti_py/Optimizer/OptimizerFactory.h File Reference 152
9.42.1 Detailed Description 153
9.43 OptimizerFactory.h 153
9.44 src/opti_py/cpp/include/opti_py/Optimizer/OptResult.h File Reference 154
9.44.1 Detailed Description 154
9.45 OptResult.h 154
9.46 ParticleSwarm.h 155
9.47 src/opti_py/cpp/include/opti_py/Problem/AckleyOne.h File Reference 156
9.47.1 Detailed Description 156
9.48 AckleyOne.h 156
9.49 src/opti_py/cpp/include/opti_py/Problem/AckleyTwo.h File Reference 157
9.49.1 Detailed Description 157
9.50 AckleyTwo.h 157
9.51 DeJongOne.h 158
9.52 src/opti_py/cpp/include/opti_py/Problem/EggHolder.h File Reference 158
9.52.1 Detailed Description 158
9.53 EggHolder.h 159
9.54 src/opti_py/cpp/include/opti_py/Problem/Griewangk.h File Reference 159
9.54.1 Detailed Description 159
9.55 Griewangk.h 160
9.56 src/opti_py/cpp/include/opti_py/Problem/Problem.h File Reference 160
9.56.1 Detailed Description 160
9.57 Problem.h 161
9.58 ProblemResult.h 161
9.59 src/opti_py/cpp/include/opti_py/Problem/Rastrigin.h File Reference 162
9.59.1 Detailed Description 162
9.60 Rastrigin.h 162
9.61 src/opti_py/cpp/include/opti_py/Problem/Rosenbrock.h File Reference 163
9.61.1 Detailed Description 163
9.62 Rosenbrock.h 163
9.63 src/opti_py/cpp/include/opti_py/Problem/Schwefel.h File Reference 164
9.63.1 Detailed Description 164
9.64 Schwefel.h 164
9.65 src/opti_py/cpp/include/opti_py/Problem/SineEnvelope.h File Reference 165
9.65.1 Detailed Description 165
9.66 SineEnvelope.h 165
9.67 src/opti_py/cpp/include/opti_py/Problem/StretchedV.h File Reference 166
9.67.1 Detailed Description 166
9.68 StretchedV.h 166
9.69 src/opti_py/cpp/include/opti_py/ProblemFactory.h File Reference 167
9.69.1 Detailed Description 167
9.70 ProblemFactory.h 167
9.71 src/opti_py/cpp/include/opti_py/SolutionBuilder.h File Reference 168

9.71.1 Detailed Description 168
9.72 SolutionBuilder.h 168

Index 171

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Flow Shop Scheduling 11
Flow Shop Results 11
Optimization Algorithms 11
  Differential Evolution Optimizer 12
    DE Crossover Strategies 12
    DE Mutation Strategies 13
    DE Optimization Results 14
  Concept Definitions 14
Optimization Problems 14

Chapter 2

Concept Index

2.1 Concepts

Here is a list of all documented concepts with brief descriptions:

Evaluable
Concept defining the interface for optimization problems 15

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Crossover . . . 35
  - BinCrossover . . . 28
  - ExpCrossover . . . 45
- DeJong . . . 38
- DifferentialEvolution . . . 40
- ExperimentConfig . . . 51
- FlowShop . . . 51
- FlowShopResult . . . 61
- MersenneTwister . . . 72
- Mutation . . . 75
  - Best1 . . . 22
  - Best2 . . . 27
  - Rand1 . . . 97
  - Rand2 . . . 102
  - RandBest1 . . . 107
- Optimizer . . . 81
  - Blind . . . 33
  - LocalSearch . . . 70
  - ParticleSwarm . . . 91
- OptimizerFactory . . . 84
- OptResult . . . 85
- Problem . . . 93
  - AckleyOne . . . 19
  - AckleyTwo . . . 21
  - DeJongOne . . . 38
  - EggHolder . . . 44
  - Griewangk . . . 68
  - Rastrigin . . . 115
  - Rosenbrock . . . 117
  - Schwefel . . . 118
  - SineEnvelope . . . 120
  - StretchedV . . . 125
- ProblemFactory . . . 96
- ProblemResult . . . 97
- SolutionBuilder . . . 121

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AckleyOne
Implements the Ackley 1 benchmark function 19
AckleyTwo
Implements the Ackley 2 benchmark function 21
Best1
Implements the DE/best/1 mutation strategy 22
Best2 27
BinCrossover
Implements binomial (uniform) crossover strategy 28
Blind
Implements a blind (random walk) optimization algorithm 33
Crossover
Abstract base class for Differential Evolution crossover strategies 35
DeJong
Implements the DeJong 1 benchmark function 38
DeJongOne 38
DifferentialEvolution
Implements the Differential Evolution metaheuristic algorithm 40
EggHolder
Implements the Egg Holder benchmark function 44
ExpCrossover
Implements exponential (continuous block) crossover strategy 45
ExperimentConfig
Container for all parameters required to execute a benchmark run 51
FlowShop
Implements the classical flow shop scheduling problem 51
FlowShopResult
Encapsulates the complete output of a flow shop scheduling optimization 61
Griewangk
Implements the Griewangk benchmark function 68
LocalSearch
Implements a local search optimization algorithm 70
MersenneTwister 72
Mutation
Abstract base class for Differential Evolution mutation strategies 75

Optimizer
Abstract base class for all optimization algorithms 81
OptimizerFactory
Factory class for creating optimizer instances 84
OptResult
Encapsulates the complete output of a Differential Evolution optimization run 85
ParticleSwarm
Implements the Particle Swarm Optimization algorithm 91
Problem
Abstract base class for all optimization benchmark problems 93
ProblemFactory
Utility to create problem instances dynamically 96
ProblemResult 97
Rand1
Implements the DE/rand/1 mutation strategy 97
Rand2
Implements the DE/rand/2 mutation strategy 102
RandBest1
Implements the DE/rand-to-best/1 mutation strategy 107
Rastrigin
Implements the Rastrigin benchmark function 115
Rosenbrock
Implements the Rosenbrock benchmark function 117
Schwefel
Implements the Schwefel benchmark function 118
SineEnvelope
Implements the Sine Envelope benchmark function 120
SolutionBuilder
Responsible for creating random solutions and neighborhood samples 121
StretchedV
Implements the StretchedV benchmark function 125

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

src/opti_py/cpp/bindings.cpp
Python bindings for opti-py optimization library 127
src/opti_py/cpp/include/opti_py/debug.h 128
src/opti_py/cpp/include/opti_py/ExperimentConfig.h
Defines the configuration structure for optimization experiments 128
src/opti_py/cpp/include/opti_py/ProblemFactory.h
Factory for instantiating benchmark problems by ID 167
src/opti_py/cpp/include/opti_py/SolutionBuilder.h
Utility class for generating and manipulating candidate solutions 168
src/opti_py/cpp/include/opti_py/External/mt.h 129
src/opti_py/cpp/include/opti_py/FlowShop/FlowShop.h
Flow shop scheduling problem implementation 129
src/opti_py/cpp/include/opti_py/FlowShop/FlowShopResult.h
Result object for flow shop scheduling optimization 133
src/opti_py/cpp/include/opti_py/Optimizer/Blind.h
Header file for the Blind (Random Walk) optimization algorithm 134
src/opti_py/cpp/include/opti_py/Optimizer/Evaluable.h
Evaluable concept for optimization problem definitions 149
src/opti_py/cpp/include/opti_py/Optimizer/LocalSearch.h
Header file for the Local Search optimization algorithm 150
src/opti_py/cpp/include/opti_py/Optimizer/Optimizer.h 151
src/opti_py/cpp/include/opti_py/Optimizer/OptimizerFactory.h
Factory utility for instantiating different optimizer types 152
src/opti_py/cpp/include/opti_py/Optimizer/OptResult.h
Result object for Differential Evolution optimization 154
src/opti_py/cpp/include/opti_py/Optimizer/ParticleSwarm.h 155
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/DifferentialEvolution.h
Differential Evolution (DE) optimization algorithm implementation 139
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/AllCrossovers.h
Convenience header aggregating all DE crossover strategy implementations 135
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/BinCrossover.h
Binomial (uniform) crossover strategy implementation 136
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/Crossover.h
Base class for Differential Evolution crossover strategies 137
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h
Exponential (continuous block) crossover strategy implementation 138

src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/AllMutations.h
Convenience header aggregating all DE mutation strategy implementations 141
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best1.h
DE/best/1 mutation strategy implementation 142
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best2.h
DE/best/2 mutation strategy implementation 143
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Mutation.h
Base class for Differential Evolution mutation strategies 144
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand1.h
DE/rand/1 mutation strategy implementation 146
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand2.h
DE/rand/2 mutation strategy implementation 147
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/RandBest1.h
DE/rand-to-best/1 mutation strategy implementation 148
src/opti_py/cpp/include/opti_py/Problem/AckleyOne.h
Implementation of the Ackley 1 function 156
src/opti_py/cpp/include/opti_py/Problem/AckleyTwo.h
Implementation of the Ackley 2 function 157
src/opti_py/cpp/include/opti_py/Problem/DeJongOne.h 158
src/opti_py/cpp/include/opti_py/Problem/EggHolder.h
Implementation of the Egg Holder function 158
src/opti_py/cpp/include/opti_py/Problem/Griewangk.h
Implementation of the Griewangk function 159
src/opti_py/cpp/include/opti_py/Problem/Problem.h 160
src/opti_py/cpp/include/opti_py/Problem/ProblemResult.h 161
src/opti_py/cpp/include/opti_py/Problem/Rastrigin.h
Implementation of the Ratrigin function 162
src/opti_py/cpp/include/opti_py/Problem/Rosenbrock.h
Implementation of the Rosenbrock function 163
src/opti_py/cpp/include/opti_py/Problem/Schwefel.h
Implementation of the Schwefel function 164
src/opti_py/cpp/include/opti_py/Problem/SineEnvelope.h
Implementation of the Sine Envelope function 165
src/opti_py/cpp/include/opti_py/Problem/StretchedV.h
Implementation of the Stretched V function 166

Chapter 6

Topic Documentation

6.1 Flow Shop Scheduling

Classes

- class FlowShop

  Implements the classical flow shop scheduling problem.

6.1.1 Detailed Description

Provides a complete implementation of the classical flow shop scheduling problem with support for both traditional NEH heuristic optimization and metaheuristic optimization via Differential Evolution. Supports makespan minimization and tardiness minimization objectives, with optional job blocking constraints.

6.2 Flow Shop Results

Classes

- struct FlowShopResult

  Encapsulates the complete output of a flow shop scheduling optimization.

6.2.1 Detailed Description

Defines the FlowShopResult structure that encapsulates all outputs from flow shop scheduling optimizations (NEH heuristic or Differential Evolution). Provides complete visibility into the solution quality and schedule details.

6.3 Optimization Algorithms

Base class interface for all numerical optimization algorithms.

Topics

- Differential Evolution Optimizer
- Concept Definitions

Files

- file Blind.h

  Header file for the Blind (Random Walk) optimization algorithm.
- file LocalSearch.h

  Header file for the Local Search optimization algorithm.
- file OptimizerFactory.h

  Factory utility for instantiating different optimizer types.
- file DifferentialEvolution.h

  Differential Evolution (DE) optimization algorithm implementation.

6.3.1 Detailed Description

Base class interface for all numerical optimization algorithms.

6.3.2 Differential Evolution Optimizer

Topics

- DE Crossover Strategies
- DE Mutation Strategies
- DE Optimization Results

Classes

- class DifferentialEvolution

  Implements the Differential Evolution metaheuristic algorithm.

6.3.2.1 Detailed Description

Provides a template-based implementation of the Differential Evolution algorithm for continuous optimization problems. Supports multiple mutation and crossover strategies with OpenMP parallelization for efficient population evaluation.

6.3.2.2 DE Crossover Strategies

Files

- file AllCrossovers.h

  Convenience header aggregating all DE crossover strategy implementations.
- file BinCrossover.h

  Binomial (uniform) crossover strategy implementation.
- file ExpCrossover.h

  Exponential (continuous block) crossover strategy implementation.

Classes

- class BinCrossover

  Implements binomial (uniform) crossover strategy.
- class Crossover

  Abstract base class for Differential Evolution crossover strategies.
- class ExpCrossover

  Implements exponential (continuous block) crossover strategy.

6.3.2.2.1 Detailed Description

Defines the abstract Crossover base class and provides the common interface for all DE crossover strategy implementations.

6.3.2.3 DE Mutation Strategies

Files

- file AllMutations.h

  Convenience header aggregating all DE mutation strategy implementations.
- file Best1.h

  DE/best/1 mutation strategy implementation.
- file Best2.h

  DE/best/2 mutation strategy implementation.
- file Rand1.h

  DE/rand/1 mutation strategy implementation.
- file Rand2.h

  DE/rand/2 mutation strategy implementation.
- file RandBest1.h

  DE/rand-to-best/1 mutation strategy implementation.

Classes

- class Best1

  Implements the DE/best/1 mutation strategy.
- class Mutation

  Abstract base class for Differential Evolution mutation strategies.
- class Rand1

  Implements the DE/rand/1 mutation strategy.
- class Rand2

  Implements the DE/rand/2 mutation strategy.
- class RandBest1

  Implements the DE/rand-to-best/1 mutation strategy.

6.3.2.3.1 Detailed Description

Defines the abstract Mutation base class and provides the common interface and utilities for all DE mutation strategy implementations.

6.3.2.4 DE Optimization Results

Classes

- struct OptResult

  Encapsulates the complete output of a Differential Evolution optimization run.

6.3.2.4.1 Detailed Description

Defines the OptResult structure that encapsulates all outputs from Differential Evolution optimization runs, including the best solution found, final fitness value, and convergence history.

6.3.3 Concept Definitions

Concepts

- concept Evaluable

  Concept defining the interface for optimization problems.

Defines the Evaluable concept that all optimization problems must satisfy to be compatible with the DifferentialEvolution optimizer and other numerical optimization algorithms.

6.4 Optimization Problems

Base class and group definition for all optimization benchmark functions.

Files

- file AckleyOne.h

  Implementation of the Ackley 1 function.
- file AckleyTwo.h

  Implementation of the Ackley 2 function.
- file EggHolder.h

  Implementation of the Egg Holder function.
- file Griewangk.h

  Implementation of the Griewangk function.
- file Rastrigin.h

  Implementation of the Ratrigin function.
- file Rosenbrock.h

  Implementation of the Rosenbrock function.
- file Schwefel.h

  Implementation of the Schwefel function.
- file SineEnvelope.h

  Implementation of the Sine Envelope function.
- file StretchedV.h

  Implementation of the Stretched V function.
- file ProblemFactory.h

  Factory for instantiating benchmark problems by ID.

6.4.1 Detailed Description

Base class and group definition for all optimization benchmark functions.

Chapter 7

Concept Documentation

7.1 Evaluable Concept Reference

Concept defining the interface for optimization problems.

#include <Evaluable.h>

7.1.1 Concept definition

```
template<typename T>
concept Evaluable = requires(T t, std::vector<std::vector<double»& pop, const std::vector<double>& sol) {
    { t.getInitialSolutions(pop) } -> std::same_as<void>;
    { t.getLowerBounds() } -> std::convertible_to<double>;
    { t.getUpperBounds() } -> std::convertible_to<double>;
    { t.evaluateSolution(sol) } -> std::convertible_to<double>;
}
```

7.1.2 Detailed Description

Concept defining the interface for optimization problems.

The Evaluable concept specifies the required methods and semantics that an optimization problem must provide to be compatible with metaheuristic optimizers like Differential Evolution.

Semantic Requirements:

An Evaluable type models a continuous optimization problem over a rectangular domain [lower, upper]^n. The optimizer's task is to find the vector x = (x , x , ..., x) that minimizes the scalar objective function f(x), where each x [lower, upper].

Method Requirements:

A type T satisfies Evaluable if it provides the following methods:

1. getInitialSolutions()

   ```
   void getInitialSolutions(std::vector<std::vector<double»& population);
   ```

   - Purpose: Generate an initial population of candidate solutions
   - Parameters:

  - population: Non-const reference to a pre-allocated vector of popSize solution vectors. Each solution is a vector of doubles with dimension matching the problem's parameter space.
- Postcondition: All solutions are filled with initial candidate values. Values should be feasible (within [lower, upper]) but can employ any initialization strategy (random, Latin hypercube, user-provided, etc.).
- Example: For a 3D problem with popSize=10, population has size 10, and each population[i] has size 3.

2. getLowerBounds()

```
double getLowerBounds() const;
```

- Purpose: Return the lower bound of the feasible domain
- Returns: A scalar value L such that all problem dimensions have lower bound L.
- Semantics: The optimizer constrains all parameters to [L, upper].
- Note: Assumes a uniform bound across all dimensions (if per-dimension bounds are needed, modify this interface to return a vector).
- Example: For a problem with domain [-5, 5]^n, returns -5.0.

3. getUpperBounds()

```
double getUpperBounds() const;
```

- Purpose: Return the upper bound of the feasible domain
- Returns: A scalar value U such that all problem dimensions have upper bound U.
- Semantics: The optimizer constrains all parameters to [lower, U].
- Example: For a problem with domain [-5, 5]^n, returns 5.0.

4. evaluateSolution()

```
double evaluateSolution(const std::vector<double>& solution) const;
```

- Purpose: Evaluate the objective function at a given solution
- Parameters:
  - solution: Const reference to a solution vector (dimension matches problem).
- Returns: A scalar fitness value (double) representing the objective function.
- Semantics: Lower values are considered better (minimization problem). The optimizer seeks to minimize this value.
- Thread Safety: Must be thread-safe if the optimizer uses parallelization (e.g., OpenMP). Concurrent calls from different threads must not cause data races or undefined behavior.
- Performance: Typically the bottleneck; should be optimized for speed.
- Example: For Rosenbrock function $f(x,y) = (1-x)^2 + 100(y-x^2)^2$, evaluateSolution({x, y}) returns that value.

Example Implementation:

```
class RosenbrockProblem {
public:
    void getInitialSolutions(std::vector<std::vector<double»& population) {
        for (auto& sol : population) {
            for (auto& x : sol) {
                x = -2.0 + (rand() / (double)RAND_MAX) * 4.0;  // [-2, 2]
            }
        }
    }

    double getLowerBounds() const { return -2.0; }
    double getUpperBounds() const { return 2.0; }

    double evaluateSolution(const std::vector<double>& x) const {
        double result = 0.0;
        for (size_t i = 0; i < x.size() - 1; ++i) {
```

```
        double t1 = 1.0 - x[i];
        double t2 = x[i+1] - x[i] * x[i];
        result += t1 * t1 + 100.0 * t2 * t2;
    }
    return result;
  }
};
```

Usage with DifferentialEvolution:

```
RosenbrockProblem problem;
OptResult result = DifferentialEvolution::optimize(
  problem,
  popSize = 30,
  f = 0.8,
  cr = 0.9,
  maxGenerations = 200,
  seed = 42,
  mutationStrategy = "best1",
  crossoverStrategy = "bin"
);
```

Notes on Concept Checks:

The concept uses std::convertible_to<double> for getLowerBounds(), getUpperBounds(), and evaluate↩ Solution() return types, allowing implicit conversions (e.g., returning float or int). This provides flexibility while ensuring numeric compatibility.

The concept requires const-correctness on const methods (getters and evaluateSolution), enabling the optimizer to use const references to problems and enforce read-only semantics.

Note

> This is a C++20 concept using the <concepts> header. Ensure your compiler supports concepts and is invoked with -std=c++20 or later.

Chapter 8

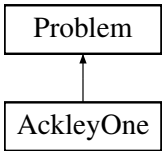
Class Documentation

8.1 AckleyOne Class Reference

Implements the Ackley 1 benchmark function.

#include <AckleyOne.h>

Inheritance diagram for AckleyOne:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.1.1 Detailed Description

Implements the Ackley 1 benchmark function.

8.1.2 Member Function Documentation

8.1.2.1 evaluate()

double AckleyOne::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

  Returns

  The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

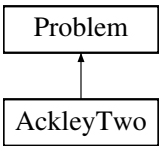
- src/opti_py/cpp/include/opti_py/Problem/AckleyOne.h

8.2 AckleyTwo Class Reference

Implements the Ackley 2 benchmark function.

#include <AckleyTwo.h>

Inheritance diagram for AckleyTwo:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.2.1 Detailed Description

Implements the Ackley 2 benchmark function.

8.2.2 Member Function Documentation

8.2.2.1 evaluate()

double AckleyTwo::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).
- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

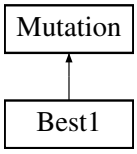
- src/opti_py/cpp/include/opti_py/Problem/AckleyTwo.h

8.3 Best1 Class Reference

Implements the DE/best/1 mutation strategy.

#include <Best1.h>

Inheritance diagram for Best1:



Public Member Functions

- std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound) override

  Generates a mutant vector using DE/best/1 strategy.

Public Member Functions inherited from Mutation

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.

Additional Inherited Members

Protected Member Functions inherited from Mutation

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.3.1 Detailed Description

Implements the DE/best/1 mutation strategy.

Strategy: DE/best/1 (Best-1)

A highly exploitative mutation strategy that biases mutations toward the best (lowest fitness) solution found so far. This creates a directed search toward known good regions while still maintaining diversity through difference vectors.

Formula:

$$v_i(t) = x_{best}(t) + F \cdot (x_{r1}(t) - x_{r2}(t))$$

Where:

- v_i: Mutant vector for target i
- x_best: Best (lowest fitness) individual in current population
- x_r1, x_r2: Two randomly selected distinct individuals (r1 i, r2 i)
- F: Differential weight controlling step size

Algorithm:

1. Select 2 random distinct individuals from population (excluding target)
2. For each dimension j:
   - mutant[j] = best[j] + F ∗ (r1[j] - r2[j])
   - Clamp mutant[j] to [lowerBound, upperBound]
3. Return the mutant vector

Characteristics:

Exploration: Very low

- Uses best solution as base, biasing search toward known good region
- Single difference vector provides minimal variation
- Risk of getting stuck in local optima

Exploitation: Very high

- Focuses search efforts in promising regions
- Excellent for fine-tuning solutions
- Rapid convergence to local optima

Robustness: Poor to moderate

- May converge prematurely on multimodal landscapes
- Good on unimodal or smooth problems
- Vulnerable to deceptive landscapes

Convergence Speed: Very fast

- Reaches good solutions quickly
- Excellent for time-limited problems
- May sacrifice final solution quality for speed

Parameter Recommendations:

- F (Differential Weight):
  - Smaller F (0.4-0.7): Smoother convergence, less risk of overshooting
  - Larger F (0.7-1.0): Stronger initial progress, may miss targets
  - Typical: 0.7-0.9
  - Note: Larger F is less critical than in Rand1 since base is already good
- CR (Crossover Rate):
  - Lower CR (0.5-0.8): More conservative, can reduce diversity
  - Higher CR (0.8-1.0): Better diversity, recommended
  - Typical: 0.9
- Population Size:
  - Minimum: 3 × dimension (needs at least 2 random individuals + best)
  - Recommended: 10-15 × dimension
  - Smaller populations acceptable (less computational cost)

Comparison with Other Strategies:

Strategy	Base	Differences	Exploration	Convergence
Rand1	Random	1	Very High	Very Slow

Strategy	Base	Differences	Exploration	Convergence
Best1	Best	1	Very Low	Very Fast
Rand2	Random	2	Extreme	Slow
Best2	Best	2	Moderate	Moderate
RandBest1	Hybrid	1	Moderate	Moderate

Use Cases:

- Unimodal or smooth optimization landscapes
- When computational budget is limited
- Fine-tuning near known solutions
- Time-critical optimization
- Problems with clear gradients toward optima
- Combining with adaptive F values for robustness

Avoid When:

- Problem is highly multimodal
- Global exploration is critical
- Multiple local optima with significantly different qualities
- Limited population diversity is a concern

Synergy with Other Components:

Best1 works best when combined with:

- High CR (0.9-1.0): Maintains diversity from difference vectors
- Moderate F (0.7-0.9): Balances convergence and stability
- Appropriate population size: Large enough to maintain diversity
- Adaptive F/CR: jDE-style adaptation helps prevent premature convergence

Implementation Details:

The implementation is straightforward:

1. Call getSubset() to randomly select r1, r2
2. Use the provided bestVector as the base
3. For each dimension, compute: best + F ∗ (r1 - r2)
4. Clamp to bounds using std::clamp
5. Return result

Complexity:

Time: O(dimension) Space: O(dimension) for the mutant vector

Example Usage:

```
// Create Best1 strategy
auto strategy = std::make_unique<Best1>();

// Generate mutant vector
std::vector<double> mutant = strategy->mutate(
    population,      // Current population
    5,               // Target individual index
    0.8,             // F = 0.8
    bestSolution,    // Best individual found
    rng,
    -1.0, 1.0        // Domain bounds
);

// Use mutant for crossover and selection
```

Typical Performance Profile:

- Rosenbrock function: Excellent (smooth landscape)
- Rastrigin function: Poor (highly multimodal)
- Sphere function: Excellent (simple unimodal)
- Schwefel function: Moderate (deceptive)

Historical Context:

DE/best/1 is one of the earliest variants developed after the original DE/rand/1. It remains widely used and is the default strategy in many DE implementations. More recent strategies like RandBest1 attempt to balance Best1's speed with Rand1's robustness.

See also

Mutation for interface documentation

Rand1 for a more exploratory variant

Best2 for higher variation

RandBest1 for a hybrid approach

8.3.2 Member Function Documentation

8.3.2.1 mutate()

```
std::vector< double > Best1::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [inline], [override], [virtual]
```

Generates a mutant vector using DE/best/1 strategy.

Applies the formula: mutant = best + F ∗ (r1 - r2) where r1 and r2 are randomly selected distinct individuals.

Parameters

in	population	Current population of solutions

in	targetIndex	Index of the target individual (excluded from selection)
in	F	Differential weight; typical range 0.5-1.0
in	bestVector	The best solution found so far (used as base)
in,out	mt	Random number generator for selection
in	lowerBound	Lower domain bound
in	upperBound	Upper domain bound

Returns

Mutant vector with all values clamped to [lowerBound, upperBound]

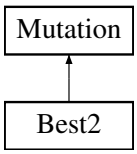
Implements Mutation.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best1.h

8.4 Best2 Class Reference

Inheritance diagram for Best2:



Public Member Functions

- std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound) override

  Generates a mutant vector using DE/best/2 strategy.

Public Member Functions inherited from Mutation

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.

Additional Inherited Members

Protected Member Functions inherited from Mutation

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.4.1 Member Function Documentation

8.4.1.1 mutate()

```
std::vector< double > Best2::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [inline], [override], [virtual]
```

Generates a mutant vector using DE/best/2 strategy.

Applies the formula: mutant = best + F ∗ (r1 - r2) + F ∗ (r3 - r4) where r1, r2, r3, r4 are four randomly selected distinct individuals.

Parameters

in	population	Current population of solutions
in	targetIndex	Index of the target individual (excluded from selection)
in	F	Differential weight; typical range 0.5-1.0
in	bestVector	The best solution found so far (used as base)
in,out	mt	Random number generator for selection
in	lowerBound	Lower domain bound
in	upperBound	Upper domain bound

Returns

Mutant vector with all values clamped to [lowerBound, upperBound]

Implements Mutation.

The documentation for this class was generated from the following file:

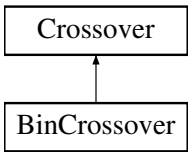
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Best2.h

8.5 BinCrossover Class Reference

Implements binomial (uniform) crossover strategy.

#include <BinCrossover.h>

Inheritance diagram for BinCrossover:



Public Member Functions

- void crossover (std::vector< double > &target, const std::vector< double > &mutant, double cr, MersenneTwister &mt) override

  Performs binomial crossover between target and mutant vectors.

Public Member Functions inherited from Crossover

- virtual ~Crossover ()=default

  Virtual destructor for safe polymorphic deletion.

8.5.1 Detailed Description

Implements binomial (uniform) crossover strategy.

Strategy: Binomial Crossover (DE/uniform/1)

The most commonly used crossover strategy. Creates trial vector by independently selecting each parameter to either inherit from mutant or retain from target. Each parameter makes an independent random decision.

Algorithm:

1. Select one random index jrand (guarantees at least one parameter from mutant)
2. For each parameter i in target:
   - If i == jrand OR random() < CR: inherit from mutant
   - Else: retain from target
3. Return via in-place modification of target

Pseudocode:

```
jrand = random_index(0, dimension-1)
for i = 0 to dimension-1:
    if (i == jrand) or (random() < CR):
        target[i] = mutant[i]
```

Characteristics:

Independence: Each parameter independently chosen

- Provides uniform mixing across all dimensions
- No spatial correlation between inherited parameters
- Symmetric treatment of all parameters

Guarantee: At least one parameter from mutant

- jrand ensures trial target (guarantees diversity)
- Prevents waste of mutant generation

Probability Distribution: Uniform

- Parameter inheritance follows binomial distribution
- Expected number of inherited parameters CR × dimension

Exploration: Moderate to High

- Higher CR → more aggressive exploration
- Lower CR → more conservative exploitation
- Symmetric mixing encourages exploration in all directions

Typical Behavior:

- CR = 0.3: Few parameters from mutant, mostly target
- CR = 0.5: Half parameters from mutant, half from target
- CR = 0.9: Most parameters from mutant, few from target

Advantages:

1. Simplicity: Straightforward to understand and implement
2. Robustness: Works well on diverse problem types
3. Default Choice: Most commonly used crossover in DE literature
4. Fast: O(dimension) complexity, minimal overhead
5. Flexible: Works with any mutation strategy

Disadvantages:

1. No Spatial Structure: Doesn't exploit problem structure
2. Independent Decisions: Can lead to disruptive mixing
3. Limited for Permutations: Less effective for permutation problems

When to Use:

- General-purpose optimization (default choice)
- Problems with no specific spatial structure
- Continuous optimization
- Unknown problem characteristics
- Need fast, robust crossover
- Combining with any mutation strategy

Parameter Recommendations:

Crossover Rate (CR):

- CR = 0.1-0.3: Conservative mixing, good with Best1 mutation
- CR = 0.5-0.7: Balanced mixing, practical default
- CR = 0.8-0.95: Aggressive mixing, good with Rand1 mutation
- CR = 1.0: All parameters from mutant (full replacement)
- Adaptive CR: Increase CR over time (jDE style) for convergence

Synergy with Mutations:

- With Rand1: Use higher CR (0.8-0.95) for exploration
- With Best1: Use lower CR (0.3-0.7) to stabilize around best
- With RandBest1: Use medium CR (0.7-0.9) for balance
- With Rand2/Best2: Use lower CR due to high mutation variation

Implementation Details:

The implementation uses:

1. mt.genrand_int32() % target.size() for random index
2. mt.genrand_real1() < cr for probability check
3. In-place modification of target
4. Loop over all dimensions

Complexity:

Time: O(dimension) - must examine each parameter Space: O(1) - modifies target in-place

Example Usage:

```
// Create binomial crossover
auto crossover = std::make_unique<BinCrossover>();

// Apply crossover (modifies target in-place)
std::vector<double> target = population[5];  // Current solution
std::vector<double> mutant = mutation->mutate(...);  // Mutation result
crossover->crossover(target, mutant, 0.9, rng);
// Now target is the trial vector
```

Typical Performance Profile:

- Rosenbrock function: Excellent (default choice)
- Rastrigin function: Good (general purpose)
- Sphere function: Excellent (simple landscape)
- Schwefel function: Good (deceptive landscape)
- CEC Benchmark: Good (practical choice)

Historical Context:

Binomial crossover was introduced in the original DE paper by Storn and Price (1997) and remains the most widely used crossover operator. Its simplicity, robustness, and effectiveness across problem types have made it the default choice in most DE implementations.

Comparison with Exponential Crossover:

Aspect	Binomial	Exponential
Selection	Independent each parameter	Contiguous block

Aspect	Binomial	Exponential
Spatial Bias	None	Weak (sequential)
Exploration	Uniform	Structured
Implementation	O(dimension) loop	O(inherited count) loop
Default	Yes (most common)	No (specialized)
Robustness	Excellent	Good
Best For	General purpose	Ordered/structured problems

See also

Crossover for interface documentation

ExpCrossover for continuous/exponential alternative

8.5.2 Member Function Documentation

8.5.2.1 crossover()

```
void BinCrossover::crossover (
            std::vector< double > & target,
            const std::vector< double > & mutant,
            double cr,
            MersenneTwister & mt)   [inline], [override], [virtual]
```

Performs binomial crossover between target and mutant vectors.

Independently selects each parameter to inherit from mutant with probability CR. Guarantees at least one parameter from mutant via jrand.

Parameters

in,out	target	Target vector, modified in-place to become trial
in	mutant	Mutant vector as crossover source
in	cr	Crossover rate; typical range 0.1-0.95
in,out	mt	Random number generator

Postcondition

target is modified to be the trial vector

target[jrand] always came from mutant

target[i] came from mutant with probability CR for i != jrand

Implements Crossover.

The documentation for this class was generated from the following file:

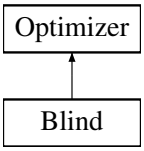
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/BinCrossover.h

8.6 Blind Class Reference

Implements a blind (random walk) optimization algorithm.

#include <Blind.h>

Inheritance diagram for Blind:



Public Member Functions

- Blind (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations)

  Constructs a Blind optimizer.
- std::vector< double > optimize () override

  Executes the blind optimization process.

Public Member Functions inherited from Optimizer

- Optimizer (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations)

  Constructs an Optimizer.
- virtual ~Optimizer ()=default

  Virtual destructor for safe polymorphic deletion.

- double getBestFitness ()
- std::vector< double > & getBestSolution ()
- std::vector< double > & getBestFitnesses ()
- std::vector< std::vector< double > > & getSolutions ()
- int getMaxIterations ()
- std::shared_ptr< Problem > getProblem ()
- std::shared_ptr< SolutionBuilder > getSolutionBuilder ()

Additional Inherited Members

Protected Attributes inherited from Optimizer

- std::shared_ptr< Problem > problem

  Optimization problem definition.
- std::shared_ptr< SolutionBuilder > solutionBuilder

  Solution generation utility.
- int maxIterations

  Maximum number of iterations.
- std::vector< double > bestSolution

  Best solution found.
- std::vector< double > bestFitnesses

  Best fitness value so far per iteration.
- std::vector< std::vector< double > > solutions

  All generated solutions.

8.6.1 Detailed Description

Implements a blind (random walk) optimization algorithm.

The Blind optimizer repeatedly samples random solutions from the solution space without using neighborhood information or gradient guidance. It keeps track of the best solution found across all iterations.

This algorithm serves as a baseline for comparison against more informed local search techniques.

8.6.2 Constructor & Destructor Documentation

8.6.2.1 Blind()

```
Blind::Blind (
            std::shared_ptr< SolutionBuilder > solutionBuilder,
            std::shared_ptr< Problem > problem,
            int maxIterations)   [inline]
```

Constructs a Blind optimizer.

Parameters

solutionBuilder	Reference to the solution generator.
problem	Reference to the optimization problem.
maxIterations	Maximum number of iterations to perform.

8.6.3 Member Function Documentation

8.6.3.1 optimize()

```
std::vector< double > Blind::optimize ()   [override], [virtual]
```

Executes the blind optimization process.

Returns

Execution time of the algorithm.

Implements Optimizer.

The documentation for this class was generated from the following files:

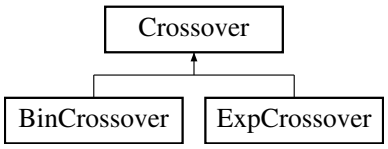
- src/opti_py/cpp/include/opti_py/Optimizer/Blind.h
- src/opti_py/cpp/src/Optimizer/Blind.cpp

8.7 Crossover Class Reference

Abstract base class for Differential Evolution crossover strategies.

#include <Crossover.h>

Inheritance diagram for Crossover:



Public Member Functions

- virtual ~Crossover ()=default

  Virtual destructor for safe polymorphic deletion.
- virtual void crossover (std::vector< double > &target, const std::vector< double > &mutant, double cr, MersenneTwister &mt)=0

  Performs crossover between target and mutant vectors.

8.7.1 Detailed Description

Abstract base class for Differential Evolution crossover strategies.

Crossover defines the interface for all DE crossover operators. Each concrete crossover strategy (BinCrossover, ExpCrossover) implements the crossover() method to recombine a target vector and mutant vector, creating a trial vector for evaluation.

Crossover Role in DE:

In Differential Evolution, crossover blends the target and mutant vectors:

1. Start with target vector from current population
2. Mix in parameters from mutant vector with probability CR
3. Create trial vector for fitness evaluation
4. Selection step compares trial vs target

Purpose of Crossover:

- Diversity Control: Crossover rate (CR) controls how aggressively new variation enters the population
- Inheritance: Selects which parameters to inherit from mutant
- Exploration-Exploitation Balance: Works with mutation (F) to control overall search behavior

Crossover Rate (CR) Semantics:

CR is a probability in range [0, 1]:

- CR = 0.0: No crossover, trial = target (no change)
- CR = 0.5: Each parameter has 50% chance to come from mutant
- CR = 1.0: All parameters from mutant (full replacement)
- Typical: 0.5-0.9 depending on problem

Crossover vs Mutation:

The pair (F, CR) controls search behavior:

- Mutation (F): Determines magnitude of variation in mutant
- Crossover (CR): Determines how much variation enters population
- Interaction: High F + High CR = aggressive exploration Low F + Low CR = conservative exploitation

Strategy Comparison:

Two main strategies:

- Binomial (Uniform): Each parameter independently inherited
- Exponential (Continuous): Parameters inherited in contiguous blocks

See also

BinCrossover for binomial (independent) crossover

ExpCrossover for exponential (continuous) crossover

8.7.2 Member Function Documentation

8.7.2.1 crossover()

```
virtual void Crossover::crossover (
            std::vector< double > & target,
            const std::vector< double > & mutant,
            double cr,
            MersenneTwister & mt)   [pure virtual]
```

Performs crossover between target and mutant vectors.

Pure virtual method that each concrete crossover strategy implements to recombine a target vector and mutant vector. The target vector is modified in-place to become a trial vector.

Algorithm Pattern:

All crossover strategies follow this general workflow:

1. Select at least one index jrand (ensures at least one parameter from mutant)
2. For each parameter in target:
   - With probability CR, inherit from mutant
   - Otherwise, keep current value from target
3. Parameter at jrand is always inherited (guarantees diversity)
4. Return via in-place modification of target

In-Place Modification:

The target vector is modified directly, replacing its contents with the trial vector. This is efficient (no copy) and standard in DE.

Guarantees:

- Target vector is always modified (in-place)
- Trial vector dimension equals target and mutant dimension
- At least one parameter comes from mutant (jrand index)
- All other parameters inherit with probability CR

Parameters:

Parameters

in,out	target	The target (parent) vector from current population. Modified in-place to become the trial vector. Size: equals mutant.size() and problem dimension.
in	mutant	The mutant vector produced by mutation strategy. Used as source for inherited parameters. Size: equals target.size().
in	cr	Crossover rate (probability of inheritance). Range: [0, 1]. • cr = 0: Very few parameters from mutant • cr = 0.5: About half parameters from mutant • cr = 1.0: All parameters from mutant Typical values: 0.5-0.9.
in,out	mt	Mersenne Twister random number generator. Passed by reference; state is modified during execution. Used to randomly select parameters to inherit.

Strategy-Specific Behavior:

Different concrete strategies implement different inheritance patterns:

- Binomial: Each parameter independently chosen with probability CR
- Exponential: Parameters inherited in contiguous blocks

Postconditions:

After crossover completes:

- target vector contains trial vector (modified in-place)
- At least one parameter came from mutant (at index jrand)
- Other parameters inherited with probability CR
- target.size() unchanged

Usage in DE:

```
std::vector<double> target = population[i];  // Current solution
std::vector<double> mutant = mutation->mutate(...);  // Mutation step
crossover->crossover(target, mutant, 0.9, rng);  // Crossover step
// Now target is the trial vector, evaluate fitness
double trial_fitness = problem.evaluate(target);
```

Complexity:

Time: O(dimension) - must examine each parameter Space: O(1) - modifies target in-place

See also

BinCrossover for independent parameter selection
ExpCrossover for contiguous block selection

Implemented in BinCrossover, and ExpCrossover.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/Crossover.h

8.8 DeJong Class Reference

Implements the DeJong 1 benchmark function.

#include <DeJongOne.h>

8.8.1 Detailed Description

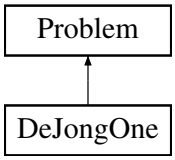
Implements the DeJong 1 benchmark function.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Problem/DeJongOne.h

8.9 DeJongOne Class Reference

Inheritance diagram for DeJongOne:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.9.1 Member Function Documentation

8.9.1.1 evaluate()

double DeJongOne::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Problem/DeJongOne.h

8.10 DifferentialEvolution Class Reference

Implements the Differential Evolution metaheuristic algorithm.

#include <DifferentialEvolution.h>

Public Member Functions

- DifferentialEvolution ()=default
  Default constructor (implicitly deleted for static utility class).
- ~DifferentialEvolution ()=default
  Default destructor.

Static Public Member Functions

Factory Methods

- static std::unique_ptr< Mutation > createMutation (const std::string &name)
  Creates a mutation strategy object by name.
- static std::unique_ptr< Crossover > createCrossover (const std::string &name)
  Creates a crossover strategy object by name.

Core Optimization

- static void crossover (std::vector< double > &target, const std::vector< double > &mutant, double cr, MersenneTwister &mt)
  Performs binomial crossover between target and mutant vectors.
- template<Evaluable Problem>
  static OptResult optimize (Problem &problem, size_t popSize, double f, double cr, size_t max↩ Generations, unsigned long seed, std::string &mutationStrategy, std::string &crossoverStrategy)
  Runs the Differential Evolution optimization algorithm.

8.10.1 Detailed Description

Implements the Differential Evolution metaheuristic algorithm.

DifferentialEvolution provides a genetic algorithm-inspired approach to continuous optimization. The algorithm maintains a population of candidate solutions and iteratively evolves them through mutation and crossover operations guided by fitness-based selection.

Key features:

- Support for multiple mutation strategies (rand1, rand2, best1, best2, randToBest1)
- Support for multiple crossover strategies (binomial, exponential)
- OpenMP parallelization for population-level operations
- Template-based design supporting arbitrary Evaluable problem types
- Deterministic seeding for reproducible results

Note

All methods are static; this class serves as a namespace for DE-related functionality.

8.10.2 Member Function Documentation

8.10.2.1 createCrossover()

std::unique_ptr< Crossover > DifferentialEvolution::createCrossover (
const std::string & name) [inline], [static]

Creates a crossover strategy object by name.

Supported crossover strategies:

- "bin": Binomial (uniform) crossover - each parameter inherited independently
- "exp": Exponential crossover - contiguous parameter blocks inherited

Parameters

name	String identifier for the crossover strategy

Returns

std::unique_ptr<Crossover> Heap-allocated crossover strategy object

Exceptions

std::runtime_error	if the strategy name is not recognized

8.10.2.2 createMutation()

std::unique_ptr< Mutation > DifferentialEvolution::createMutation (
const std::string & name) [inline], [static]

Creates a mutation strategy object by name.

Supported mutation strategies:

- "rand1": DE/rand/1 - Random base with one difference vector
- "rand2": DE/rand/2 - Random base with two difference vectors
- "best1": DE/best/1 - Best base with one difference vector
- "best2": DE/best/2 - Best base with two difference vectors
- "randToBest1": DE/rand-to-best/1 - Interpolation between rand and best

Parameters

name	String identifier for the mutation strategy

Returns

std::unique_ptr<Mutation> Heap-allocated mutation strategy object

Exceptions

std::runtime_error	if the strategy name is not recognized

8.10.2.3 crossover()

```
void DifferentialEvolution::crossover (
            std::vector< double > & target,
            const std::vector< double > & mutant,
            double cr,
            MersenneTwister & mt)    [inline], [static]
```

Performs binomial crossover between target and mutant vectors.

In binomial crossover, each parameter of the trial vector is inherited from the mutant vector with probability cr, or from the target vector with probability (1 - cr). At least one parameter is always inherited from the mutant to ensure diversity.

Parameters

in,out	target	The target (parent) vector to be modified in-place
in	mutant	The mutant vector used for crossover
in	cr	Crossover rate in range [0, 1]; controls inheritance probability
in,out	mt	Mersenne Twister random number generator

Postcondition

target is modified with parameters from mutant according to cr

8.10.2.4 optimize()

```
template<Evaluable Problem>
OptResult DifferentialEvolution::optimize (
            Problem & problem,
            size_t popSize,
            double f,
            double cr,
            size_t maxGenerations,
            unsigned long seed,
            std::string & mutationStrategy,
            std::string & crossoverStrategy)    [static]
```

Runs the Differential Evolution optimization algorithm.

Executes the complete DE optimization loop:

1. Initializes a population using the problem's initial solution generator
2. Evaluates initial population fitness
3. For each generation:
    - Applies mutation strategy to generate mutant vectors
    - Applies crossover strategy to generate trial vectors
    - Evaluates trial vector fitness
    - Selects survivors (greedy selection based on fitness)

- Tracks global best solution

4. Returns best solution found and convergence history

Thread Safety:

- Population evaluation is parallelized with OpenMP
- Global best tracking uses critical sections to prevent data races
- Each thread maintains thread-local best fitness to minimize synchronization

Template Parameters

Problem	Must satisfy the Evaluable concept with methods: • getInitialSolutions(std::vector<std::vector<double>>&) • evaluateSolution(const std::vector<double>&) -> double • getLowerBounds() -> double • getUpperBounds() -> double

Parameters

in,out	problem	The optimization problem object
in	popSize	Population size; must be > 0
in	f	Differential weight (mutation scaling factor) in range (0, 2]
in	cr	Crossover rate in range [0, 1]
in	maxGenerations	Maximum number of generations to evolve
in	seed	Seed value for random number generation; enables reproducibility
in,out	mutationStrategy	Name of mutation strategy; defaults to "rand1" if empty
in,out	crossoverStrategy	Name of crossover strategy; defaults to "bin" if empty

Returns

OptResult struct containing:

- bestSolution: best vector found
- bestFitness: fitness value of best solution
- bestFitnesses: convergence history (best fitness per generation)

Note

RNG seeding ensures deterministic results: distinct seeds are used for mutation and crossover at each (generation, individual) pair to prevent state contamination while maintaining reproducibility.

Returns empty OptResult if popSize == 0.

The documentation for this class was generated from the following file:

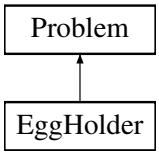
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/DifferentialEvolution.h

8.11 EggHolder Class Reference

Implements the Egg Holder benchmark function.

#include <EggHolder.h>

Inheritance diagram for EggHolder:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.11.1 Detailed Description

Implements the Egg Holder benchmark function.

8.11.2 Member Function Documentation

8.11.2.1 evaluate()

double EggHolder::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).
- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

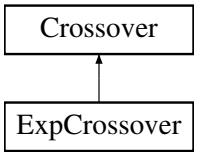
- src/opti_py/cpp/include/opti_py/Problem/EggHolder.h

8.12 ExpCrossover Class Reference

Implements exponential (continuous block) crossover strategy.

#include <ExpCrossover.h>

Inheritance diagram for ExpCrossover:



Public Member Functions

- void crossover (std::vector< double > &target, const std::vector< double > &mutant, double cr, MersenneTwister &mt) override

  Performs exponential crossover between target and mutant vectors.

Public Member Functions inherited from Crossover

- virtual ~Crossover ()=default

  Virtual destructor for safe polymorphic deletion.

8.12.1 Detailed Description

Implements exponential (continuous block) crossover strategy.

Strategy: Exponential Crossover

Inherits parameters from the mutant in contiguous blocks rather than independent selections. Creates a structured, sequential inheritance pattern that can be beneficial for problems with spatial correlations.

Algorithm:

1. Select one random starting index jrand
2. Starting from jrand, inherit parameters sequentially
3. Continue inheriting while: random() < CR AND haven't inherited all dimensions
4. Wrap around dimensions using modulo (circular)
5. Return via in-place modification of target

Pseudocode:

```
jrand = random_index(0, dimension-1)
left = 0
do:
    idx = (jrand + left) % dimension
    target[idx] = mutant[idx]
    left++
while (random() < CR AND left < dimension)
```

Characteristics:

Block Inheritance: Parameters inherited sequentially

- Creates contiguous blocks of inherited parameters
- Preserves spatial structure from mutant
- Can exploit ordered problem structure

Circular Wrapping: Wraps around at dimension boundaries

- Uses modulo arithmetic for wraparound
- Treats problem as circular/periodic
- Fair treatment of all dimensions

Continuous Probability: CR controls block length distribution

- Block length follows geometric distribution with parameter CR
- Expected block length 1 / (1 - CR)

- Higher CR → longer blocks → fewer transitions
- Lower CR → shorter blocks → more transitions

Exploration: Moderate, depends on CR and block structure

- CR = 0.3: Very short blocks, many transitions
- CR = 0.5: Medium blocks, balanced structure
- CR = 0.9: Long blocks, few transitions

Block Length Distribution:

The probability of inheriting exactly k parameters follows: P(k) = (1 - CR)^(k-1) × CR

Examples:

- CR = 0.5: P(block of 1) = 0.5, P(block of 2) = 0.25, P(block of 3) = 0.125, ...
- CR = 0.8: P(block of 1) = 0.2, P(block of 2) = 0.16, P(block of 3) = 0.128, ...

Advantages:

1. Structure Preservation: Maintains spatial correlations
2. Problem-Specific: Good for ordered/sequential parameters
3. Linked Inheritance: Related parameters inherited together
4. Lower Disruption: Fewer independent changes
5. Variable Block Sizes: Geometric distribution provides variation

Disadvantages:

1. Bias: Block starting position creates preference
2. Less General: May perform worse on unstructured problems
3. Order-Dependent: Sensitive to parameter ordering
4. Complex Behavior: Block structure less intuitive than binomial
5. Slower Selection: Loop continues until CR condition fails

When to Use:

- Problems with spatial/sequential parameter structure
- Linked parameters that should co-evolve
- Gene grouping problems (epistasis)
- Problems where parameter order is meaningful
- When binomial crossover causes disruptive mixing

- Neural network weight optimization
- Sensor/coordinate problems (x, y, z groupings)
- Avoiding mutation-like disruption from independent selection

Avoid When:

- Problem is unstructured (use binomial)
- Parameters are independent (use binomial)
- Parameter ordering is arbitrary
- Need maximum generality (use binomial)
- Problem structure is unknown

Parameter Recommendations:

Crossover Rate (CR):

- CR = 0.1-0.3: Very short blocks, high disruption
- CR = 0.5-0.7: Medium blocks, balanced structure
- CR = 0.8-0.95: Long blocks, less disruption
- Adaptive CR: Can adapt block length over optimization
- Problem-Dependent: Choose based on parameter correlations

Synergy with Mutations:

- With Rand1: Use medium-high CR (0.7-0.9) for structured exploration
- With Best1: Use lower CR (0.3-0.7) to maintain structure around best
- With RandBest1: Use medium CR (0.6-0.85) for balanced structure
- With Rand2/Best2: Use lower CR due to high mutation variation

Implementation Details:

The implementation uses:

1. mt.genrand_int32() % target.size() for random starting index
2. (jrand + left) % target.size() for circular indexing
3. mt.genrand_real1() < cr for continuation probability
4. In-place modification of target
5. Do-while loop for block inheritance

Complexity:

Time: O(inherited_count) where inherited_count is variable (0 to dimension) Expected: O(CR × dimension) Space: O(1) - modifies target in-place

Example Usage:

```
// Create exponential crossover
auto crossover = std::make_unique<ExpCrossover>();

// Apply crossover (modifies target in-place)
std::vector<double> target = population[5];  // Current solution
std::vector<double> mutant = mutation->mutate(...);  // Mutation result
crossover->crossover(target, mutant, 0.8, rng);
// Now target is the trial vector with contiguous blocks from mutant
```

Example Inheritance Patterns:

With 8-dimensional problem, CR=0.7, jrand=2:

- Iteration 1: idx=2, inherit (random < 0.7), left=1
- Iteration 2: idx=3, inherit (random < 0.7), left=2
- Iteration 3: idx=4, inherit (random < 0.7), left=3
- Iteration 4: idx=5, DON'T inherit (random 0.7), stop Result: target[2:5] from mutant, rest retained

Typical Performance Profile:

- Rosenbrock function: Good (structure-aware)
- Rastrigin function: Moderate (less general)
- Sphere function: Good (simple landscape)
- Schwefel function: Moderate (deceptive)
- Ordered Parameter Problem: Excellent (best choice)
- General CEC Benchmark: Moderate (less robust than binomial)

Historical Context:

Exponential crossover was introduced in the original DE paper by Storn and Price (1997) as an alternative to binomial crossover. While less commonly used than binomial, it provides value for problems with inherent parameter structure.

Comparison with Binomial Crossover:

Aspect	Binomial	Exponential
Selection	Independent each parameter	Contiguous block
Spatial Bias	None	Weak (sequential)
Exploration	Uniform	Structured
Block Length	Poisson-like (each parameter)	Geometric (contiguous)
Implementation	O(dimension) loop	O(inherited count) loop

Aspect	Binomial	Exponential
Default	Yes (most common)	No (specialized)
Robustness	Excellent (general)	Good (specialized)
Best For	General purpose	Ordered/structured problems

Tuning Guide:

If standard binomial isn't working well:

1. Check if parameters have spatial/sequential meaning
2. If yes, try exponential with CR = 0.5-0.8
3. Adjust CR based on expected block size
4. Monitor if blocks help or hurt fitness progress
5. Consider adaptive CR that evolves block sizes

See also

Crossover for interface documentation
BinCrossover for independent/uniform alternative

8.12.2 Member Function Documentation

8.12.2.1 crossover()

```
void ExpCrossover::crossover (
            std::vector< double > & target,
            const std::vector< double > & mutant,
            double cr,
            MersenneTwister & mt)   [inline], [override], [virtual]
```

Performs exponential crossover between target and mutant vectors.

Inherits parameters in contiguous blocks from mutant starting at random index jrand. Block length determined by CR probability.

Parameters

in,out	target	Target vector, modified in-place to become trial
in	mutant	Mutant vector as crossover source
in	cr	Crossover rate controlling block length; typical range 0.1-0.95
in,out	mt	Random number generator

Postcondition

target is modified to be the trial vector
target contains one or more contiguous blocks from mutant
block(s) start at jrand and wrap around circularly
expected number of inherited parameters $\approx$ cr $\times$ size when cr $<$ 1.0

Implements Crossover.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h

8.13 ExperimentConfig Struct Reference

Container for all parameters required to execute a benchmark run.

#include <ExperimentConfig.h>

Public Member Functions

- ExperimentConfig (int problemType, int dimensions, double lower, double upper, int seed=108664, int maxIterations=30)

Public Attributes

- int problemType

  ID of the benchmark problem to instantiate.
- int dimensions

  Number of dimensions for the problem space.
- double lower

  Global lower bound.
- double upper

  Global upper bound.
- int seed

  Pseudo-random number generator seed for reproducibility.
- int maxIterations

  Termination criteria: maximum evaluation cycles.

8.13.1 Detailed Description

Container for all parameters required to execute a benchmark run.

- This structure is typically populated by ExperimentRunner from JSON and passed to factories and runners to initialize the experiment state.

The documentation for this struct was generated from the following file:

- src/opti_py/cpp/include/opti_py/ExperimentConfig.h

8.14 FlowShop Class Reference

Implements the classical flow shop scheduling problem.

#include <FlowShop.h>

Public Member Functions

Constructors

- FlowShop ()
  Constructs a FlowShop problem with zero dimensions.
- FlowShop (size_t num_jobs, size_t num_machines)
  Constructs a FlowShop problem with specified dimensions.
- FlowShop (size_t num_jobs, size_t num_machines, std::vector< uint64_t > &&data)
  Internal constructor used by pybind11 bindings.

Data Access

- size_t num_jobs () const
  Returns the number of jobs in this problem.
- size_t num_machines () const
  Returns the number of machines in this problem.
- uint64_t * data ()
  Returns mutable pointer to flattened processing times matrix.
- const uint64_t * data () const
  Returns const pointer to flattened processing times matrix.
- uint64_t * due_dates_data ()
  Returns mutable pointer to due dates vector.
- const uint64_t * due_dates_data () const
  Returns const pointer to due dates vector.

Problem Configuration

- void set_jobs (size_t num_jobs, size_t num_machines, std::vector< uint64_t > &&new_data)
  Replaces the entire processing times matrix and dimensions.
- void set_due_dates (std::vector< uint64_t > &&new_data)
  Replaces the due dates vector.

Optimization Methods

- FlowShopResult runNEH (bool blocking=false, bool optimizeTardiness=false)
  Optimizes the flow shop schedule using the NEH heuristic.
- FlowShopResult runDE (bool blocking, bool optimizeTardiness, size_t popSize, double f, double cr, size_t maxGenerations, unsigned long seed, std::string &mutationStrategy, std::string &crossoverStrategy)
  Optimizes the flow shop schedule using Differential Evolution.

Evaluable Concept Implementation

- void getInitialSolutions (std::vector< std::vector< double > > &population)
  Generates an initial population of candidate solutions.
- constexpr double getLowerBounds () const
  Returns the lower bound for the optimization domain.
- constexpr double getUpperBounds () const
  Returns the upper bound for the optimization domain.
- double evaluateSolution (const std::vector< double > &solution)
  Evaluates the objective function for a given solution.

8.14.1 Detailed Description

Implements the classical flow shop scheduling problem.

The FlowShop class models a manufacturing environment where jobs must be processed sequentially through a series of machines in a fixed order. The goal is to find an optimal job scheduling order that minimizes either makespan (total completion time) or total tardiness (sum of lateness over all jobs).

Problem Definition:

Given:

- n jobs, each with execution times on m machines
- Optional due dates for each job
- Two objective functions: makespan or total tardiness
- Optional blocking constraint (job cannot leave a machine until next is free)

Find: A permutation of jobs that minimizes the chosen objective.

Key Features:

- Dual optimization support: NEH heuristic (fast approximation) and DE (high-quality)
- Two objective metrics: makespan minimization and tardiness minimization
- Optional job blocking for realistic manufacturing constraints
- Intelligent population initialization (NEH seed + perturbations + random)
- SPV (Smallest Position Value) encoding for seamless DE integration
- Thread-safe evaluation with OpenMP parallelization

Problem Data:

- jobs_times_: m×n matrix (row-major flattened) storing processing times. jobs_times_[i ∗ num↩ _machines + j] = time for job i on machine j.
- due_dates_: Vector of n due dates, one per job (for tardiness calculation).

Evaluable Concept:

FlowShop satisfies the Evaluable concept, allowing it to be optimized directly by DifferentialEvolution:

- getInitialSolutions(): Generates population with NEH seeding
- evaluateSolution(): Converts SPV vector to job order and evaluates schedule
- getLowerBounds()/getUpperBounds(): Returns [-1.0, 1.0] for SPV encoding

Note

The problem assumes uniform bounds across dimensions (SPV encoding).

All evaluation is deterministic once scheduling parameters are set (blocking, tardiness).

8.14.2 Constructor & Destructor Documentation

8.14.2.1 FlowShop() [1/3]

FlowShop::FlowShop () [inline]

Constructs a FlowShop problem with zero dimensions.

Creates an empty problem instance. Data must be populated via set_jobs() before optimization.

8.14.2.2 FlowShop() [2/3]

```
FlowShop::FlowShop (
            size_t num_jobs,
            size_t num_machines)   [inline]
```

Constructs a FlowShop problem with specified dimensions.

Allocates storage for a problem with num_jobs jobs and num_machines machines. All processing times initialize to zero; call set_jobs() to populate actual data.

Parameters

num_jobs	Number of jobs in the scheduling problem
num_machines	Number of machines in sequence

Exceptions

std::runtime_error	if num_jobs or num_machines is zero

8.14.2.3 FlowShop() [3/3]

```
FlowShop::FlowShop (
            size_t num_jobs,
            size_t num_machines,
            std::vector< uint64_t > && data)   [inline]
```

Internal constructor used by pybind11 bindings.

Constructs a FlowShop with pre-allocated data moved from caller. Validates that data size matches the dimensions (num_jobs × num_machines).

Parameters

num_jobs	Number of jobs
num_machines	Number of machines
data	Rvalue reference to flattened m×n matrix (row-major)

Exceptions

std::runtime_error	if data.size() != num_jobs ∗ num_machines

Note

This constructor is primarily for Python interoperability; use other constructors for C++ code unless specifically integrating with pybind11.

8.14.3 Member Function Documentation

8.14.3.1 data() [1/2]

uint64_t * FlowShop::data () [inline]

Returns mutable pointer to flattened processing times matrix.

The matrix is stored in row-major order: jobs_times[i * num_machines + j] contains the processing time of job i on machine j.

Returns

Pointer to first element of jobs_times_ vector

8.14.3.2 data() [2/2]

const uint64_t * FlowShop::data () const [inline]

Returns const pointer to flattened processing times matrix.

Returns

Const pointer to first element of jobs_times_ vector

See also

data()

8.14.3.3 due_dates_data() [1/2]

uint64_t * FlowShop::due_dates_data () [inline]

Returns mutable pointer to due dates vector.

The vector has size num_jobs; due_dates_[i] is the due date for job i. Used for tardiness calculation: tardiness = max(0, completion_time - due_date).

Returns

Pointer to first element of due_dates_ vector

8.14.3.4 due_dates_data() [2/2]

const uint64_t * FlowShop::due_dates_data () const [inline]

Returns const pointer to due dates vector.

Returns

Const pointer to first element of due_dates_ vector

See also

due_dates_data()

8.14.3.5 evaluateSolution()

double FlowShop::evaluateSolution (
const std::vector< double > & solution)

Evaluates the objective function for a given solution.

Converts an SPV-encoded vector to a job permutation, evaluates the resulting schedule, and returns the objective value (makespan or tardiness).

Algorithm:

1. Convert SPV vector to job order via spvToPerm()
2. Calculate completion times for the schedule
3. Return makespan (if optimizeTardiness=false) or total tardiness (if true)

Complexity: O(nm) for completion time calculation + O(n log n) for SPV conversion.

Parameters

in	solution	SPV-encoded solution vector (size = num_jobs, values in [-1, 1])

Returns

Objective value:

- Makespan (completion time of last job) if optimizeTardiness=false
- Total tardiness (sum of individual tardiness) if optimizeTardiness=true Returned as double for compatibility with DE optimizer.

Thread Safety: This method is thread-safe for concurrent evaluation when blocking and optimizeTardiness fields are constant. Must not be called concurrently with runDE() or field modifications.

Requirements: blocking and optimizeTardiness must be set before calling (typically by runDE() prior to optimizer invocation).

8.14.3.6 getInitialSolutions()

```
void FlowShop::getInitialSolutions (
            std::vector< std::vector< double > > & population)
```

Generates an initial population of candidate solutions.

Creates population.size() SPV-encoded solutions using three strategies:

1. First solution: NEH heuristic schedule
2. Next 25%: NEH with small random perturbations
3. Remaining: Uniformly random in [-1, 1]^n

This initialization biases the population toward high-quality regions (NEH solutions) while maintaining diversity for exploration.

Parameters

out	population	Pre-allocated vector of population.size() solution vectors. Each solution is filled with SPV values in [-1, 1].

Thread Safety: Uses OpenMP parallelization to generate initial solutions quickly.

Note

This method requires blocking and optimizeTardiness fields to be set (typically by runDE() before calling DifferentialEvolution::optimize).

See also

initPopulationVectors()

8.14.3.7 getLowerBounds()

double FlowShop::getLowerBounds () const [inline], [constexpr]

Returns the lower bound for the optimization domain.

For SPV encoding, all dimensions share a uniform lower bound of -1.0.

Returns

Constant value -1.0

8.14.3.8 getUpperBounds()

double FlowShop::getUpperBounds () const [inline], [constexpr]

Returns the upper bound for the optimization domain.

For SPV encoding, all dimensions share a uniform upper bound of 1.0.

Returns

Constant value 1.0

8.14.3.9 num_jobs()

size_t FlowShop::num_jobs () const [inline]

Returns the number of jobs in this problem.

Returns

Number of jobs (rows in processing time matrix)

8.14.3.10 num_machines()

size_t FlowShop::num_machines () const [inline]

Returns the number of machines in this problem.

Returns

Number of machines (columns in processing time matrix)

8.14.3.11 runDE()

```
FlowShopResult FlowShop::runDE (
            bool blocking,
            bool optimizeTardiness,
            size_t popSize,
            double f,
            double cr,
            size_t maxGenerations,
            unsigned long seed,
            std::string & mutationStrategy,
            std::string & crossoverStrategy)
```

Optimizes the flow shop schedule using Differential Evolution.

Applies the DE metaheuristic to find high-quality solutions. The algorithm:

1. Initializes population with NEH seed, perturbations, and random solutions

2. Iterates over generations, applying mutation and crossover
3. Evaluates each candidate by converting SPV encoding to job order
4. Returns best solution found and convergence history

Solution Encoding: SPV (Smallest Position Value)

- Continuous vector in [-1, 1]^n representing job priorities
- Higher values = earlier scheduling position
- Enables DE's continuous operators on permutation problem

Population Initialization:

- 1 solution: NEH-generated schedule (converted to SPV)
- 25% of population: NEH with small random perturbations
- Remaining: Uniformly random solutions in [-1, 1]^n

Parameters

blocking	If true, enforces job blocking constraints
optimizeTardiness	If true, minimizes tardiness; if false, minimizes makespan
popSize	Population size; typically 20-50 for small problems
f	Differential weight (mutation scale factor); typical range (0, 2], e.g., 0.8
cr	Crossover rate; range [0, 1], e.g., 0.9 (higher = more diversity)
maxGenerations	Maximum iterations; termination criterion
seed	Random seed for reproducible results
mutationStrategy	Name of mutation operator; one of: "rand1", "rand2", "best1", "best2", "rand↩ ToBest1" Defaults to "rand1" if empty
crossoverStrategy	Name of crossover operator; one of: "bin" (binomial, default), "exp" (exponential) Defaults to "bin" if empty

Returns

FlowShopResult containing:

- sequence: Best job order found
- makespan: Makespan of best solution
- completionTimes: Completion times for best solution
- tardiness: Tardiness vector for best solution
- fitnesses: Convergence history (best fitness per generation)

Thread Safety: Evaluation is parallelized with OpenMP. Concurrent solution evaluations are safe; problem state (blocking, optimizeTardiness) is read-only during optimization.

Example:

```
FlowShop problem(20, 10);
// ... set jobs and due dates ...
FlowShopResult result = problem.runDE(
    true,              // blocking
    true,              // optimize tardiness
    30,                // population size
    0.8,               // F
    0.9,               // CR
    100,               // generations
    42,                // seed
    mutationStrategy = "best1",
    crossoverStrategy = "bin"
);
```

8.14.3.12 runNEH()

```
FlowShopResult FlowShop::runNEH (
            bool blocking = false,
            bool optimizeTardiness = false)
```

Optimizes the flow shop schedule using the NEH heuristic.

The Nawaz-Enscore-Ham (NEH) algorithm is a polynomial-time approximation algorithm that produces high-quality solutions quickly:

1. Initialization: Sort jobs by total processing time (or due date).
2. Insertion: Iteratively insert each remaining job at the position that minimizes the chosen objective (makespan or tardiness).
3. Output: Return the final job order and computed metrics.

Complexity: $O(n^2 m)$ for makespan; $O(n^2 m)$ for tardiness with due dates.

Parameters

blocking	If true, enforces job blocking: a job cannot leave machine i until machine i+1 is free (models buffer constraints)
optimizeTardiness	If true, minimizes total tardiness (sum of max(0, completion - due_date)); if false, minimizes makespan (final completion time)

Returns

FlowShopResult containing:

- sequence: Job order (indices 0 to num_jobs-1)
- makespan: Total completion time of final job on final machine
- completionTimes: m×n matrix of completion times
- tardiness: Vector of tardiness for each job (if optimizeTardiness=true)
- fitnesses: Empty vector (NEH does not track convergence)

Example:

```
FlowShop problem(10, 5);  // 10 jobs, 5 machines
// ... populate jobs_times and due_dates ...
FlowShopResult result = problem.runNEH(false, true);  // Tardiness, no blocking
```

8.14.3.13 set_due_dates()

```
void FlowShop::set_due_dates (
            std::vector< uint64_t > && new_data)   [inline]
```

Replaces the due dates vector.

Sets new due dates for tardiness-based optimization. Must have exactly num_jobs elements.

Parameters

new_data	Rvalue reference to due dates vector (size = num_jobs)

Exceptions

std::runtime_error	if new_data.size() != num_jobs_

Postcondition

due_dates_ contains the new due date values

Note

Due dates are only used when optimizeTardiness=true in runNEH() or runDE().

8.14.3.14 set_jobs()

```
void FlowShop::set_jobs (
            size_t num_jobs,
            size_t num_machines,
            std::vector< uint64_t > && new_data)   [inline]
```

Replaces the entire processing times matrix and dimensions.

Sets new problem dimensions and processing time data. Validates that the data size matches dimensions. Resets due_dates to zero for all jobs.

Parameters

num_jobs	New number of jobs
num_machines	New number of machines
new_data	Rvalue reference to flattened m×n matrix (row-major, size = num_jobs × num_machines)

Exceptions

std::runtime_error	if new_data.size() != num_jobs ∗ num_machines

Postcondition

num_jobs_ and num_machines_ are updated

jobs_times_ contains the new data

due_dates_ is reset to [0, 0, ..., 0]

The documentation for this class was generated from the following files:

- src/opti_py/cpp/include/opti_py/FlowShop/FlowShop.h
- src/opti_py/cpp/src/FlowShop/FlowShop.cpp

8.15 FlowShopResult Struct Reference

Encapsulates the complete output of a flow shop scheduling optimization.

#include <FlowShopResult.h>

Public Attributes

Solution

- std::vector< size_t > sequence
  Job execution sequence (job ordering).

Timing Metrics

- uint64_t makespan
  Total completion time (makespan) of the schedule.
- std::vector< std::vector< uint64_t > > completionTimes
  Completion times matrix: when each job finishes on each machine.

Tardiness Metrics

- std::vector< uint64_t > tardiness
  Per-job tardiness values (lateness from due dates).

Optimization Progress

- std::vector< double > fitnesses
  Convergence history: best fitness per generation.

8.15.1 Detailed Description

Encapsulates the complete output of a flow shop scheduling optimization.

FlowShopResult contains all relevant data from a flow shop optimization run, including the best job sequence found, timing metrics, and (for metaheuristic optimizations) the convergence history showing how fitness improved over time.

Output from NEH Heuristic:

When calling FlowShop::runNEH(), the result contains:

- sequence: The job ordering produced by the NEH algorithm
- makespan: Total completion time (final machine finish time)
- completionTimes: Detailed completion times for each job on each machine
- tardiness: Per-job tardiness values (if optimizeTardiness=true)
- fitnesses: Empty vector (NEH is deterministic, no convergence history)

Output from Differential Evolution:

When calling FlowShop::runDE(), the result contains:

- sequence: The best job ordering found by DE
- makespan: Makespan of the best solution
- completionTimes: Completion times for the best solution

- tardiness: Per-job tardiness for the best solution (if optimizeTardiness=true)
- fitnesses: Convergence history (best fitness per generation), enables tracking optimization progress and assessing convergence quality

Data Relationships:

The fields are internally consistent and derived from a single optimal job sequence. Specifically:

- completionTimes is computed from sequence according to flow shop rules
- makespan = completionTimes[num_jobs-1][num_machines-1] (final completion time)
- tardiness[i] = max(0, completionTimes[i][num_machines-1] - due_date[sequence[i]])
- fitnesses tracks the best objective value (makespan or tardiness) per generation

Usage Example:

```
FlowShop problem(20, 10);
// ... populate problem data ...

// Optimize with DE and analyze results
FlowShopResult result = problem.runDE(true, true, 30, 0.8, 0.9, 100, 42, mut, cross);

// Print final solution
std::cout « "Best makespan: " « result.makespan « std::endl;
std::cout « "Job sequence: ";
for (size_t job : result.sequence)
    std::cout « job « " ";
std::cout « std::endl;

// Analyze convergence
std::cout « "Initial fitness: " « result.fitnesses.front() « std::endl;
std::cout « "Final fitness: " « result.fitnesses.back() « std::endl;
std::cout « "Improvement: " « (result.fitnesses.front() - result.fitnesses.back()) « std::endl;
```

Thread Safety:

FlowShopResult is not thread-safe for concurrent modifications. However, once fully constructed by the optimizer, it can be safely read by multiple threads without synchronization.

Note

All vectors are guaranteed to have consistent sizes after construction:

- sequence.size() = num_jobs
- completionTimes.size() = num_jobs, each row has num_machines elements
- tardiness.size() = num_jobs
- fitnesses.size() = num_generations (0 for NEH, maxGenerations for DE)

See also

FlowShop::runNEH() for NEH output semantics

FlowShop::runDE() for DE output semantics

8.15.2 Member Data Documentation

8.15.2.1 completionTimes

std::vector<std::vector<uint64_t> > FlowShopResult::completionTimes

Completion times matrix: when each job finishes on each machine.

A matrix where completionTimes[i][j] represents the time at which the job at position i in the sequence completes on machine j.

Dimensions: num_jobs rows × num_machines columns

Indexing:

- completionTimes[i] = completion times for the job at position i in sequence
- completionTimes[i][j] = finish time of position-i job on machine j

Example: For a 3-job, 2-machine problem:

```
completionTimes = [
    [5, 15],   // Job at position 0 finishes at t=5 on M1, t=15 on M2
    [8, 22],   // Job at position 1 finishes at t=8 on M1, t=22 on M2
    [16, 26]   // Job at position 2 finishes at t=16 on M1, t=26 on M2
]
```

Flow Shop Properties:

- completionTimes[i][0] >= completionTimes[i-1][0] (machines process in sequence)
- completionTimes[i][j] >= completionTimes[i][j-1] (job finishes on next machine)
- completionTimes[i][j] >= completionTimes[i-1][j] (machine respects previous job)

Blocking Constraint: If blocking is enabled, additional constraint:

- completionTimes[i][j] >= completionTimes[i-1][j+1] (buffer constraint)

Tardiness Calculation: tardiness[i] = max(0, completionTimes[i][num_machines-1] - due_date[sequence↩[i]])

Size: num_jobs × num_machines

Note

All values are non-negative and non-decreasing along rows and columns (except for blocking constraints which may create more complex patterns).

The final element completionTimes.back().back() equals makespan.

8.15.2.2 fitnesses

std::vector<double> FlowShopResult::fitnesses

Convergence history: best fitness per generation.

A vector tracking the best objective value found at each generation during metaheuristic optimization. Enables analysis of optimization progress and assessment of convergence quality.

Semantics:

- fitnesses[i] = best objective value found through generation i (inclusive)
- fitnesses[0] = best value in generation 0 (initial population)
- fitnesses[num_generations-1] = final best value (equals makespan or total tardiness)
- Vector is monotone non-increasing (or non-decreasing for maximization)

Content:

- For makespan minimization: fitness = makespan (time units)
- For tardiness minimization: fitness = total tardiness (time units)

Size:

- NEH heuristic: empty vector (size 0); NEH is deterministic
- Differential Evolution: size = maxGenerations; one value per generation

Example: For a DE run with maxGenerations=100:

```
fitnesses.size() = 100
fitnesses[0] = 1500      // Initial best (from population)
fitnesses[50] = 1200     // Best after 50 generations
fitnesses[99] = 1050     // Final best value

// Improvement over 100 generations:
improvement = fitnesses[0] - fitnesses[99] = 450 time units
```

Analysis Use Cases:

1. Convergence Speed:
```
size_t generations_to_converge = 0;
for (size_t i = 0; i < result.fitnesses.size(); ++i) {
    if (result.fitnesses[i] == result.fitnesses.back()) {
        generations_to_converge = i;
        break;
    }
}
```
2. Convergence Quality:
```
double initial = result.fitnesses.front();
double final = result.fitnesses.back();
double improvement = initial - final;
double improvement_percent = 100.0 * improvement / initial;
```
3. Stagnation Detection:
```
bool converged_early = result.fitnesses[50] == result.fitnesses.back();
```

When to Use:

- To assess optimization quality and performance
- To tune algorithm parameters (F, CR, popSize, etc.)
- To compare different mutation/crossover strategies
- To detect if more generations are needed

When Empty:

- If produced by FlowShop::runNEH() (heuristics don't have convergence history)
- Check size before accessing: if (!result.fitnesses.empty())

Invariant

fitnesses is empty for NEH results
fitnesses.size() = maxGenerations for DE results
fitnesses[i] >= fitnesses[i+1] (monotone decreasing for minimization)
fitnesses.back() = makespan or total_tardiness (final best value)

Note

To get total tardiness: std::accumulate(result.tardiness.begin(), result.tardiness.end(), 0ull)

8.15.2.3 makespan

uint64_t FlowShopResult::makespan

Total completion time (makespan) of the schedule.

The time at which the final job finishes on the final machine. Represents the total elapsed time from the start of the first job until the completion of the last job.

Definition: makespan = completionTimes[num_jobs - 1][num_machines - 1]

Unit: Time units as specified in the problem data (seconds, minutes, etc.)

Semantics:

- Lower values are better (minimization objective)
- Achievable lower bound: max(total processing time / num_machines, longest job, longest path through all machines)
- For makespan minimization, this is the primary objective optimized

Example: If a problem with 3 jobs and 2 machines has:

- Job 0: [5, 10] time units
- Job 1: [3, 7]
- Job 2: [8, 4] With sequence [1, 0, 2], makespan might be 29 (the final job finishes at t=29).

Invariant

makespan >= 0
makespan = completionTimes.back().back() (if completionTimes is filled)

8.15.2.4 sequence

std::vector<size_t> FlowShopResult::sequence

Job execution sequence (job ordering).

A permutation of job indices [0, num_jobs-1] representing the optimal schedule found by the optimizer. The vector has length num_jobs.

Interpretation:

- sequence[0] = the first job to schedule
- sequence[1] = the second job to schedule
- sequence[i] = the job at position i in the schedule

Example: If sequence = [2, 0, 1], the execution order is: job 2, then job 0, then job 1.

Size: num_jobs (the number of jobs in the problem)

Valid Range: Each element is in [0, num_jobs-1], all elements are distinct.

Note

This sequence is derived from the optimizer's SPV encoding by sorting jobs by their SPV values in descending order.

8.15.2.5 tardiness

std::vector<uint64_t> FlowShopResult::tardiness

Per-job tardiness values (lateness from due dates).

A vector where tardiness[i] represents how late the job at position i is relative to its due date.

Definition: tardiness[i] = max(0, completionTimes[i][num_machines-1] - due_date[sequence[i]])

Semantics:

- 0 = job completed on time (no lateness)
- > 0 = job is late by this many time units
- Sum of all tardiness values is the total tardiness objective

Unit: Time units (same as completion times)

Example: If due_dates = [20, 30, 25] and jobs finish at [15, 35, 22]:

```
tardiness = [
    max(0, 15 - 20) = 0,    // Job 0 on time
    max(0, 35 - 30) = 5,    // Job 1 late by 5
    max(0, 22 - 25) = 0     // Job 2 on time
]
```

Size: num_jobs (length matches sequence and completionTimes rows)

When Populated:

- Always filled by runNEH() and runDE()
- Non-zero values only if due dates are meaningful (not all zero)

Related Objective:

- Sum of tardiness = std::accumulate(tardiness.begin(), tardiness.end(), 0)
- Used for tardiness minimization objective

Invariant

tardiness[i] >= 0 for all i

tardiness.size() = sequence.size()

Note

This is computed from completionTimes and due dates; modifying it directly will not affect makespan or completionTimes.

The documentation for this struct was generated from the following file:

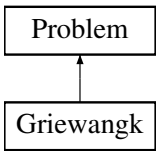
- src/opti_py/cpp/include/opti_py/FlowShop/FlowShopResult.h

8.16 Griewangk Class Reference

Implements the Griewangk benchmark function.

#include <Griewangk.h>

Inheritance diagram for Griewangk:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.16.1 Detailed Description

Implements the Griewangk benchmark function.

8.16.2 Member Function Documentation

8.16.2.1 evaluate()

double Griewangk::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

  Returns

  The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

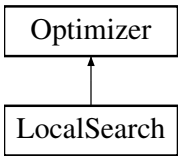
- src/opti_py/cpp/include/opti_py/Problem/Griewangk.h

8.17 LocalSearch Class Reference

Implements a local search optimization algorithm.

#include <LocalSearch.h>

Inheritance diagram for LocalSearch:



Public Member Functions

- LocalSearch (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations, double delta, int numNeighbors)

  Constructs a LocalSearch optimizer.
- std::vector< double > optimize () override

  Executes the local search optimization process.

Public Member Functions inherited from Optimizer

- Optimizer (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations)

  Constructs an Optimizer.
- virtual ~Optimizer ()=default

  Virtual destructor for safe polymorphic deletion.

- double getBestFitness ()
- std::vector< double > & getBestSolution ()
- std::vector< double > & getBestFitnesses ()
- std::vector< std::vector< double > > & getSolutions ()
- int getMaxIterations ()
- std::shared_ptr< Problem > getProblem ()
- std::shared_ptr< SolutionBuilder > getSolutionBuilder ()

Additional Inherited Members

Protected Attributes inherited from Optimizer

- std::shared_ptr< Problem > problem

  Optimization problem definition.
- std::shared_ptr< SolutionBuilder > solutionBuilder

  Solution generation utility.
- int maxIterations

  Maximum number of iterations.
- std::vector< double > bestSolution

  Best solution found.
- std::vector< double > bestFitnesses

  Best fitness value so far per iteration.
- std::vector< std::vector< double > > solutions

  All generated solutions.

8.17.1 Detailed Description

Implements a local search optimization algorithm.

The LocalSearch optimizer explores the neighborhood of the current solution by generating candidate neighbors within a fixed delta. The search continues until a local minimum is reached or the maximum number of iterations is exceeded.

This class supports both single-run local search and repeated local search depending on the iteration limit provided.

8.17.2 Constructor & Destructor Documentation

8.17.2.1 LocalSearch()

```
LocalSearch::LocalSearch (
            std::shared_ptr< SolutionBuilder > solutionBuilder,
            std::shared_ptr< Problem > problem,
            int maxIterations,
            double delta,
            int numNeighbors)   [inline]
```

Constructs a LocalSearch optimizer.

Parameters

solutionBuilder	Reference to the solution generator.
problem	Reference to the optimization problem.
maxIterations	Maximum number of local search iterations.
delta	Neighborhood radius for neighbor generation.
numNeighbors	Number of neighbors sampled per iteration.

8.17.3 Member Function Documentation

8.17.3.1 optimize()

```
std::vector< double > LocalSearch::optimize ()   [override], [virtual]
```

Executes the local search optimization process.

Returns

The functions execution time.

Implements Optimizer.

The documentation for this class was generated from the following files:

- src/opti_py/cpp/include/opti_py/Optimizer/LocalSearch.h
- src/opti_py/cpp/src/Optimizer/LocalSearch.cpp

8.18 MersenneTwister Class Reference

#include <mt.h>

Public Member Functions

- MersenneTwister (void)
- ~MersenneTwister (void)
- double random (void)
- void print (void)
- void init_genrand (unsigned long s)
- void init_by_array (unsigned long *init_key, int key_length)
- unsigned long genrand_int32 (void)
- long genrand_int31 (void)
- double genrand_real1 (void)
- double genrand_real2 (void)
- double genrand_real3 (void)
- double genrand_res53 (void)

8.18.1 Detailed Description

mt.h: Mersenne Twister header file

Jason R. Blevins jrblevin@sdf.lonestar.org Durham, March 7, 2007 Mersenne Twister.

M. Matsumoto and T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudorandom number generator", ACM Trans. on Modeling and Computer Simulation Vol. 8, No. 1, January pp.3-30 (1998).

http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html.

8.18.2 Constructor & Destructor Documentation

8.18.2.1 MersenneTwister()

MersenneTwister::MersenneTwister (
void)

C++ Mersenne Twister wrapper class written by Jason R. Blevins jrblevin@sdf.lonestar.org on July 24, 2006. Based on the original MT19937 C code by Takuji Nishimura and Makoto Matsumoto. Constructor

8.18.2.2 ~MersenneTwister()

MersenneTwister::~MersenneTwister (
void)

Destructor

8.18.3 Member Function Documentation

8.18.3.1 genrand_int31()

```
long MersenneTwister::genrand_int31 (
            void )
```

Generates a random integer on [0,0x7fffffff].

Returns

a random integer on [0,0x7fffffff]

8.18.3.2 genrand_int32()

```
unsigned long MersenneTwister::genrand_int32 (
            void )
```

Generates a random number on [0,0xffffffff]-interval

Returns

random number on [0, 0xffffffff]

8.18.3.3 genrand_real1()

```
double MersenneTwister::genrand_real1 (
            void )
```

Generates a random real number on [0,1].

Returns

a random real number on [0,1]

8.18.3.4 genrand_real2()

```
double MersenneTwister::genrand_real2 (
            void )
```

Generates a random real number on [0,1).

Returns

a random real number on [0,1)

8.18.3.5 genrand_real3()

```
double MersenneTwister::genrand_real3 (
            void )
```

Generates a random real number on (0,1).

Returns

a random real number on (0,1)

8.18.3.6 genrand_res53()

```
double MersenneTwister::genrand_res53 (
            void )
```

Generates a random real number on [0,1) with 53-bit precision.

Returns

a random 53-bit real number on [0,1)

8.18.3.7 init_by_array()

```
void MersenneTwister::init_by_array (
            unsigned long * init_key,
            int key_length)
```

Seed the Mersenne Twister using an array.

Parameters

init_key	an array for initializing keys
key_length	the length of init_key

8.18.3.8 init_genrand()

```
void MersenneTwister::init_genrand (
            unsigned long s)
```

Initializes the Mersenne Twister with a seed.

Parameters

s	seed

8.18.3.9 print()

void MersenneTwister::print (
void)

Print interesting information about the Mersenne Twister.

The documentation for this class was generated from the following files:

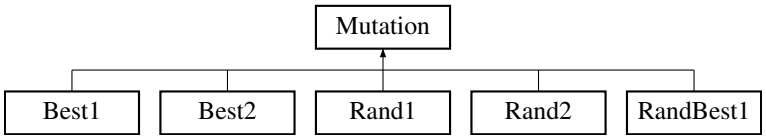
- src/opti_py/cpp/include/opti_py/External/mt.h
- src/opti_py/cpp/src/External/mt.cpp

8.19 Mutation Class Reference

Abstract base class for Differential Evolution mutation strategies.

#include <Mutation.h>

Inheritance diagram for Mutation:



Public Member Functions

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.
- virtual std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound)=0

  Generates a mutant vector using the mutation strategy.

Protected Member Functions

Shared Utilities

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.19.1 Detailed Description

Abstract base class for Differential Evolution mutation strategies.

Mutation defines the interface and shared utilities for all DE mutation operators. Each concrete mutation strategy (Rand1, Best1, Rand2, Best2, RandBest1) implements the mutate() method to generate a mutant vector from a population based on a selected mutation formula.

Mutation Role in DE:

In Differential Evolution, mutation is the primary source of variation:

1. Select random individuals from the population
2. Apply a weighted combination of differences to create new variation
3. The mutation formula determines which individuals and differences are used

Mutation Formula Notation:

Standard DE mutation formulas follow the pattern: DE/base/N, where:

- base: The base vector used as starting point (either random or best)
- N: The number of difference vectors applied (1 or 2)

Common formulas:

- DE/rand/1: mutant = r0 + F∗(r1 - r2)
- DE/rand/2: mutant = r0 + F∗(r1 - r2) + F∗(r3 - r4)
- DE/best/1: mutant = best + F∗(r1 - r2)
- DE/best/2: mutant = best + F∗(r1 - r2) + F∗(r3 - r4)
- DE/rand-to-best/1: mutant = x + ∗(best - x) + F∗(r1 - r2)

Scaling Factor F:

The differential weight F (typically in range 0.0 to 2.0) controls the amplitude of variation. Key considerations:

- F 0.5-0.8: Conservative exploration, good for fine-tuning
- F 0.8-1.5: Moderate exploration-exploitation balance
- F > 1.5: Aggressive exploration, may miss good regions

Individual Selection:

The getSubset() helper selects random distinct individuals from the population (excluding the target individual). This ensures the mutant is derived from different genetic material than the target.

Note

All mutation strategies inherit from this base class and override mutate().

The public interface consists solely of the virtual mutate() method.

See also

Rand1, Rand2, Best1, Best2, RandBest1 for concrete implementations

8.19.2 Member Function Documentation

8.19.2.1 getSubset()

```
std::vector< size_t > Mutation::getSubset (
            size_t populationSize,
            size_t subsetSize,
            size_t source,
            MersenneTwister & mt)   [inline], [protected]
```

Selects a random subset of population indices, excluding a source index.

This is a shared utility used by all mutation strategies to randomly select distinct individuals from the population. The selection uses a partial Fisher-Yates shuffle for efficiency and guarantees no duplicates.

Algorithm:

1. Create a list of all valid indices [0, populationSize-1] except source
2. Perform first subsetSize iterations of Fisher-Yates shuffle:
   - For each position i, randomly swap with position j in range [i, size)
3. Return the first subsetSize elements (which are now randomized)

Complexity:

Time: O(populationSize + subsetSize) for setup and shuffle Space: O(populationSize) for temporary index vector

Parameters:

Parameters

in	populationSize	Total number of individuals in the population. Range: subsetSize + 1 to infinity.
in	subsetSize	Number of distinct indices to select. Range: [1, populationSize - 1]. Must be less than populationSize (to exclude source).
in	source	Index to exclude from the selection. This individual is not included in the returned subset. Range: [0, populationSize - 1].
in,out	mt	Mersenne Twister random number generator. Passed by reference; state is modified. Used to generate random swap positions.

Return Value:

Returns

A vector of subsetSize distinct indices from [0, populationSize-1], excluding source. All elements are unique. Size: exactly subsetSize.

Guarantees:

- All returned indices are distinct (no duplicates)
- No returned index equals source
- All indices are in range [0, populationSize - 1]
- The selection is uniformly random (all subsets equally likely)

Usage in Mutation Strategies:

Mutation formulas require 2-5 distinct random individuals:

```
// For DE/rand/1: need 3 individuals
std::vector<size_t> r = getSubset(popSize, 3, targetIndex, mt);
// r[0], r[1], r[2] are three distinct random individuals

// For DE/rand/2: need 5 individuals
std::vector<size_t> r = getSubset(popSize, 5, targetIndex, mt);
```

Example:

```
// Select 3 random individuals from a population of 20, excluding individual 5
auto indices = getSubset(20, 3, 5, rng);
// Possible result: [8, 15, 2] (three random distinct values != 5)
```

Preconditions:

- populationSize >= subsetSize + 1 (at least room for subset + source)
- source < populationSize
- subsetSize > 0

Implementation Note:

The partial Fisher-Yates algorithm is used instead of full shuffle or repeated sampling for O(population↩Size + subsetSize) complexity without the risk of duplicate rejection sampling.

8.19.2.2 mutate()

```
virtual std::vector< double > Mutation::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [pure virtual]
```

Generates a mutant vector using the mutation strategy.

Pure virtual method that each concrete mutation strategy implements to produce a mutant vector. The mutant combines selected population individuals according to the strategy's formula.

Algorithm Pattern:

All mutation strategies follow this general workflow:

1. Select N random distinct individuals from the population (via getSubset())
2. Compute a weighted combination of these individuals
3. Clamp the result to the domain bounds
4. Return the mutant vector

Parameters:

Parameters

in	population	The complete population of candidate solutions. Each element is a vector of dimension equal to the optimization problem. Size: popSize × dimension.

in	targetIndex	Index of the target individual in the population. This individual is excluded from random selection (cannot use its own genetic material for mutation). Range: [0, popSize-1].
in	F	Differential weight (mutation scaling factor). Controls the amplitude of variation in the mutation formula. Typical range: (0, 2], common values: 0.5-1.0. • Larger F: more aggressive exploration • Smaller F: more conservative fine-tuning
in	bestVector	The best (lowest fitness) individual found so far. Used in "best" strategies (Best1, Best2, RandBest1). For "rand" strategies, may be ignored. Dimension: same as population elements.
in,out	mt	Mersenne Twister random number generator. Used to randomly select individuals and generate variations. Passed by reference; state is modified during execution.
in	lowerBound	Lower bound of the feasible domain. All dimensions share this uniform bound. Used to clamp mutant values to valid range.
in	upperBound	Upper bound of the feasible domain. All dimensions share this uniform bound. Used to clamp mutant values to valid range.

Return Value:

Returns

A new mutant vector with dimension matching the population. All values are within [lowerBound, upperBound] due to clamping. Size: equal to population[0].size().

Postconditions:

The returned mutant vector satisfies:

- Dimension matches population dimension
- All elements are within [lowerBound, upperBound]
- Is derived from 2-5 random population members (depends on strategy)

Strategy-Specific Behavior:

Different concrete strategies implement different formulas:

- Rand1: Uses 3 random individuals; more exploratory
- Best1: Uses best individual as base; more exploitative
- Rand2: Uses 5 random individuals; more variation, slower
- Best2: Uses best with 4 additional individuals; balanced
- RandBest1: Interpolates between best and random; hybrid approach

Complexity:

Time: O(dimension) - linear in the problem dimension Space: O(dimension) - for the returned mutant vector

Usage Example:

```
std::unique_ptr<Mutation> strategy = std::make_unique<Best1>();
std::vector<double> mutant = strategy->mutate(
    population,
    targetIdx,
    0.8,          // F
    bestSolution,
    rng,
    -1.0, 1.0     // bounds
);
```

See also

getSubset() for the random selection mechanism

Implemented in Best1, Best2, Rand1, Rand2, and RandBest1.

The documentation for this class was generated from the following file:

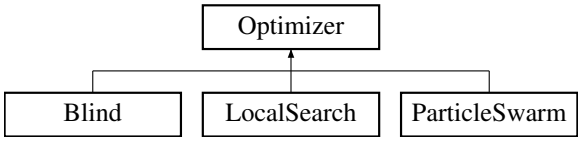
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Mutation.h

8.20 Optimizer Class Reference

Abstract base class for all optimization algorithms.

#include <Optimizer.h>

Inheritance diagram for Optimizer:



Public Member Functions

- Optimizer (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations)

  Constructs an Optimizer.
- virtual ~Optimizer ()=default

  Virtual destructor for safe polymorphic deletion.
- virtual std::vector< double > optimize ()=0

  Runs the optimization algorithm.

Accessors

- double getBestFitness ()
- std::vector< double > & getBestSolution ()
- std::vector< double > & getBestFitnesses ()
- std::vector< std::vector< double > > & getSolutions ()
- int getMaxIterations ()
- std::shared_ptr< Problem > getProblem ()
- std::shared_ptr< SolutionBuilder > getSolutionBuilder ()

Protected Attributes

- std::shared_ptr< Problem > problem

  Optimization problem definition.
- std::shared_ptr< SolutionBuilder > solutionBuilder

  Solution generation utility.
- int maxIterations

  Maximum number of iterations.
- std::vector< double > bestSolution

  Best solution found.
- std::vector< double > bestFitnesses

  Best fitness value so far per iteration.
- std::vector< std::vector< double > > solutions

  All generated solutions.

8.20.1 Detailed Description

Abstract base class for all optimization algorithms.

The Optimizer class defines a common interface and shared state for optimization algorithms operating on benchmark problems. Derived classes must implement the optimize() method. Results for each run are stored in fields and accessed through getter methods

8.20.2 Constructor & Destructor Documentation

8.20.2.1 Optimizer()

```
Optimizer::Optimizer (
            std::shared_ptr< SolutionBuilder > solutionBuilder,
            std::shared_ptr< Problem > problem,
            int maxIterations)    [inline]
```

Constructs an Optimizer.

Parameters

solutionBuilder	Shared pointer to the solution generator.
problem	Shared pointer to the optimization problem.
maxIterations	Maximum number of iterations allowed.

8.20.3 Member Function Documentation

8.20.3.1 getBestFitness()

```
double Optimizer::getBestFitness ()    [inline]
```

Returns

The final best fitness value

8.20.3.2 getBestFitnesses()

std::vector< double > & Optimizer::getBestFitnesses () [inline]

Returns

Best fitness value so far per iteration

8.20.3.3 getBestSolution()

std::vector< double > & Optimizer::getBestSolution () [inline]

Returns

Reference to the best solution vector

8.20.3.4 getMaxIterations()

int Optimizer::getMaxIterations () [inline]

Returns

Maximum number of iterations

8.20.3.5 getProblem()

std::shared_ptr< Problem > Optimizer::getProblem () [inline]

Returns

Reference to the optimization problem

8.20.3.6 getSolutionBuilder()

std::shared_ptr< SolutionBuilder > Optimizer::getSolutionBuilder () [inline]

Returns

Reference to the solution builder

8.20.3.7 getSolutions()

std::vector< std::vector< double > > & Optimizer::getSolutions () [inline]

Returns

All solutions evaluated during optimization

8.20.3.8 optimize()

virtual std::vector< double > Optimizer::optimize () [pure virtual]

Runs the optimization algorithm.

Returns

Execution time of the algorithm

Implemented in Blind, LocalSearch, and ParticleSwarm.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Optimizer/Optimizer.h

8.21 OptimizerFactory Class Reference

Factory class for creating optimizer instances.

#include <OptimizerFactory.h>

Static Public Member Functions

- static std::unique_ptr< Optimizer > initOptimizer (Problem &problem, ExperimentConfig &config, SolutionBuilder &builder)

  Initializes an optimizer based on configuration settings.

8.21.1 Detailed Description

Factory class for creating optimizer instances.

The OptimizerFactory encapsulates the logic for selecting and constructing the appropriate optimization algorithm based on experimental configuration parameters.

8.21.2 Member Function Documentation

8.21.2.1 initOptimizer()

std::unique_ptr< Optimizer > OptimizerFactory::initOptimizer (
Problem & problem,
ExperimentConfig & config,
SolutionBuilder & builder) [inline], [static]

Initializes an optimizer based on configuration settings.

Parameters

problem	Reference to the optimization problem.

config	Experiment configuration specifying optimizer type and parameters.
builder	Reference to the solution builder.

Returns

A unique pointer to the initialized Optimizer instance, or nullptr if the optimizer type is unsupported.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Optimizer/OptimizerFactory.h

8.22 OptResult Struct Reference

Encapsulates the complete output of a Differential Evolution optimization run.

#include <OptResult.h>

Public Attributes

Solution

- std::vector< double > bestSolution
  The best parameter vector found during optimization.

Fitness Values

- std::vector< double > bestFitnesses
  Convergence history: best fitness per generation.
- double bestFitness
  The best (lowest) fitness value found during optimization.

8.22.1 Detailed Description

Encapsulates the complete output of a Differential Evolution optimization run.

OptResult contains all relevant data produced by DifferentialEvolution::optimize(), including the best solution discovered, its fitness value, and the convergence trajectory showing how fitness improved over generations.

Output from DifferentialEvolution::optimize():

When calling DifferentialEvolution::optimize(), the returned OptResult contains:

- bestSolution: The parameter vector with lowest fitness found
- bestFitness: The fitness value of bestSolution (scalar)
- bestFitnesses: Convergence history (best fitness per generation)

Data Relationships:

The fields are internally consistent:

- bestFitness = minimum value in bestFitnesses (final value)
- bestFitnesses[i] = best fitness found through generation i (inclusive)
- bestFitnesses is non-increasing (monotone decreasing for minimization)
- bestSolution.size() = problem dimension
- bestFitnesses.size() = maxGenerations (from optimizer)

Usage Example:

```
// Create problem and run DE
MyOptimizationProblem problem;
OptResult result = DifferentialEvolution::optimize(
    problem,
    popSize = 30,
    F = 0.8,
    CR = 0.9,
    maxGenerations = 100,
    seed = 42,
    mutationStrategy = "best1",
    crossoverStrategy = "bin"
);

// Access solution
std::cout « "Best fitness: " « result.bestFitness « std::endl;
std::cout « "Solution: ";
for (double x : result.bestSolution)
    std::cout « x « " ";
std::cout « std::endl;

// Analyze convergence
std::cout « "Initial fitness: " « result.bestFitnesses.front() « std::endl;
std::cout « "Final fitness: " « result.bestFitnesses.back() « std::endl;
std::cout « "Improvement: "
    « (result.bestFitnesses.front() - result.bestFitnesses.back())
    « std::endl;
```

Thread Safety:

OptResult is not thread-safe for concurrent modifications. However, once fully constructed by the optimizer, it can be safely read by multiple threads without synchronization.

Typical Size:

For a 30-dimensional problem with 100 generations:

- bestSolution.size() = 30
- bestFitnesses.size() = 100
- Memory usage: ∼30 doubles + 100 doubles 1 KB

Note

All vectors are guaranteed to be non-empty after successful optimization.

bestFitness is always the last (best) value in bestFitnesses.

All fitness values are monotone non-increasing (for minimization).

See also

DifferentialEvolution::optimize() for method that produces this result

8.22.2 Member Data Documentation

8.22.2.1 bestFitness

double OptResult::bestFitness

The best (lowest) fitness value found during optimization.

The minimum fitness achieved by the optimizer across all generations. This scalar represents the quality of the solution.

Semantics:

- Lower values are better (minimization problem)
- This is the fitness of bestSolution
- Objective function value: f(bestSolution) = bestFitness

Relationship to bestFitnesses:

- bestFitness = bestFitnesses.back() (final value)
- bestFitness = minimum element in bestFitnesses
- bestFitness achieved at some generation index

Example:

For Sphere function minimization:

```
// Sphere: f(x) = sum(x_i²), optimum at x = 0 with f = 0
result.bestFitness = 0.0042  // Final best fitness
result.bestSolution = [0.04, 0.02, 0.03, ...]  // Parameters
// Verify: sum([0.04²,0.02²,0.03²,...])  0.0042
```

Convergence Assessment:

```
// Check if optimization reached target
double target_fitness = 0.001;
if (result.bestFitness <= target_fitness) {
    std::cout « "Target reached!" « std::endl;
} else {
    std::cout « "Target not reached. Gap: "
        « (result.bestFitness - target_fitness) « std::endl;
}
```

Comparison Between Runs:

```
// Run optimization multiple times, track best results
std::vector<OptResult> results;
for (int seed = 0; seed < 10; ++seed) {
    results.push_back(DifferentialEvolution::optimize(..., seed));
}

// Find best run overall
auto best_run = std::min_element(
    results.begin(), results.end(),
    [](const OptResult& a, const OptResult& b) {
        return a.bestFitness < b.bestFitness;
    }
);
```

Statistical Analysis:

```
// For multiple runs
std::vector<OptResult> results;  // Filled from multiple optimizations
```

```
std::vector<double> fitnesses;

for (const auto& result : results) {
    fitnesses.push_back(result.bestFitness);
}

// Compute statistics
double mean = std::accumulate(fitnesses.begin(), fitnesses.end(), 0.0)
            / fitnesses.size();
double min = *std::min_element(fitnesses.begin(), fitnesses.end());
double max = *std::max_element(fitnesses.begin(), fitnesses.end());

std::cout « "Best: " « min « ", Mean: " « mean « ", Worst: " « max;
```

Unit and Scale:

- Units depend on objective function (user-defined)
- Scale depends on problem (can be 0.001 to 1e9)
- Always comparable within same problem
- May need normalization for cross-problem comparisons

Invariant

bestFitness = min(bestFitnesses)

bestFitness is finite (not NaN or infinity)

bestFitness is the objective function value at bestSolution

Note

This is the primary metric of solution quality

Lower values are better

8.22.2.2 bestFitnesses

std::vector<double> OptResult::bestFitnesses

Convergence history: best fitness per generation.

A record of the best fitness value found at each generation during the optimization. Enables analysis of how the optimization progressed and assessment of convergence behavior.

Content:

- bestFitnesses[i] = best fitness found through generation i (inclusive)
- Lower values are better (minimization problem)
- Values are monotone non-increasing (or plateau)

Size: maxGenerations (from optimizer call)

- For 100 generation run: size = 100
- For 1000 generation run: size = 1000

Indexing:

- bestFitnesses[0] = best fitness in generation 0 (initial population)
- bestFitnesses[50] = best fitness after 50 generations
- bestFitnesses[99] = best fitness in generation 99 (final)

Invariant Properties:

- Monotone non-increasing: bestFitnesses[i] >= bestFitnesses[i+1]
- Starts with initial population fitness
- Ends with final best value: bestFitnesses.back() == bestFitness

Example Convergence:

```
Generation 0:   fitness = 1500.0  (random initial population)
Generation 10:  fitness = 1200.5  (improvement)
Generation 50:  fitness = 850.3   (good improvement)
Generation 99:  fitness = 425.1   (final best)

bestFitnesses = [1500.0, 1499.2, 1498.1, ..., 850.3, ..., 425.1]
Size = 100, all non-increasing
```

Analysis Use Cases:

1. Convergence Speed:

```
size_t generations_to_converge = 0;
for (size_t i = 0; i < result.bestFitnesses.size(); ++i) {
    if (result.bestFitnesses[i] == result.bestFitness) {
        generations_to_converge = i;
        break;
    }
}
```

2. Convergence Quality:

```
double initial = result.bestFitnesses.front();
double final = result.bestFitnesses.back();
double improvement = initial - final;
double improvement_percent = 100.0 * improvement / initial;
```

3. Stagnation Detection:

```
// Check if optimization stalled after halfway point
size_t halfway = result.bestFitnesses.size() / 2;
bool stalled = (result.bestFitnesses[halfway] == result.bestFitness);
```

4. Learning Curve:

```
// Plot fitnesses per generation to visualize convergence curve
for (size_t gen = 0; gen < result.bestFitnesses.size(); ++gen) {
    double fitness = result.bestFitnesses[gen];
    std::cout « gen « "," « fitness « "\n";
}
```

5. Performance Assessment:

```
// Compare two runs' convergence
auto convergence_ratio = [](const OptResult& r) {
    double init = r.bestFitnesses.front();
    double final = r.bestFitnesses.back();
    return (init - final) / init;  // Improvement ratio
};
double ratio1 = convergence_ratio(result1);
double ratio2 = convergence_ratio(result2);
```

Visualization:

```
// Log convergence for plotting
std::ofstream file("convergence.csv");
for (size_t gen = 0; gen < result.bestFitnesses.size(); ++gen) {
    file « gen « "," « result.bestFitnesses[gen] « "\n";
}
file.close();

// Then plot with your favorite tool:
// gnuplot: plot "convergence.csv" using 1:2 with lines
```

Parameter Tuning Based on Convergence:

```
// If stagnating early, increase exploration
if (result.bestFitnesses[50] == result.bestFitness) {
    // Run with larger F or higher CR
}

// If slow convergence, increase exploitation
if (result.bestFitness > acceptable_threshold) {
    // Run with smaller F or lower CR, or use Best1 mutation
}
```

Invariant

bestFitnesses.size() = maxGenerations

bestFitnesses[i] >= bestFitnesses[i+1] (non-increasing)

bestFitnesses.back() == bestFitness (last value is best)

All values are finite (not NaN or infinity)

Note

Used to track optimization progress over time

Essential for understanding algorithm behavior

Enables convergence diagnostics and tuning

8.22.2.3 bestSolution

std::vector<double> OptResult::bestSolution

The best parameter vector found during optimization.

The parameter vector that achieved the lowest (best) fitness value during the entire optimization run. This is the primary output of the optimizer representing the solution to the optimization problem.

Content:

- Each element is a parameter/variable to be optimized
- Values are constrained to domain [lowerBound, upperBound]
- Dimension matches the optimization problem dimension

Size: problem.dimension

- For 30-dimensional problem: size = 30
- For 100-dimensional problem: size = 100

Semantics:

- These are the x-values that minimize f(x)
- Each element is a continuous value in the domain
- Order and meaning depend on problem definition

Example:

For a 3D Rosenbrock minimization:

```
// Rosenbrock: f(x,y,z) = (1-x)² + 100(y-x²)² + (1-z)² + 100(z-y²)²
// Optimum at (1, 1, 1) with fitness 0
result.bestSolution = [0.9999, 0.9998, 1.0001]  // Close to optimum
result.bestFitness = 0.000042  // Near-optimal fitness
```

Access Pattern:

```
// Iterate through solution
for (size_t i = 0; i < result.bestSolution.size(); ++i) {
    double param = result.bestSolution[i];
    // Use parameter in your application
}

// Or with range-based for (C++11+)
for (double param : result.bestSolution) {
    // Use parameter
}
```

Post-Processing:

You may want to post-process the solution:

```
// Clamp to exact bounds if numerical precision drifted
for (auto& x : result.bestSolution)
    x = std::clamp(x, lowerBound, upperBound);

// Round if integer solution expected
for (auto& x : result.bestSolution)
    x = std::round(x);
```

Invariant

bestSolution.size() = problem dimension

All elements are within [lowerBound, upperBound]

fitness(bestSolution) = bestFitness

Note

This is non-empty after successful optimization

The documentation for this struct was generated from the following file:

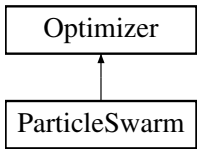
- src/opti_py/cpp/include/opti_py/Optimizer/OptResult.h

8.23 ParticleSwarm Class Reference

Implements the Particle Swarm Optimization algorithm.

#include <ParticleSwarm.h>

Inheritance diagram for ParticleSwarm:



Public Member Functions

- ParticleSwarm (const ExperimentConfig &config, double c1, double c2, int popSize)
- std::vector< double > optimize () override

  Executes the Differential Evolution optimization process.

Public Member Functions inherited from Optimizer

- Optimizer (std::shared_ptr< SolutionBuilder > solutionBuilder, std::shared_ptr< Problem > problem, int maxIterations)

  Constructs an Optimizer.
- virtual ~Optimizer ()=default

  Virtual destructor for safe polymorphic deletion.

- double getBestFitness ()
- std::vector< double > & getBestSolution ()
- std::vector< double > & getBestFitnesses ()
- std::vector< std::vector< double > > & getSolutions ()
- int getMaxIterations ()
- std::shared_ptr< Problem > getProblem ()
- std::shared_ptr< SolutionBuilder > getSolutionBuilder ()

Additional Inherited Members

Protected Attributes inherited from Optimizer

- std::shared_ptr< Problem > problem

  Optimization problem definition.
- std::shared_ptr< SolutionBuilder > solutionBuilder

  Solution generation utility.
- int maxIterations

  Maximum number of iterations.
- std::vector< double > bestSolution

  Best solution found.
- std::vector< double > bestFitnesses

  Best fitness value so far per iteration.
- std::vector< std::vector< double > > solutions

  All generated solutions.

8.23.1 Detailed Description

Implements the Particle Swarm Optimization algorithm.

This optimizer simulates a swarm of particles moving in the search space, updating velocities and positions based on personal and global bests.

8.23.2 Member Function Documentation

8.23.2.1 optimize()

std::vector< double > ParticleSwarm::optimize () [override], [virtual]

Executes the Differential Evolution optimization process.

Returns

The functions execution time.

Implements Optimizer.

The documentation for this class was generated from the following files:

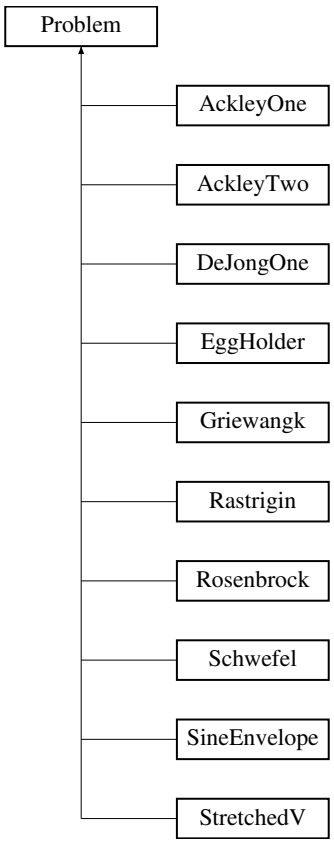
- src/opti_py/cpp/include/opti_py/Optimizer/ParticleSwarm.h
- src/opti_py/cpp/src/Optimizer/ParticleSwarm.cpp

8.24 Problem Class Reference

Abstract base class for all optimization benchmark problems.

#include <Problem.h>

Inheritance diagram for Problem:



Public Member Functions

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- virtual double evaluate (const std::vector< double > &x) const =0

  Evaluates the fitness of a candidate solution.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

Accessors

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Protected Attributes

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.24.1 Detailed Description

Abstract base class for all optimization benchmark problems.

Provides a common interface for evaluating fitness and retrieving search space boundaries.

8.24.2 Constructor & Destructor Documentation

8.24.2.1 Problem()

```
Problem::Problem (
            double lb,
            double ub,
            const std::string_view n)    [inline]
```

Constructs a Problem instance.

- Parameters

lb	Lower bound for the solution space.

ub	Upper bound for the solution space.
n	Name of the optimization function.

8.24.3 Member Function Documentation

8.24.3.1 evaluate()

virtual double Problem::evaluate (
const std::vector< double > & x) const [pure virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).
- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implemented in AckleyOne, AckleyTwo, DeJongOne, EggHolder, Griewangk, Rastrigin, Rosenbrock, Schwefel, SineEnvelope, and StretchedV.

8.24.3.2 getLowerBounds()

double Problem::getLowerBounds () const [inline]

Returns

The lower boundary of the search space.

8.24.3.3 getName()

const std::string Problem::getName () const [inline]

Returns

The name of the benchmark function.

8.24.3.4 getUpperBounds()

double Problem::getUpperBounds () const [inline]

Returns

The upper boundary of the search space.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Problem/Problem.h

8.25 ProblemFactory Class Reference

Utility to create problem instances dynamically.

#include <ProblemFactory.h>

Static Public Member Functions

- static std::shared_ptr< Problem > create (int id)

  Factory method to instantiate a specific benchmark problem.
- static OptResult optimizeDE (int id, double f, double cr, size_t maxGenerations, size_t pop↩Size, unsigned long seed, size_t dimensions, std::string &mutationStrategy, std::string &crossover↩Strategy)

8.25.1 Detailed Description

Utility to create problem instances dynamically.

8.25.2 Member Function Documentation

8.25.2.1 create()

std::shared_ptr< Problem > ProblemFactory::create (
int id) [static]

Factory method to instantiate a specific benchmark problem.

- Parameters

id	Integer ID corresponding to the desired problem (e.g., 1 for Ackley, 2 for DeJong).

Returns

std::unique_ptr<Problem> A pointer to the newly created problem instance.

Exceptions

std::invalid_argument	if the ID does not match a known problem.

The documentation for this class was generated from the following files:

- src/opti_py/cpp/include/opti_py/ProblemFactory.h
- src/opti_py/cpp/src/ProblemFactory.cpp

8.26 ProblemResult Class Reference

Public Attributes

- std::vector< double > solution
- double fitness

The documentation for this class was generated from the following file:

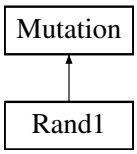
- src/opti_py/cpp/include/opti_py/Problem/ProblemResult.h

8.27 Rand1 Class Reference

Implements the DE/rand/1 mutation strategy.

#include <Rand1.h>

Inheritance diagram for Rand1:



Public Member Functions

- std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound) override

  Generates a mutant vector using DE/rand/1 strategy.

Public Member Functions inherited from Mutation

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.

Additional Inherited Members

Protected Member Functions inherited from Mutation

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.27.1 Detailed Description

Implements the DE/rand/1 mutation strategy.

Strategy: DE/rand/1 (Random-1)

The most basic and exploratory DE mutation strategy. Uses a random base vector and applies a single weighted difference vector to generate variation.

Formula:

$$v_i(t) = x_{r0}(t) + F \cdot (x_{r1}(t) - x_{r2}(t))$$

Where:

- v_i: Mutant vector for target i
- x_r0, x_r1, x_r2: Three randomly selected distinct individuals (r0 i)
- F: Differential weight controlling step size

Algorithm:

1. Select 3 random distinct individuals from population (excluding target)
2. For each dimension j:
   - mutant[j] = r0[j] + F ∗ (r1[j] - r2[j])
   - Clamp mutant[j] to [lowerBound, upperBound]
3. Return the mutant vector

Characteristics:

Exploration: Very high

- Uses random base instead of best, so explores widely
- Single difference vector provides moderate variation
- Good for escaping local optima

Exploitation: Very low

- Random base means no bias toward good solutions
- Slow convergence to final solution
- May waste evaluations on poor regions

Robustness: Excellent

- Least likely to converge prematurely

- Performs well on diverse problem landscapes
- Good for multimodal problems

Convergence Speed: Slow

- Requires many generations to refine solutions
- Well-suited for high-dimensional problems
- Best with large population sizes

Parameter Recommendations:

- F (Differential Weight):
  - Smaller F (0.5-0.8): Finer control, slower convergence
  - Larger F (0.8-1.5): Stronger exploration, faster initial progress
  - Typical: 0.8-1.0
- CR (Crossover Rate):
  - Lower CR (0.3-0.5): More conservative mixing
  - Higher CR (0.7-0.9): More aggressive mixing
  - Typical: 0.9-1.0
- Population Size:
  - Minimum: 4 × dimension (needs at least 3 random individuals + target)
  - Recommended: 10-20 × dimension
  - Larger populations improve robustness

Comparison with Other Strategies:

Strategy	Base	Differences	Exploration	Convergence
Rand1	Random	1	Very High	Very Slow
Best1	Best	1	Low	Very Fast
Rand2	Random	2	Extreme	Slow
Best2	Best	2	Moderate	Moderate
RandBest1	Hybrid	1	Moderate	Moderate

Use Cases:

- Highly multimodal optimization landscapes
- When global exploration is prioritized
- Problems with unknown structure
- As a baseline for algorithm comparison
- When premature convergence is a risk

Avoid When:

- Problem is unimodal and smooth (use Best1 instead)
- Computational budget is very limited (use Best1)
- Need rapid convergence (use Best2 or RandBest1)

Implementation Details:

The implementation follows a straightforward pattern:

1. Call getSubset() to randomly select r0, r1, r2
2. For each dimension, compute: r0 + F ∗ (r1 - r2)
3. Clamp to bounds using std::clamp
4. Return result

Complexity:

Time: O(dimension) Space: O(dimension) for the mutant vector

Example Usage:

```
// Create Rand1 strategy
auto strategy = std::make_unique<Rand1>();

// Generate mutant vector
std::vector<double> mutant = strategy->mutate(
    population,      // Current population
    7,               // Target individual index
    0.8,             // F = 0.8
    bestSolution,    // (unused in Rand1)
    rng,
    -1.0, 1.0        // Domain bounds
);

// Use mutant for crossover and selection
```

Historical Note:

DE/rand/1 was the original DE mutation strategy proposed by Storn and Price (1997). Despite the availability of more refined strategies, it remains a benchmark for algorithm robustness and is commonly used as the default strategy.

See also

Mutation for interface documentation

Best1 for a more exploitative variant

Rand2 for higher variation

8.27.2 Member Function Documentation

8.27.2.1 mutate()

```
std::vector< double > Rand1::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [inline], [override], [virtual]
```

Generates a mutant vector using DE/rand/1 strategy.

Applies the formula: mutant = r0 + F ∗ (r1 - r2) where r0, r1, r2 are three randomly selected distinct individuals.

Parameters

in	population	Current population of solutions

in	targetIndex	Index of the target individual (excluded from selection)
in	F	Differential weight; typical range 0.5-1.5
in	bestVector	Best solution (unused by Rand1, but required by interface)
in,out	mt	Random number generator for selection
in	lowerBound	Lower domain bound
in	upperBound	Upper domain bound

Returns

Mutant vector with all values clamped to [lowerBound, upperBound]

Implements Mutation.

The documentation for this class was generated from the following file:

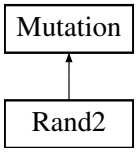
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand1.h

8.28 Rand2 Class Reference

Implements the DE/rand/2 mutation strategy.

#include <Rand2.h>

Inheritance diagram for Rand2:



Public Member Functions

- std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound) override

  Generates a mutant vector using DE/rand/2 strategy.

Public Member Functions inherited from Mutation

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.

Additional Inherited Members

Protected Member Functions inherited from Mutation

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.28.1 Detailed Description

Implements the DE/rand/2 mutation strategy.

Strategy: DE/rand/2 (Random-2)

An extremely exploratory mutation strategy that applies two independent difference vectors to a random base. This creates maximum variation and is most robust to premature convergence at the cost of slow optimization.

Formula:

$$v_i(t) = x_{r0}(t) + F \cdot (x_{r1}(t) - x_{r2}(t)) + F \cdot (x_{r3}(t) - x_{r4}(t))$$

Where:

- v_i: Mutant vector for target i
- x_r0, x_r1, x_r2, x_r3, x_r4: Five randomly selected distinct individuals
- F: Differential weight controlling step size (applied to both differences)

Algorithm:

1. Select 5 random distinct individuals from population (excluding target)
2. For each dimension j:
   - mutant[j] = r0[j] + F ∗ (r1[j] - r2[j]) + F ∗ (r3[j] - r4[j])
   - Clamp mutant[j] to [lowerBound, upperBound]
3. Return the mutant vector

Characteristics:

Exploration: Extreme

- Two independent difference vectors provide maximum variation
- Random base ensures no bias toward good regions
- Creates very diverse mutants

Exploitation: Minimal

- No guidance toward known good solutions
- Essentially random walk with directional information
- Requires many generations for refinement

Robustness: Outstanding

- Extremely difficult to cause premature convergence
- Excellent for deceptive and multimodal landscapes
- May explore useless regions extensively

Convergence Speed: Very slow

- Requires many more evaluations than Rand1
- Much larger search steps may miss good regions
- Best suited for unlimited computational budget

Population Size Requirements:

- Minimum: 6 × dimension
  - Needs at least 5 random individuals + target
  - Requires significantly larger population than Rand1/Best1
- Recommended: 20-30 × dimension
  - Large populations help manage the high variation
  - Small populations may not sustain diversity with large mutations

Parameter Recommendations:

- F (Differential Weight):
  - Smaller F (0.4-0.7): Reduces massive step sizes, more manageable variation
  - Larger F (0.7-1.0): Even larger steps, extreme exploration
  - Typical: 0.5-0.8 (smaller than Rand1 due to two differences)
  - Important: Two differences mean total step can be 2F, so keep F moderate
- CR (Crossover Rate):
  - Lower CR (0.3-0.5): Helps reduce excessive variation
  - Higher CR (0.7-0.9): Can amplify the large mutations
  - Typical: 0.5-0.8
  - Note: Lower CR often better with Rand2 to stabilize mutations
- Generations:
  - Typically 2-5× more than Best1
  - Needs time to accumulate improvements
  - May require early stopping if no improvement

Comparison with Other Strategies:

Strategy	Base	Differences	Exploration	Convergence
Rand1	Random	1	Very High	Very Slow

Strategy	Base	Differences	Exploration	Convergence
Best1	Best	1	Very Low	Very Fast
Rand2	Random	2	Extreme	Extremely Slow
Best2	Best	2	Moderate	Moderate
RandBest1	Hybrid	1	Moderate	Moderate

Use Cases:

- Highly deceptive optimization landscapes
- Extremely multimodal problems with many similar-quality local optima
- When finding any good solution is more important than finding the best
- Unbounded computational resources
- Baseline robustness testing of problems
- Comparison studies measuring strategy differences
- Problems where false gradients lead other strategies astray

Avoid When:

- Time is limited (convergence is very slow)
- Problem has clear structure (too much variation is wasteful)
- Population size is constrained (needs larger populations)
- Moderate exploration with better convergence is preferred

Computational Considerations:

Cost Factors:

- Requires 5 individuals per mutation (vs 3 for Rand1, 2 for Best1)
- Very large mutations may be less effective on large-scale problems
- Needs larger population to maintain diversity

When It Pays Off:

- Problem has few evaluations due to extreme complexity
- Need guaranteed robustness over any other consideration
- Problem structure is completely unknown

Implementation Details:

The implementation uses two independent difference vectors:

1. Call getSubset() to randomly select 5 individuals
2. Compute first difference: F ∗ (r1 - r2)
3. Compute second difference: F ∗ (r3 - r4)
4. Sum both with random base r0
5. Clamp to bounds
6. Return result

Complexity:

Time: O(dimension) Space: O(dimension) for the mutant vector

Example Usage:

```
// Create Rand2 strategy
auto strategy = std::make_unique<Rand2>();

// Generate mutant vector (typically with larger population)
std::vector<double> mutant = strategy->mutate(
    population,     // Needs larger population (20-30x dimension)
    12,             // Target individual index
    0.6,            // F = 0.6 (often smaller with Rand2)
    bestSolution,   // (unused in Rand2)
    rng,
    -1.0, 1.0       // Domain bounds
);

// Use mutant for crossover and selection
```

Typical Performance Profile:

- Rosenbrock function: Poor (overly exploratory for simple landscape)
- Rastrigin function: Excellent (multimodal, finds multiple optima)
- Sphere function: Poor (overkill exploration)
- Schwefel function: Excellent (deceptive landscape benefits from robustness)
- CEC Benchmark (multimodal): Good (if budget allows)

Historical Context:

DE/rand/2 provides the theoretical maximum exploration within the DE framework. While rarely the best choice for a specific problem, it serves as a robustness baseline and is valuable for problems with unknown structure.

See also

Mutation for interface documentation

Rand1 for balanced exploration

Best2 for controlled exploration with best-bias

RandBest1 for practical robustness with better convergence

8.28.2 Member Function Documentation

8.28.2.1 mutate()

```
std::vector< double > Rand2::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [inline], [override], [virtual]
```

Generates a mutant vector using DE/rand/2 strategy.

Applies the formula: mutant = r0 + F ∗ (r1 - r2) + F ∗ (r3 - r4) where r0, r1, r2, r3, r4 are five randomly selected distinct individuals.

Parameters

in	population	Current population of solutions
in	targetIndex	Index of the target individual (excluded from selection)
in	F	Differential weight; typical range 0.4-0.8
in	bestVector	Best solution (unused by Rand2, but required by interface)
in,out	mt	Random number generator for selection
in	lowerBound	Lower domain bound
in	upperBound	Upper domain bound

Returns

Mutant vector with all values clamped to [lowerBound, upperBound]

Implements Mutation.

The documentation for this class was generated from the following file:

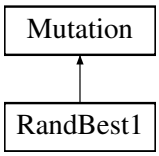
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/Rand2.h

8.29 RandBest1 Class Reference

Implements the DE/rand-to-best/1 mutation strategy.

#include <RandBest1.h>

Inheritance diagram for RandBest1:



Public Member Functions

- RandBest1 (double lambda)

  Constructs a RandBest1 mutation strategy with specified interpolation.
- std::vector< double > mutate (const std::vector< std::vector< double > > &population, size_t targetIndex, double F, const std::vector< double > &bestVector, MersenneTwister &mt, double lowerBound, double upperBound) override

  Generates a mutant vector using DE/rand-to-best/1 strategy.

Public Member Functions inherited from Mutation

- virtual ~Mutation ()=default

  Virtual destructor for safe polymorphic deletion.

Additional Inherited Members

Protected Member Functions inherited from Mutation

- std::vector< size_t > getSubset (size_t populationSize, size_t subsetSize, size_t source, MersenneTwister &mt)

  Selects a random subset of population indices, excluding a source index.

8.29.1 Detailed Description

Implements the DE/rand-to-best/1 mutation strategy.

Strategy: DE/rand-to-best/1 (Rand-to-Best-1)

A hybrid mutation strategy that continuously interpolates between random exploration (Rand1) and best-directed search (Best1). This provides a principled way to balance exploration and exploitation, offering robustness without sacrificing convergence speed.

Formula:

$$v_i(t) = x_i(t) + \lambda \cdot (x_{best}(t) - x_i(t)) + F \cdot (x_{r1}(t) - x_{r2}(t))$$

Where:

- v_i: Mutant vector for target i
- x_i: Target (current) individual
- x_best: Best (lowest fitness) individual in current population
- x_r1, x_r2: Two randomly selected distinct individuals
- : Interpolation parameter (0 = Rand1, 1 = Best1, 0.5 = balanced)
- F: Differential weight controlling step size

Decomposition:

The formula can be viewed as three components:

1. Exploitation: $\times$ (best - current) - pulls toward best
2. Exploration: F $\times$ (r1 - r2) - provides random variation
3. Anchoring: current - keeps solution in neighborhood

Algorithm:

1. Select 2 random distinct individuals from population (excluding target)
2. Compute attraction to best: $\times$ (best - current)
3. Compute random difference: F $\times$ (r1 - r2)
4. For each dimension j:
   - mutant[j] = current[j] + $*$(best[j] - current[j]) + F$*$(r1[j] - r2[j])
   - Clamp mutant[j] to [lowerBound, upperBound]
5. Return the mutant vector

Characteristics:

Exploration (-dependent):

- = 0.0: Pure Rand1, extreme exploration
- = 0.5: Balanced exploration-exploitation
- = 1.0: Pure Best1, minimal exploration
- Typical: 0.5-0.8 for good compromise

Exploitation (-dependent):

- = 0.0: No bias toward best
- = 0.5: Moderate bias toward best
- = 1.0: Strong bias toward best

Robustness:

- = 0.5: Very good (reduces premature convergence)
- $<$ 0.5: Excellent (more like Rand1)
- $>$ 0.5: Good (more like Best1)

Convergence Speed:

- = 0.5: Moderate (better than Rand1, slower than Best1)

- < 0.5: Slow (approaches Rand1)
- > 0.5: Fast (approaches Best1)

Population Size Requirements:

- Minimum: 3 × dimension
  - Needs at least 2 random individuals + target
  - Does not specifically need best, though best helps convergence
- Recommended: 10-20 × dimension
  - Depends on value
  - Larger (closer to Best1) tolerates smaller populations
  - Smaller (closer to Rand1) benefits from larger populations

Parameter Recommendations:

- (Interpolation Parameter):
  - 0.3-0.5: Robust, good for unknown problems
  - 0.5-0.7: Balanced, good for most problems
  - 0.7-0.9: Fast, good for smooth problems
  - Adaptive : Can increase over time (as in jDE) for adaptive control
  - Default (our implementation): 0.8 (slightly exploitative)
  - Note: Experiment with based on problem characteristics
- F (Differential Weight):
  - Smaller F (0.5-0.8): Balanced with bias
  - Larger F (0.8-1.2): Stronger random component
  - Typical: 0.7-1.0
  - Interaction: Larger allows smaller F; smaller benefits from larger F
- CR (Crossover Rate):
  - Lower CR (0.5-0.8): More selective mixing
  - Higher CR (0.8-1.0): More aggressive mixing
  - Typical: 0.9
  - Varies less with than with strategy choice

Comparison with Other Strategies:

Strategy	Base	Differences		Exploration	Convergence	Balance
Rand1	Rand	1	0	Very High	Very Slow	Robust
Best1	Best	1	1	Very Low	Very Fast	Local
Rand2	Rand	2	0	Extreme	Slow	Robust
Best2	Best	2	1	Moderate	Moderate	Good

Strategy	Base	Differences		Exploration	Convergence	Balance
RandBest1	Hybrid	1		Variable	Variable	Tunable

Use Cases:

- General-purpose optimization where problem structure is unknown
- Portfolio approaches as a safe default when one strategy unavailable
- Adaptive schemes where evolves during optimization
- Bridging exploration-exploitation gap between Rand1 and Best1
- Benchmark comparisons to test relative strategy performance
- When Best1 converges prematurely but Rand1 is too slow
- Problems with moderate multimodality where balanced search helps
- Educational purposes to understand strategy continuum

Avoid When:

- Extreme robustness required (use Rand2 with =0)
- Maximum convergence speed needed (use Best1 with =1)
- =0.8 doesn't match problem characteristics (use tuned Best1/Rand1)

Adaptive Usage (jDE-style):

RandBest1 works excellently with parameter adaptation:

- Increasing over time: Start with 0.4, increase toward 0.9 as best improves
- Decreasing F over time: Start with 1.0, decrease as convergence nears
- Crossover adaptation: Increase CR if population diversity declines

This creates a self-tuning approach that balances exploration early and exploitation late in the optimization run.

Mathematical Interpretation:

The interpolation parameter can be understood as:

- Component Weighting: How much to blend best-attraction vs random-difference
- Search Bias: Controls whether search is biased toward current, best, or unbiased
- Convergence Rate: Higher leads to faster convergence curves
- Escape Probability: Lower maintains better escape from local optima

Implementation Details:

The implementation combines three search components:

1. Call getSubset() to select 2 random individuals
2. Compute attraction toward best: × (best - current)
3. Compute random difference: F × (r1 - r2)
4. Sum all components with current individual
5. Clamp to bounds
6. Return result

Complexity:

Time: O(dimension) Space: O(dimension) for the mutant vector

Example Usage:

```
// Create RandBest1 with  =0.5 (balanced)
auto strategy = std::make_unique<RandBest1>(0.5);

// Generate mutant vector
std::vector<double> mutant = strategy->mutate(
    population,      // Current population
    3,               // Target individual index
    0.8,             // F = 0.8
    bestSolution,    // Best individual found
    rng,
    -1.0, 1.0        // Domain bounds
);

// Use mutant for crossover and selection

// For adaptive  :
// auto strategy = std::make_unique<RandBest1>(0.4); // Start exploratory
// lambda_value = std::min(lambda_value + 0.01, 0.9); // Increase toward exploitative
```

Typical Performance Profile (with =0.8):

- Rosenbrock function: Excellent (balanced approach works well)
- Rastrigin function: Good (better than Best1, weaker than Rand1)
- Sphere function: Excellent (bias accelerates convergence)
- Schwefel function: Very Good (balances deception with improvement)
- CEC Benchmark: Good to Excellent (depends on tuning)

Historical Context:

DE/rand-to-best/1 was introduced to provide a principled middle ground between exploration-focused and exploitation-focused strategies. It's been widely adopted in adaptive DE variants and is considered one of the most practical default strategies for unknown problems.

See also

Mutation for interface documentation

Rand1 for pure exploration

Best1 for pure exploitation

Best2 for best-biased with higher variation

8.29.2 Constructor & Destructor Documentation

8.29.2.1 RandBest1()

RandBest1::RandBest1 (
double lambda) [inline], [explicit]

Constructs a RandBest1 mutation strategy with specified interpolation.

Parameter Selection Guidance:

- = 0.3-0.5: Highly exploratory, robust to multimodality
  - Good default for unknown problems
  - Recommended for highly multimodal landscapes
- = 0.5-0.7: Balanced exploration-exploitation
  - Practical default for most problems
  - Good compromise between speed and robustness
- = 0.7-0.9: More exploitative, faster convergence
  - Good for smooth or unimodal problems
  - Faster convergence but higher premature convergence risk
- = 0.8 (typical): Slightly exploitative, practical default
  - Reasonable balance in most situations
  - Starting point for tuning toward problem specifics

Parameters

in	lambda	Interpolation parameter in range [0, 1]. 0 = Rand1 (exploration), 1 = Best1 (exploitation). Typical values: 0.5-0.8. Default often 0.8.

Invariant

0 <= lambda <= 1 (not enforced, but expected by caller)

Example:

```
auto exploratory = std::make_unique<RandBest1>(0.4);  // Explore-biased
auto balanced = std::make_unique<RandBest1>(0.5);     // Balanced
auto exploitative = std::make_unique<RandBest1>(0.8); // Exploit-biased (default)
```

8.29.3 Member Function Documentation

8.29.3.1 mutate()

```
std::vector< double > RandBest1::mutate (
            const std::vector< std::vector< double > > & population,
            size_t targetIndex,
            double F,
            const std::vector< double > & bestVector,
            MersenneTwister & mt,
            double lowerBound,
            double upperBound)   [inline], [override], [virtual]
```

Generates a mutant vector using DE/rand-to-best/1 strategy.

Applies the formula: mutant = current + ∗(best - current) + F∗(r1 - r2) where r1 and r2 are randomly selected, and controls exploration-exploitation.

Parameters

in	population	Current population of solutions

in	targetIndex	Index of the target individual
in	F	Differential weight; typical range 0.5-1.2
in	bestVector	The best solution found so far
in,out	mt	Random number generator for selection
in	lowerBound	Lower domain bound
in	upperBound	Upper domain bound

Returns

Mutant vector with all values clamped to [lowerBound, upperBound]

Implements Mutation.

The documentation for this class was generated from the following file:

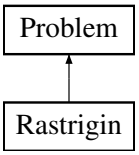
- src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution/Mutation/RandBest1.h

8.30 Rastrigin Class Reference

Implements the Rastrigin benchmark function.

#include <Rastrigin.h>

Inheritance diagram for Rastrigin:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.30.1 Detailed Description

Implements the Rastrigin benchmark function.

8.30.2 Member Function Documentation

8.30.2.1 evaluate()

double Rastrigin::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

  Returns

  The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

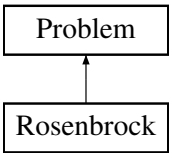
- src/opti_py/cpp/include/opti_py/Problem/Rastrigin.h

8.31 Rosenbrock Class Reference

Implements the Rosenbrock benchmark function.

#include <Rosenbrock.h>

Inheritance diagram for Rosenbrock:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.31.1 Detailed Description

Implements the Rosenbrock benchmark function.

8.31.2 Member Function Documentation

8.31.2.1 evaluate()

double Rosenbrock::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).
- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

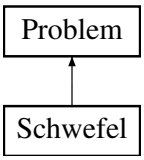
- src/opti_py/cpp/include/opti_py/Problem/Rosenbrock.h

8.32 Schwefel Class Reference

Implements the Schwefel benchmark function.

#include <Schwefel.h>

Inheritance diagram for Schwefel:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override
  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.32.1 Detailed Description

Implements the Schwefel benchmark function.

8.32.2 Member Function Documentation

8.32.2.1 evaluate()

double Schwefel::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

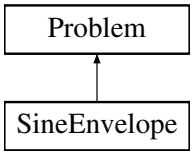
- src/opti_py/cpp/include/opti_py/Problem/Schwefel.h

8.33 SineEnvelope Class Reference

Implements the Sine Envelope benchmark function.

#include <SineEnvelope.h>

Inheritance diagram for SineEnvelope:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.33.1 Detailed Description

Implements the Sine Envelope benchmark function.

8.33.2 Member Function Documentation

8.33.2.1 evaluate()

double SineEnvelope::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).

- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Problem/SineEnvelope.h

8.34 SolutionBuilder Class Reference

Responsible for creating random solutions and neighborhood samples.

#include <SolutionBuilder.h>

Public Member Functions

- SolutionBuilder (int dimensions, int lower, int upper, int seed)
  Constructs a SolutionBuilder with specific space constraints.
- std::vector< double > getRand ()
  Generates a single random solution vector within bounds.
- std::vector< std::vector< double > > getNeighbors (const std::vector< double > &center, int num↩Neighbors, double maxDelta)
  Generates a set of neighboring solutions around a central point.
- double getDimensions ()
  Returns the dimensionality of the solution space.
- std::vector< int > getSubset (int populationSize, int subsetSize, int source)
  Generates a subset of unique solutions from given population.
- std::vector< double > getDifference (const std::vector< double > vec1, const std::vector< double > vec2)
- double checkBounds (double value)
  Ensures a coordinate stays within the defined [lower, upper] bounds.
- double randNum ()
- int randNum (int x, int y)
- std::vector< double > getInitialVelocity ()
- double clampVelocity (double v)

8.34.1 Detailed Description

Responsible for creating random solutions and neighborhood samples.

This class encapsulates the logic for generating initial random positions within the search space and perturbing existing solutions to find neighbors. It utilizes the Mersenne Twister algorithm for high-quality random number generation.

8.34.2 Constructor & Destructor Documentation

8.34.2.1 SolutionBuilder()

```
SolutionBuilder::SolutionBuilder (
            int dimensions,
            int lower,
            int upper,
            int seed)   [inline]
```

Constructs a SolutionBuilder with specific space constraints.

Parameters

dimensions	Number of variables in the solution vector.
lower	Minimum value for any given dimension.
upper	Maximum value for any given dimension.
seed	Value used to initialize the Mersenne Twister generator.

8.34.3 Member Function Documentation

8.34.3.1 checkBounds()

```
double SolutionBuilder::checkBounds (
            double value)
```

Ensures a coordinate stays within the defined [lower, upper] bounds.

Parameters

value	The value to check.

Returns

The clamped value.

8.34.3.2 getNeighbors()

```
std::vector< std::vector< double > > SolutionBuilder::getNeighbors (
            const std::vector< double > & center,
            int numNeighbors,
            double maxDelta)
```

Generates a set of neighboring solutions around a central point.

Used primarily by Local Search algorithms to explore the immediate vicinity of the current best candidate.

Parameters

center	The original solution vector to perturb.

numNeighbors	Number of neighbor vectors to generate.
maxDelta	The maximum step size allowed for perturbation in any dimension.

Returns

A collection of neighboring solution vectors.

8.34.3.3 getRand()

```
std::vector< double > SolutionBuilder::getRand ()
```

Generates a single random solution vector within bounds.

Returns

A vector of size dimensions with values in range [lower, upper].

8.34.3.4 getSubset()

```
std::vector< int > SolutionBuilder::getSubset (
            int populationSize,
            int subsetSize,
            int source)
```

Generates a subset of unique solutions from given population.

Parameters

population	Size population from which subset is selected.
subsetSize	Number of solutions to include in returned subset.

Returns

A collection of unique indices from initial population.

The documentation for this class was generated from the following files:

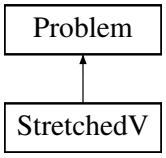
- src/opti_py/cpp/include/opti_py/SolutionBuilder.h
- src/opti_py/cpp/src/SolutionBuilder.cpp

8.35 StretchedV Class Reference

Implements the StretchedV benchmark function.

#include <StretchedV.h>

Inheritance diagram for StretchedV:



Public Member Functions

- double evaluate (const std::vector< double > &x) const override

  Evaluates the fitness of a candidate solution.

Public Member Functions inherited from Problem

- Problem (double lb, double ub, const std::string_view n)

  Constructs a Problem instance.
- virtual ~Problem ()=default

  Virtual destructor for safe polymorphic cleanup.
- double evaluateSolution (const std::vector< double > &solution)
- void getInitialSolutions (std::vector< std::vector< double > > &population)

- double getLowerBounds () const
- double getUpperBounds () const
- const std::string getName () const
- unsigned long getSeed () const
- void setSeed (unsigned long s)
- void setDimensions (size_t dims)

Additional Inherited Members

Protected Attributes inherited from Problem

- const double lowerBound

  Lower bound of the search space.
- const double upperBound

  Upper bound of the search space.
- const std::string name

  Name of the benchmark function.
- size_t dimensions
- unsigned long seed = 0

8.35.1 Detailed Description

Implements the StretchedV benchmark function.

8.35.2 Member Function Documentation

8.35.2.1 evaluate()

double StretchedV::evaluate (
const std::vector< double > & x) const [inline], [override], [virtual]

Evaluates the fitness of a candidate solution.

- This is a pure virtual function that must be implemented by specific benchmark functions (e.g., Ackley, Rosenbrock).
- Parameters

x	The solution vector to evaluate.

Returns

The scalar fitness value (cost).

Implements Problem.

The documentation for this class was generated from the following file:

- src/opti_py/cpp/include/opti_py/Problem/StretchedV.h

Chapter 9

File Documentation

9.1 src/opti_py/cpp/bindings.cpp File Reference

Python bindings for opti-py optimization library.

#include <pybind11/pybind11.h>
#include <pybind11/stl.h>
#include <pybind11/numpy.h>
#include "FlowShop/FlowShop.h"
#include "Optimizer/DifferentialEvolution/DifferentialEvolution.h"
#include "Optimizer/ParticleSwarm.h"
#include "Optimizer/OptResult.h"
#include "ProblemFactory.h"
#include "Problem/ProblemResult.h"
#include "ExperimentConfig.h"

Functions

- PYBIND11_MODULE (_opti_py, m)

9.1.1 Detailed Description

Python bindings for opti-py optimization library.

Author

Alex Buckley

Exposes C++ optimization classes and functions to Python via pybind11. Includes FlowShop, optimization algorithms (DE, PSO), and benchmark problems.

GIL Management: All computationally intensive operations (runNEH, runDE, optimize) release the Python GIL to enable OpenMP parallelization. This prevents threading bottlenecks during optimization.

NumPy Integration:

- Zero-copy views for job matrices and due dates
- Automatic memory layout conversion (C-style to/from numpy)
- Strides configured for row-major access patterns

See also

FlowShop (C++ class documentation for method details)
DifferentialEvolution (C++ optimizer documentation)

9.2 debug.h

```
namespace debug {

constexpr bool enabled =
#ifndef NDEBUG
    true;
#else
    false;
#endif

template<typename... Args>
inline void log(Args&&... args) {
    if constexpr (enabled) {
        (std::cerr « ... « args) « '\n';
    }
}

} // namespace debug
```

9.3 src/opti_py/cpp/include/opti_py/ExperimentConfig.h File Reference

Defines the configuration structure for optimization experiments.

#include <string>
#include <map>

Classes

- struct ExperimentConfig

  Container for all parameters required to execute a benchmark run.

9.3.1 Detailed Description

Defines the configuration structure for optimization experiments.

Author

Alex Buckley

9.4 ExperimentConfig.h

Go to the documentation of this file.

```
#ifndef EXPERIMENT_CONFIG_H
#define EXPERIMENT_CONFIG_H

#include <string>
#include <map>


struct ExperimentConfig {
    int problemType;
    int dimensions;
    double lower;
    double upper;
```

```
    int seed;
    int maxIterations;

    ExperimentConfig(
        int problemType,
        int dimensions,
        double lower,
        double upper,
        int seed = 108664,
        int maxIterations = 30
    )
        : problemType(problemType),
          dimensions(dimensions),
          lower(lower),
          upper(upper),
          seed(seed),
          maxIterations(maxIterations)
    {}


};


#endif
```

9.5 mt.h

```


#ifndef METRICS_MT_H
#define METRICS_MT_H

class MersenneTwister
{
public:
    MersenneTwister(void);
    ~MersenneTwister(void);

    double random(void) { return genrand_real1(); }
    void print(void);

    void init_genrand(unsigned long s);
    void init_by_array(unsigned long* init_key, int key_length);

    unsigned long genrand_int32(void);
    long genrand_int31(void);
    double genrand_real1(void);
    double genrand_real2(void);
    double genrand_real3(void);
    double genrand_res53(void);

private:
    static const int N                  = 624;
    static const int M                  = 397;
    // constant vector a
    static const unsigned long MATRIX_A   = 0x9908b0dfUL;
    // most significant w-r bits
    static const unsigned long UPPER_MASK = 0x80000000UL;
    // least significant r bits
    static const unsigned long LOWER_MASK = 0x7fffffffUL;

    unsigned long* mt_;                 // the state vector
    int mti_;                         // mti == N+1 means mt not initialized

    unsigned long* init_key_;           // Storage for the seed vector
    int key_length_;                  // Seed vector length
    unsigned long s_;                 // Seed integer
    bool seeded_by_array_;              // Seeded by an array
    bool seeded_by_int_;              // Seeded by an integer
};

#endif // METRICS_MT_H
```

9.6 src/opti_py/cpp/include/opti_py/FlowShop/FlowShop.h File Reference

Flow shop scheduling problem implementation.

#include <vector>
#include <cstddef>
#include <cstdint>
#include <stdexcept>
#include <cstring>
#include
#include "FlowShop/FlowShopResult.h"

Classes

- class FlowShop

  Implements the classical flow shop scheduling problem.

9.6.1 Detailed Description

Flow shop scheduling problem implementation.

Author

Alex Buckley

9.7 FlowShop.h

Go to the documentation of this file.

```
00001
00013
00014
00015 #ifndef FLOW_SHOP_H
00016 #define FLOW_SHOP_H
00017
00018
00019 #include <vector>
00020 #include <cstddef>
00021 #include <cstdint>
00022 #include <stdexcept>
00023 #include <cstring>
00024 #include <span>
00025
00026 #include "FlowShop/FlowShopResult.h"
00027
00028
00075
00076 class FlowShop {
00077 public:
00080
00081
00087     FlowShop()
00088         : num_jobs_(0),
00089           num_machines_(0),
00090           due_dates_(0) {}
00091
00092
00103     FlowShop(size_t num_jobs,
00104              size_t num_machines)
00105         : num_jobs_(num_jobs),
00106           num_machines_(num_machines),
00107           jobs_times_(num_jobs * num_machines, 0),
00108           due_dates_(num_jobs, 0) {}
00109
00110
00125     FlowShop(size_t num_jobs,
00126              size_t num_machines,
00127              std::vector<uint64_t>&& data)
00128         : num_jobs_(num_jobs),
00129           num_machines_(num_machines),
```

```
        jobs_times_(std::move(data)),
        due_dates_(num_jobs, 0)
    {
        if (jobs_times_.size() != num_jobs_ * num_machines_)
            throw std::runtime_error("Data size mismatch");
    }




    size_t num_jobs() const { return num_jobs_; }


    size_t num_machines() const { return num_machines_; }


    uint64_t* data() { return jobs_times_.data(); }


    const uint64_t* data() const { return jobs_times_.data(); }


    uint64_t* due_dates_data() { return due_dates_.data(); }


    const uint64_t* due_dates_data() const { return due_dates_.data(); }




    void set_jobs(
        size_t num_jobs,
        size_t num_machines,
        std::vector<uint64_t>&& new_data
    ) {
        // Throw error if size mismatch
        if(new_data.size() != num_jobs * num_machines)
            throw std::runtime_error("Data size mismatch");

        num_jobs_ = num_jobs;
        num_machines_ = num_machines;
        jobs_times_ = std::move(new_data);
        due_dates_.assign(num_jobs_, 0);
    }


    void set_due_dates(std::vector<uint64_t>&& new_data) {
        // Throw error if size mismatch
        if(new_data.size() != num_jobs_)
            throw std::runtime_error("due_dates size must equal num_jobs");

        due_dates_ = std::move(new_data);
    }




    FlowShopResult runNEH(
        bool blocking = false,
        bool optimizeTardiness = false
    );


    FlowShopResult runDE(
        bool blocking,
        bool optimizeTardiness,
        size_t popSize,
        double f,
        double cr,
        size_t maxGenerations,
        unsigned long seed,
        std::string& mutationStrategy,
        std::string& crossoverStrategy
    );




    void getInitialSolutions(std::vector<std::vector<double»& population);


    constexpr double getLowerBounds() const { return -1.0; }
```

```
    constexpr double getUpperBounds() const { return 1.0; }



    double evaluateSolution(const std::vector<double>& solution);


private:

    size_t num_jobs_;

    size_t num_machines_;

    std::vector<uint64_t> jobs_times_;

    std::vector<uint64_t> due_dates_;



    bool blocking;

    bool optimizeTardiness;

    unsigned long seed;



    FlowShopResult buildResult(
        std::vector<size_t> jobOrder,
        bool blocking,
        bool optimizeTardiness,
        std::vector<double> ranks
    );


    void computeRowSums(std::vector<uint64_t>& totalJobTimes);

    void argSortJobs(
        std::vector<uint64_t>& totalJobTimes,
        std::vector<size_t>& jobOrder,
        bool sortDueDates
    );


    std::span<uint64_t> getJob(
        size_t jobNum,
        const std::vector<size_t>& jobOrder
    );


    void updateCompletions(
        const std::span<uint64_t> jobTimes,
        std::vector<std::vector<uint64_t»& completionTimes,
        size_t insertRow,
        bool blocking
    );


    void insertJob(
        std::vector<std::vector<uint64_t»& completionTimes,
        std::vector<size_t>& jobOrder,
        size_t jobNum,
        bool blocking,
        bool optimizeTardiness
    );



    uint64_t evaluateSchedule(
        const std::vector<size_t>& jobOrder,
        bool blocking,
        bool optimizeTardiness
    );


    std::vector<uint64_t> calculateTardiness(
        const std::vector<std::vector<uint64_t»& completionTimes,
        const std::vector<size_t>& jobOrder
    );

    uint64_t calculateTotalTardiness(
```

```
        const std::vector<std::vector<uint64_t»& completionTimes,
        const std::vector<size_t>& jobOrder
    );
    std::vector<double> permToSPV(const std::vector<size_t>& permutation);
    void spvToPerm(
        const std::vector<double>& spvVec,
        std::vector<size_t>& permutation
    );
    void initPopulationVectors(
        std::vector<std::vector<double»& population,
        bool blocking,
        bool optimizeTardiness,
        unsigned long seed
    );
};
#endif
```

9.8 src/opti_py/cpp/include/opti_py/FlowShop/FlowShopResult.h File Reference

Result object for flow shop scheduling optimization.

#include <vector>
#include <cstddef>
#include <cstdint>

Classes

- struct FlowShopResult

  Encapsulates the complete output of a flow shop scheduling optimization.

9.8.1 Detailed Description

Result object for flow shop scheduling optimization.

Author

Alex Buckley

9.9 FlowShopResult.h

Go to the documentation of this file.

```
00001
00012
00013 #ifndef FLOW_SHOP_RESULT_H
00014 #define FLOW_SHOP_RESULT_H
00015
00016 #include <vector>
00017 #include <cstddef>
00018 #include <cstdint>
00019
00020
00095 struct FlowShopResult {
00098
00120     std::vector<size_t> sequence;
00121
00123
00126
00155     uint64_t makespan;
00156
00197     std::vector<std::vector<uint64_t» completionTimes;
00198
00200
00203
00245     std::vector<uint64_t> tardiness;
00246
00248
00251
00329     std::vector<double> fitnesses;
00330
00332 };
00333
00334
00335 #endif
```

9.10 src/opti_py/cpp/include/opti_py/Optimizer/Blind.h File Reference

Header file for the Blind (Random Walk) optimization algorithm.

#include "Optimizer/Optimizer.h"

Classes

- class Blind

  Implements a blind (random walk) optimization algorithm.

9.10.1 Detailed Description

Header file for the Blind (Random Walk) optimization algorithm.

Author

Alex Buckley

9.11 Blind.h

Go to the documentation of this file.

```
#ifndef BLIND_H
#define BLIND_H

#include "Optimizer/Optimizer.h"


class Blind : public Optimizer {
public:

    Blind(std::shared_ptr<SolutionBuilder> solutionBuilder, std::shared_ptr<Problem> problem, int maxIterations)
        : Optimizer(solutionBuilder, problem, maxIterations)
    {
        bestFitnesses.resize(maxIterations);
        bestSolution.resize(solutionBuilder->getDimensions());
        solutions.resize(maxIterations);
    }

    std::vector<double> optimize() override;
};

#endif
```

9.12 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Crossover/AllCrossovers.h File Reference

Convenience header aggregating all DE crossover strategy implementations.

#include "Optimizer/DifferentialEvolution/Crossover/BinCrossover.h"
#include "Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h"

9.12.1 Detailed Description

Convenience header aggregating all DE crossover strategy implementations.

Author

Alex Buckley

This header provides a single include point for all Differential Evolution crossover strategy classes. Use this to include all strategies at once, or include individual strategy headers for selective use.

Included Strategies:

- BinCrossover: Binomial (uniform, independent) crossover
- ExpCrossover: Exponential (continuous block) crossover

Usage:

Include all strategies:

```
#include "Optimizer/DifferentialEvolution/Crossover/AllCrossovers.h"
```

Or include selectively:

```
#include "Optimizer/DifferentialEvolution/Crossover/BinCrossover.h"
#include "Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h"
```

Note: The base Crossover class is not automatically included. Include "Crossover.h" separately if needed for the abstract interface.

See also

Crossover for the base class interface

BinCrossover for binomial (default) crossover documentation

ExpCrossover for exponential (structured) crossover documentation

9.13 AllCrossovers.h

Go to the documentation of this file.

```
00001
00036
00037
00038
00039 #ifndef ALL_CROSSOVERS_H
00040 #define ALL_CROSSOVERS_H
00041
00042 #include "Optimizer/DifferentialEvolution/Crossover/BinCrossover.h"
00043 #include "Optimizer/DifferentialEvolution/Crossover/ExpCrossover.h"
00044
00045 #endif
```

9.14 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Crossover/BinCrossover.h File Reference

Binomial (uniform) crossover strategy implementation.

#include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"
#include <cstddef>

Classes

- class BinCrossover

  Implements binomial (uniform) crossover strategy.

9.14.1 Detailed Description

Binomial (uniform) crossover strategy implementation.

Author

Alex Buckley

Implements the binomial crossover operator, which independently selects each parameter to inherit from the mutant with uniform probability.

9.15 BinCrossover.h

Go to the documentation of this file.

```
#ifndef BIN_CROSSOVER_H
#define BIN_CROSSOVER_H

#include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"

#include <cstddef>


class BinCrossover : public Crossover {
public:
    void crossover(
        std::vector<double>& target,
        const std::vector<double>& mutant,
        double cr,
        MersenneTwister& mt
    ) override {
        size_t jrand = mt.genrand_int32() % target.size();

        for(size_t i = 0; i < target.size(); i++) {
            if (i == jrand || mt.genrand_real1() < cr)
                target[i] = mutant[i];
        }
    }
};



#endif
```

9.16 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Crossover/Crossover.h File Reference

Base class for Differential Evolution crossover strategies.

#include <vector>
#include "External/mt.h"

Classes

- class Crossover

  Abstract base class for Differential Evolution crossover strategies.

9.16.1 Detailed Description

Base class for Differential Evolution crossover strategies.

Author

Alex Buckley

9.17 Crossover.h

Go to the documentation of this file.

```
#ifndef CROSSOVER_H
#define CROSSOVER_H


#include <vector>

#include "External/mt.h"


class Crossover {
public:
    virtual ~Crossover() = default;

    virtual void crossover(
        std::vector<double>& target,
        const std::vector<double>& mutant,
        double cr,
        MersenneTwister& mt
    ) = 0;
};

#endif
```

9.18 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Crossover/ExpCrossover.h File Reference

Exponential (continuous block) crossover strategy implementation.

#include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"
#include <cstddef>

Classes

- class ExpCrossover

  Implements exponential (continuous block) crossover strategy.

9.18.1 Detailed Description

Exponential (continuous block) crossover strategy implementation.

Author

Alex Buckley

Implements the exponential crossover operator, which inherits parameters in contiguous blocks from the mutant, creating structured variation.

9.19 ExpCrossover.h

Go to the documentation of this file.

```
#ifndef EXP_CROSSOVER_H
#define EXP_CROSSOVER_H


#include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"

#include <cstddef>


class ExpCrossover : public Crossover {
public:
    void crossover(
        std::vector<double>& target,
        const std::vector<double>& mutant,
        double cr,
        MersenneTwister& mt
    ) override {
        size_t jrand = mt.genrand_int32() % target.size();
        size_t left = 0;

        do {
            size_t idx = (jrand + left) % target.size();
            target[idx] = mutant[idx];
            left++;
        } while (mt.genrand_real1() < cr && left < target.size());
    }
};


#endif
```

9.20 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/DifferentialEvolution.h File Reference

Differential Evolution (DE) optimization algorithm implementation.

#include <vector>
#include <cstddef>
#include "External/mt.h"
#include "Optimizer/Evaluable.h"
#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include "Optimizer/DifferentialEvolution/Mutation/AllMutations.h"
#include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"
#include "Optimizer/DifferentialEvolution/Crossover/AllCrossovers.h"
#include "Optimizer/OptResult.h"
#include "Optimizer/DifferentialEvolution/DifferentialEvolution.tpp"

Classes

- class DifferentialEvolution

  Implements the Differential Evolution metaheuristic algorithm.

9.20.1 Detailed Description

Differential Evolution (DE) optimization algorithm implementation.

Header file for the Differential Evolution optimization algorithm.

Author

Alex Buckley

9.21 DifferentialEvolution.h

Go to the documentation of this file.

```
00014 #ifndef DIFFERENTIAL_EVOLUTION_H
00015 #define DIFFERENTIAL_EVOLUTION_H
00017 #include <vector>
00018 #include <cstddef>
00020 #include "External/mt.h"
00021 #include "Optimizer/Evaluable.h"
00022 #include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
00023 #include "Optimizer/DifferentialEvolution/Mutation/AllMutations.h"
00024 #include "Optimizer/DifferentialEvolution/Crossover/Crossover.h"
00025 #include "Optimizer/DifferentialEvolution/Crossover/AllCrossovers.h"
00026 #include "Optimizer/OptResult.h"
00048 class DifferentialEvolution {
00048 public:
00052     DifferentialEvolution() = default;
00057     ~DifferentialEvolution() = default;
00076     static inline std::unique_ptr<Mutation> createMutation(
00077         const std::string& name
00078     ) {
00079         if(name == "rand1")
00080             return std::make_unique<Rand1>();
00081         if(name == "rand2")
00082             return std::make_unique<Rand2>();
00083         if(name == "best1")
00084             return std::make_unique<Best1>();
00085         if(name == "best2")
00086             return std::make_unique<Best2>();
00087         if(name == "randToBest1")
00088             return std::make_unique<RandBest1>(0.8);
00090         throw std::runtime_error("Unknown mutation strategy: " + name);
00091     }
00104     static inline std::unique_ptr<Crossover> createCrossover(
00105         const std::string& name
00106     ) {
00107         if(name == "bin")
00108             return std::make_unique<BinCrossover>();
00109         else if(name == "exp")
00110             return std::make_unique<ExpCrossover>();
00111         else
00112             throw std::runtime_error("Unknown crossover strategy");
00113     }
00136     static inline void crossover(
00137         std::vector<double>& target,
00138         const std::vector<double>& mutant,
00139         double cr,
00140         MersenneTwister& mt
00141     );
00189     template<Evaluable Problem>
00190     static OptResult optimize(
00191         Problem& problem,
00192         size_t popSize,
00193         double f,
00194         double cr,
00195         size_t maxGenerations,
00196         unsigned long seed,
00197         std::string& mutationStrategy,
00198         std::string& crossoverStrategy
00199     );
00202 };
00204 #include "Optimizer/DifferentialEvolution/DifferentialEvolution.tpp"
00207 #endif
```

9.22 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/AllMutations.h File Reference

Convenience header aggregating all DE mutation strategy implementations.

#include "Optimizer/DifferentialEvolution/Mutation/Best1.h"
#include "Optimizer/DifferentialEvolution/Mutation/Best2.h"
#include "Optimizer/DifferentialEvolution/Mutation/Rand1.h"
#include "Optimizer/DifferentialEvolution/Mutation/Rand2.h"
#include "Optimizer/DifferentialEvolution/Mutation/RandBest1.h"

9.22.1 Detailed Description

Convenience header aggregating all DE mutation strategy implementations.

Author

Alex Buckley

This header provides a single include point for all Differential Evolution mutation strategy classes. Use this to include all strategies at once, or include individual strategy headers for selective use.

Included Strategies:

- Rand1: DE/rand/1 (pure exploration)
- Rand2: DE/rand/2 (maximum exploration)
- Best1: DE/best/1 (pure exploitation)
- Best2: DE/best/2 (balanced)
- RandBest1: DE/rand-to-best/1 (tunable hybrid)

Usage:

Include all strategies:

```
#include "Optimizer/DifferentialEvolution/Mutation/AllMutations.h"
```

Or include selectively:

```
#include "Optimizer/DifferentialEvolution/Mutation/Best1.h"
#include "Optimizer/DifferentialEvolution/Mutation/Rand1.h"
```

Note: The base Mutation class is not automatically included. Include "Mutation.h" separately if needed for the abstract interface.

See also

Mutation for the base class interface

Rand1, Rand2, Best1, Best2, RandBest1 for individual strategy documentation

9.23 AllMutations.h

Go to the documentation of this file.

```
#ifndef ALL_MUTATIONS_H
#define ALL_MUTATIONS_H


#include "Optimizer/DifferentialEvolution/Mutation/Best1.h"
#include "Optimizer/DifferentialEvolution/Mutation/Best2.h"
#include "Optimizer/DifferentialEvolution/Mutation/Rand1.h"
#include "Optimizer/DifferentialEvolution/Mutation/Rand2.h"
#include "Optimizer/DifferentialEvolution/Mutation/RandBest1.h"

#endif
```

9.24 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/Best1.h File Reference

DE/best/1 mutation strategy implementation.

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>

Classes

- class Best1

  Implements the DE/best/1 mutation strategy.

9.24.1 Detailed Description

DE/best/1 mutation strategy implementation.

Author

Alex Buckley

Implements the DE/best/1 mutation operator, which applies a single weighted difference vector to the best (lowest fitness) individual found so far.

9.25 Best1.h

Go to the documentation of this file.

```
#ifndef BEST1_H
#define BEST1_H


#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>


class Best1 : public Mutation {
public:
    std::vector<double> mutate(
        const std::vector<std::vector<double»& population,
        size_t targetIndex,
        double F,
        const std::vector<double>& bestVector,
        MersenneTwister& mt,
        double lowerBound,
        double upperBound
    ) override {
        std::vector<size_t> subset = getSubset(population.size(), 2, targetIndex, mt);

        const std::vector<double>& xr1 = population[subset[0]];
        const std::vector<double>& xr2 = population[subset[1]];

        std::vector<double> mutated(bestVector.size());

        for(int j = 0; j < mutated.size(); j++) {
            mutated[j] = bestVector[j] + F * (xr1[j] - xr2[j]);
            mutated[j] = std::clamp(mutated[j], lowerBound, upperBound);
        }

        return mutated;
    }
};


#endif
```

9.26 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/Best2.h File Reference

DE/best/2 mutation strategy implementation.

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>

Classes

- class Best2

9.26.1 Detailed Description

DE/best/2 mutation strategy implementation.

Author

Alex Buckley

Implements the DE/best/2 mutation operator, which applies two weighted difference vectors to the best (lowest fitness) individual found so far.

9.27 Best2.h

Go to the documentation of this file.

```
#ifndef BEST2_H
#define BEST2_H

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"

#include <algorithm>


class Best2 : public Mutation {
public:


    std::vector<double> mutate(
        const std::vector<std::vector<double»& population,
        size_t targetIndex,
        double F,
        const std::vector<double>& bestVector,
        MersenneTwister& mt,
        double lowerBound,
        double upperBound
    ) override {

        std::vector<size_t> subset = getSubset(population.size(), 4, targetIndex, mt);

        // Get population subset
        const std::vector<double>& r1 = population[subset[0]];
        const std::vector<double>& r2 = population[subset[1]];
        const std::vector<double>& r3 = population[subset[2]];
        const std::vector<double>& r4 = population[subset[3]];


        std::vector<double> mutated(bestVector.size());

        for(int j = 0; j < mutated.size(); j++) {
            mutated[j] = bestVector[j]
                + F * (r1[j] - r2[j])
                + F * (r3[j] - r4[j]);

            mutated[j] = std::clamp(mutated[j], lowerBound, upperBound);
        }

        return mutated;
    }
};


#endif
```

9.28 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/Mutation.h File Reference

Base class for Differential Evolution mutation strategies.

#include <vector>
#include <cstddef>
#include "External/mt.h"

Classes

- class Mutation

  Abstract base class for Differential Evolution mutation strategies.

9.28.1 Detailed Description

Base class for Differential Evolution mutation strategies.

Author

Alex Buckley

9.29 Mutation.h

Go to the documentation of this file.

```
00001
00011
00012
00013 #ifndef MUTATION_H
00014 #define MUTATION_H
00015
00016 #include <vector>
00017 #include <cstddef>
00018
00019 #include "External/mt.h"
00020
00021
00070 class Mutation {
00071 public:
00075     virtual ~Mutation() = default;
00076
00077
00170     virtual std::vector<double> mutate(
00171         const std::vector<std::vector<double»& population,
00172         size_t targetIndex,
00173         double F,
00174         const std::vector<double>& bestVector,
00175         MersenneTwister& mt,
00176         double lowerBound,
00177         double upperBound
00178     ) = 0;
00179
00180
00181 protected:
00184
00185
00267     std::vector<size_t> getSubset(
00268         size_t populationSize,
00269         size_t subsetSize,
00270         size_t source,
00271         MersenneTwister& mt
00272     ) {
00273         std::vector<size_t> indices(populationSize - 1);
00274         size_t idx = 0;
00275
00276         // Prepare vector of all valid indices except source
00277         for(size_t i = 0; i < populationSize; i++) {
00278             if(i != source) {
00279                 indices[idx] = i;
00280                 ++idx;
00281             }
00282         }
00283
00284         // Partial Fisher-Yates shuffle to select subsetSize random indices
00285         for(size_t i = 0; i < subsetSize; i++) {
00286             int j = i + (mt.genrand_int32() % (indices.size() - i));
00287             std::swap(indices[i], indices[j]);
00288         }
00289
00290         // Return only the first 'subsetSize' indices
00291         indices.resize(subsetSize);
00292         return indices;
00293     }
00294 };
00295
00296 #endif
```

9.30 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/Rand1.h File Reference

DE/rand/1 mutation strategy implementation.

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>

Classes

- class Rand1

  Implements the DE/rand/1 mutation strategy.

9.30.1 Detailed Description

DE/rand/1 mutation strategy implementation.

Author

Alex Buckley

Implements the classical DE/rand/1 mutation operator, which applies a single weighted difference vector to a random base individual.

9.31 Rand1.h

Go to the documentation of this file.

```
00001
00010
00011
00012 #ifndef RAND1_H
00013 #define RAND1_H
00014
00015 #include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
00016
00017 #include <algorithm>
00018
00019
00154
00155 class Rand1 : public Mutation {
00156 public:
00173     std::vector<double> mutate(
00174         const std::vector<std::vector<double»& population,
00175         size_t targetIndex,
00176         double F,
00177         const std::vector<double>& bestVector,
00178         MersenneTwister& mt,
00179         double lowerBound,
00180         double upperBound
00181     ) override {
00182         std::vector<size_t> subset = getSubset(population.size(), 3, targetIndex, mt);
00183
00184         // Get population subset
00185         const std::vector<double>& r1 = population[subset[0]];
00186         const std::vector<double>& r2 = population[subset[1]];
00187         const std::vector<double>& r3 = population[subset[2]];
00188
00189         std::vector<double> mutated(bestVector.size());
00190
00191         // Create mutated vector
00192         for(int i = 0; i < mutated.size(); i++) {
```

```
            mutated[i] = population[subset[1]][i] - population[subset[2]][i];
            mutated[i] *= F;
            mutated[i] += population[subset[0]][i];
            mutated[i] = std::clamp(mutated[i], lowerBound, upperBound);
        }

        return mutated;
    }
};

#endif
```

9.32 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/Rand2.h File Reference

DE/rand/2 mutation strategy implementation.

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>

Classes

- class Rand2

  Implements the DE/rand/2 mutation strategy.

9.32.1 Detailed Description

DE/rand/2 mutation strategy implementation.

Author

Alex Buckley

Implements the DE/rand/2 mutation operator, which applies two weighted difference vectors to a random base individual.

9.33 Rand2.h

Go to the documentation of this file.

```
#ifndef RAND2_H
#define RAND2_H

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"

#include <algorithm>



class Rand2 : public Mutation {
public:
    std::vector<double> mutate(
        const std::vector<std::vector<double»& population,
        size_t targetIndex,
        double F,
```

```
        const std::vector<double>& bestVector,
        MersenneTwister& mt,
        double lowerBound,
        double upperBound
    ) override {
        std::vector<size_t> subset = getSubset(population.size(), 5, targetIndex, mt);

        // Get population subset
        const std::vector<double>& r1 = population[subset[0]];
        const std::vector<double>& r2 = population[subset[1]];
        const std::vector<double>& r3 = population[subset[2]];
        const std::vector<double>& r4 = population[subset[3]];
        const std::vector<double>& r5 = population[subset[4]];

        std::vector<double> mutated(r5.size());

        for(int j = 0; j < mutated.size(); j++) {
            mutated[j] = r5[j]
                + F * (r1[j] - r2[j])
                + F * (r3[j] - r4[j]);

            mutated[j] = std::clamp(mutated[j], lowerBound, upperBound);
        }

        return mutated;
    }
};

#endif
```

9.34 src/opti_py/cpp/include/opti_py/Optimizer/Differential↩ Evolution/Mutation/RandBest1.h File Reference

DE/rand-to-best/1 mutation strategy implementation.

#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"
#include <algorithm>

Classes

- class RandBest1

  Implements the DE/rand-to-best/1 mutation strategy.

9.34.1 Detailed Description

DE/rand-to-best/1 mutation strategy implementation.

Author

Alex Buckley

Implements the DE/rand-to-best/1 mutation operator, which interpolates between random exploration and best-directed search using an interpolation parameter.

9.35 RandBest1.h

Go to the documentation of this file.

```
#ifndef RANDBEST1_H
#define RANDBEST1_H


#include "Optimizer/DifferentialEvolution/Mutation/Mutation.h"

#include <algorithm>


class RandBest1 : public Mutation {
private:
    double lambda_;


public:
    explicit RandBest1(double lambda)
            : lambda_(lambda) {}


    std::vector<double> mutate(
        const std::vector<std::vector<double»& population,
        size_t targetIndex,
        double F,
        const std::vector<double>& bestVector,
        MersenneTwister& mt,
        double lowerBound,
        double upperBound
    ) override {
        std::vector<size_t> subset = getSubset(population.size(), 2, targetIndex, mt);

        // Get population subset
        const auto& xi  = population[targetIndex];
        const auto& r1  = population[subset[0]];
        const auto& r2  = population[subset[1]];

        std::vector<double> mutated(xi.size());

        for (int i = 0; i < mutated.size(); i++) {
            mutated[i] =
                xi[i]
                + lambda_ * (bestVector[i] - xi[i])
                + F * (r1[i] - r2[i]);

            mutated[i] = std::clamp(mutated[i], lowerBound, upperBound);
        }

        return mutated;
    }
};

#endif
```

9.36 src/opti_py/cpp/include/opti_py/Optimizer/Evaluable.h File Reference

Evaluable concept for optimization problem definitions.

#include <vector>

Concepts

- concept Evaluable

  Concept defining the interface for optimization problems.

9.36.1 Detailed Description

Evaluable concept for optimization problem definitions.

Author

Alex Buckley

9.37 Evaluable.h

Go to the documentation of this file.

```
00001
00012
00013 #pragma once
00014 #include <vector>
00015
00143 template<typename T>
00144 concept Evaluable = requires(T t, std::vector<std::vector<double»& pop, const std::vector<double>& sol) {
00145     { t.getInitialSolutions(pop) } -> std::same_as<void>;
00146     { t.getLowerBounds() } -> std::convertible_to<double>;
00147     { t.getUpperBounds() } -> std::convertible_to<double>;
00148     { t.evaluateSolution(sol) } -> std::convertible_to<double>;
00149 };
```

9.38 src/opti_py/cpp/include/opti_py/Optimizer/LocalSearch.h File Reference

Header file for the Local Search optimization algorithm.

#include "Optimizer/Optimizer.h"

Classes

- class LocalSearch

  Implements a local search optimization algorithm.

9.38.1 Detailed Description

Header file for the Local Search optimization algorithm.

Author

Alex Buckley

9.39 LocalSearch.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef LOCAL_SEARCH_H
00010 #define LOCAL_SEARCH_H
00011
00012 #include "Optimizer/Optimizer.h"
00013
00026 class LocalSearch : public Optimizer {
00027 private:
00029     const double delta;
00030
00032     const int numNeighbors;
00033
00040     void localSearch();
00041
00042 public:
00052     LocalSearch(
00053         std::shared_ptr<SolutionBuilder> solutionBuilder,
00054         std::shared_ptr<Problem> problem,
00055         int maxIterations,
00056         double delta,
00057         int numNeighbors)
00058         : Optimizer(solutionBuilder, problem, maxIterations),
00059           delta(delta),
00060           numNeighbors(numNeighbors)
00061     {}
00062
00068     std::vector<double> optimize() override;
00069 };
00070
00071 #endif
00072
```

9.40 src/opti_py/cpp/include/opti_py/Optimizer/Optimizer.h File Reference

#include <vector>
#include <memory>
#include "Problem/Problem.h"
#include "SolutionBuilder.h"

Classes

- class Optimizer

  Abstract base class for all optimization algorithms.

9.40.1 Detailed Description

Author

Alex Buckley

9.41 Optimizer.h

Go to the documentation of this file.

```
#ifndef OPTIMIZER_H
#define OPTIMIZER_H

#include <vector>
#include <memory>

#include "Problem/Problem.h"
#include "SolutionBuilder.h"


class Optimizer {
public:
    Optimizer(std::shared_ptr<SolutionBuilder> solutionBuilder,
              std::shared_ptr<Problem> problem,
              int maxIterations)
        : problem(problem),
          solutionBuilder(solutionBuilder),
          maxIterations(maxIterations)
    {}


    virtual ~Optimizer() = default;

    virtual std::vector<double>optimize() = 0;

    // Getters

    double getBestFitness() { return bestFitnesses.back(); }

    std::vector<double>& getBestSolution() { return bestSolution; }

    std::vector<double>& getBestFitnesses() { return bestFitnesses; }

    std::vector<std::vector<double»& getSolutions() { return solutions; }

    int getMaxIterations() { return maxIterations; }

    std::shared_ptr<Problem> getProblem() { return problem; }

    std::shared_ptr<SolutionBuilder> getSolutionBuilder() { return solutionBuilder; }



protected:
    std::shared_ptr<Problem> problem;

    std::shared_ptr<SolutionBuilder> solutionBuilder;

    int maxIterations;

    std::vector<double> bestSolution;

    std::vector<double> bestFitnesses;

    std::vector<std::vector<double» solutions;
};

#endif
```

9.42 src/opti_py/cpp/include/opti_py/Optimizer/OptimizerFactory.h File Reference

Factory utility for instantiating different optimizer types.

#include "Optimizer/Optimizer.h"
#include "Optimizer/Blind.h"
#include "Optimizer/LocalSearch.h"
#include "Optimizer/DifferentialEvolution.h"
#include "Optimizer/Mutation/AllMutations.h"

#include "Optimizer/Crossover/AllCrossovers.h"
#include <memory>
#include <string>

Classes

- class OptimizerFactory

  Factory class for creating optimizer instances.

9.42.1 Detailed Description

Factory utility for instantiating different optimizer types.

Author

Alex Buckley

9.43 OptimizerFactory.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef OPTIMIZER_FACTORY_H
00010 #define OPTIMIZER_FACTORY_H
00011
00012 #include "Optimizer/Optimizer.h"
00013 #include "Optimizer/Blind.h"
00014 #include "Optimizer/LocalSearch.h"
00015 #include "Optimizer/DifferentialEvolution.h"
00016
00017 #include "Optimizer/Mutation/AllMutations.h"
00018 #include "Optimizer/Crossover/AllCrossovers.h"
00019
00020 #include <memory>
00021 #include <string>
00022
00023
00032 class OptimizerFactory {
00033 private:
00034     static std::unique_ptr<DifferentialEvolution> createDefaultDE(
00035         Problem& problem,
00036         SolutionBuilder& builder,
00037         int maxGenerations,
00038         int popSize,
00039         double scale,
00040         double crossoverRate
00041     ) {
00042         // Choose fixed strategies here
00043         auto mutation = std::make_unique<Rand1>();
00044         auto crossover = std::make_unique<BinCrossover>();
00045
00046         return std::make_unique<DifferentialEvolution>(
00047             builder,
00048             problem,
00049             maxGenerations,
00050             popSize,
00051             scale,
00052             crossoverRate,
00053             std::move(crossover),
00054             std::move(mutation)
00055         );
00056     }
00057
00058 public:
00059
```

```
    static std::unique_ptr<Optimizer> initOptimizer(Problem& problem, ExperimentConfig& config, SolutionBuilder& builder) {
        if(config.optimizer == "blind")
            return std::make_unique<Blind>(builder, problem, config.maxIterations);
        else if(config.optimizer == "local")
            return std::make_unique<LocalSearch>(builder, problem, 1, config.neighborDelta, config.numNeighbors);
        else if(config.optimizer == "repeated local")
            return std::make_unique<LocalSearch>(builder, problem, config.maxIterations, config.neighborDelta, config.numNeighbors);
        else if(config.optimizer == "DE")
            return OptimizerFactory::createDefaultDE(problem, builder, config.maxIterations, config.popSize, config.scale, config.cr);

        return nullptr;
    }


};

#endif
```

9.44 src/opti_py/cpp/include/opti_py/Optimizer/OptResult.h File Reference

Result object for Differential Evolution optimization.

#include <vector>

Classes

- struct OptResult

  Encapsulates the complete output of a Differential Evolution optimization run.

9.44.1 Detailed Description

Result object for Differential Evolution optimization.

Author

Alex Buckley

9.45 OptResult.h

Go to the documentation of this file.

```


#ifndef OPT_RESULT_H
#define OPT_RESULT_H

#include <vector>


struct OptResult {

    std::vector<double> bestSolution;


    std::vector<double> bestFitnesses;

    double bestFitness;

};


#endif
```

9.46 ParticleSwarm.h

```
#ifndef PARTICLE_SWARM_H
#define PARTICLE_SWARM_H

#include <memory>

#include "Optimizer/Optimizer.h"
#include "ProblemFactory.h"
#include "ExperimentConfig.h"


class ParticleSwarm : public Optimizer {
private:
    const int popSize;

    const double c1;
    const double c2;

    ParticleSwarm(
        std::shared_ptr<Problem> prob,
        const ExperimentConfig& config,
        int popSize,
        double c1,
        double c2)
        : Optimizer(
            std::make_shared<SolutionBuilder>(
                config.dimensions,
                config.lower,
                config.upper,
                config.seed
            ),
            prob,
            config.maxIterations
        ),
        popSize(popSize),
        c1(c1),
        c2(c2)
    {}

    std::vector<std::vector<double» initPopulation();
    std::vector<std::vector<double» initVelocities();

    void updatePosition(
        std::vector<double>& solution,
        std::vector<double>& velocity
    );

    void updateVelocity(
        std::vector<double>& velocity,
        const std::vector<double>& pop,
        const std::vector<double>& pBestPos,
        const std::vector<double>& gBestPos
    );

public:
    ParticleSwarm(
        const ExperimentConfig& config,
        double c1,
        double c2,
        int popSize)
        : ParticleSwarm(
            ProblemFactory::create(config.problemType),
            config,
            popSize,
            c1,
            c2
        )
    {}



    std::vector<double> optimize() override;


};

#endif
```

9.47 src/opti_py/cpp/include/opti_py/Problem/AckleyOne.h File Reference

Implementation of the Ackley 1 function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>

Classes

- class AckleyOne

  Implements the Ackley 1 benchmark function.

9.47.1 Detailed Description

Implementation of the Ackley 1 function.

Author

Alex Buckley

9.48 AckleyOne.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef ACKLEY_ONE_H
00010 #define ACKLEY_ONE_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014 #include <cmath>
00015
00016
00021 class AckleyOne : public Problem {
00022 private:
00023     static constexpr double LOWER = -32.0;
00024     static constexpr double UPPER = 32.0;
00025     static constexpr std::string_view NAME = "AckleyOne";
00026
00027 public:
00028     AckleyOne() : Problem(LOWER, UPPER, NAME) {}
00029
00030     double evaluate(const std::vector<double>& x) const override {
00031         double sum = 0.0;
00032
00033         for(size_t i = 0; i < x.size() - 1; i++) {
00034             double term1 = std::exp(-0.2) * std::sqrt(x[i] * x[i] + x[i+1] * x[i+1]);
00035             double term2 = 3 * (std::cos(2 * x[i]) + std::sin(2 * x[i+1]));
00036             sum += term1 + term2;
00037         }
00038
00039         return sum;
00040     }
00041 };
00042
00043 #endif
```

9.49 src/opti_py/cpp/include/opti_py/Problem/AckleyTwo.h File Reference

Implementation of the Ackley 2 function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>
#include <numbers>

Classes

- class AckleyTwo

  Implements the Ackley 2 benchmark function.

9.49.1 Detailed Description

Implementation of the Ackley 2 function.

Author

Alex Buckley

9.50 AckleyTwo.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef ACKLEY_TWO_H
00010 #define ACKLEY_TWO_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014 #include <cmath>
00015 #include <numbers>
00016
00017
00022 class AckleyTwo : public Problem {
00023 private:
00024     static constexpr double LOWER = -32.0;
00025     static constexpr double UPPER = 32.0;
00026     static constexpr std::string_view NAME = "AckleyTwo";
00027
00028 public:
00029     AckleyTwo() : Problem(LOWER, UPPER, NAME) {}
00030
00031     double evaluate(const std::vector<double>& x) const override {
00032         double sum = 0.0;
00033
00034         for(size_t i = 0; i < x.size() - 1; i++) {
00035             double sqrtTerm = std::sqrt((x[i] * x[i] + x[i+1] * x[i+1]) / 2);
00036             double term3 = -20 / (std::exp(0.2 * sqrtTerm));
00037             double term4 = -1 * std::exp(0.5 * std::cos(2 * std::numbers::pi * x[i]) + std::cos(2 * std::numbers::pi * x[i+1]));
00038
00039             sum += 20 + std::exp(1.0) + term3 + term4;
00040         }
00041
00042         return sum;
00043     }
00044 };
00045
00046 #endif
```

9.51 DeJongOne.h

```
00001
00007
00008 #ifndef DEJONG_H
00009 #define DEJONG_H
00010
00011 #include "Problem/Problem.h"
00012 #include <vector>
00013
00014
00019 class DeJongOne : public Problem {
00020 private:
00021     static constexpr double LOWER = -100.0;
00022     static constexpr double UPPER = 100.0;
00023     static constexpr std::string_view NAME = "DeJong_1";
00024
00025 public:
00026     // Constructor method
00027     DeJongOne() : Problem(LOWER, UPPER, NAME) {}
00028
00029     // Evaluate function
00030     double evaluate(const std::vector<double>& x) const override {
00031         double sum = 0.0;
00032
00033         for(double xi : x)
00034             sum += xi * xi;
00035
00036         return sum;
00037     }
00038 };
00039
00040 #endif
```

9.52 src/opti_py/cpp/include/opti_py/Problem/EggHolder.h File Reference

Implementation of the Egg Holder function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>

Classes

- class EggHolder

  Implements the Egg Holder benchmark function.

9.52.1 Detailed Description

Implementation of the Egg Holder function.

Author

Alex Buckley

9.53 EggHolder.h

Go to the documentation of this file.

```
#ifndef EGG_HOLDER_H
#define EGG_HOLDER_H

#include "Problem/Problem.h"
#include <vector>
#include <cmath>


class EggHolder : public Problem {
private:
    static constexpr double LOWER = -500.0;
    static constexpr double UPPER = 500.0;
    static constexpr std::string_view NAME = "EggHolder";

    inline double sinSqrtAbs(double n) const {
        return std::sin(std::sqrt(std::abs(n)));
    }

public:
    EggHolder() : Problem(LOWER, UPPER, NAME) {}

    double evaluate(const std::vector<double>& x) const override {
        double sum = 0.0;

        for(size_t i = 0; i < x.size() - 1; i++) {
            double term1 = -x[i] * sinSqrtAbs(x[i] - x[i+1] - 47);
            double term2 = -(x[i+1] + 47) * sinSqrtAbs(x[i+1] + 47 + x[i] / 2);

            sum += term1 + term2;
        }

        return sum;
    }
};

#endif
```

9.54 src/opti_py/cpp/include/opti_py/Problem/Griewangk.h File Reference

Implementation of the Griewangk function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>

Classes

- class Griewangk

  Implements the Griewangk benchmark function.

9.54.1 Detailed Description

Implementation of the Griewangk function.

Author

Alex Buckley

9.55 Griewangk.h

Go to the documentation of this file.

```
#ifndef GRIEWANGK_H
#define GRIEWANGK_H

#include "Problem/Problem.h"
#include <vector>
#include <cmath>


class Griewangk : public Problem {
private:
    static constexpr double LOWER = -500.0;
    static constexpr double UPPER = 500.0;
    static constexpr std::string_view NAME = "Griewangk";

public:
    Griewangk() : Problem(LOWER, UPPER, NAME) {}

    double evaluate(const std::vector<double>& x) const override {
        double sum = 0.0;
        double prod = 1.0;

        for(size_t i = 0; i < x.size(); i++) {
            sum += x[i] * x[i];
            prod *= std::cos(x[i] / std::sqrt(i + 1));
        }

        return 1.0 + sum / 4000.0 - prod;
    }
};

#endif
```

9.56 src/opti_py/cpp/include/opti_py/Problem/Problem.h File Reference

#include <vector>
#include <omp.h>
#include "External/mt.h"

Classes

- class Problem

  Abstract base class for all optimization benchmark problems.

9.56.1 Detailed Description

Author

Alex Buckley

9.57 Problem.h

Go to the documentation of this file.

```
#ifndef PROBLEM_H
#define PROBLEM_H

#include <vector>
#include <omp.h>

#include "External/mt.h"

class Problem {
protected:
    const double lowerBound;
    const double upperBound;
    const std::string name;
    size_t dimensions;
    unsigned long seed = 0;

public:
    Problem(double lb, double ub, const std::string_view n)
        : lowerBound(lb), upperBound(ub), name(n) {}

    virtual ~Problem() = default;

    virtual double evaluate(const std::vector<double>& x) const = 0;

    double evaluateSolution(const std::vector<double>& solution) {
        return evaluate(solution);
    }

    void getInitialSolutions(std::vector<std::vector<double»& population) {
        MersenneTwister mt;
        mt.init_genrand(seed);

        for(int i = 0; i < population.size(); i++) {
            population[i].resize(dimensions);
            for(int j = 0; j < population[i].size(); j++) {
                population[i][j] = lowerBound + (upperBound - lowerBound) * mt.genrand_real1();
            }
        }
    }


    double getLowerBounds() const { return lowerBound; }

    double getUpperBounds() const { return upperBound; }

    const std::string getName() const { return name; }

    unsigned long getSeed() const {
        return seed;
    }

    void setSeed(unsigned long s) {
        seed = s;
    }

    void setDimensions(size_t dims) {
        dimensions = dims;
    }


};

#endif // PROBLEM_H
```

9.58 ProblemResult.h

```
#ifndef PROBLEM_RESULT_H
#define PROBLEM_RESULT_H

#include <vector>

class ProblemResult {
public:
    std::vector<double> solution;
    double fitness;
```

```
};

#endif
```

9.59 src/opti_py/cpp/include/opti_py/Problem/Rastrigin.h File Reference

Implementation of the Ratrigin function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>
#include <numbers>

Classes

- class Rastrigin

  Implements the Rastrigin benchmark function.

9.59.1 Detailed Description

Implementation of the Ratrigin function.

Author

Alex Buckley

9.60 Rastrigin.h

Go to the documentation of this file.

```
#ifndef RASTRIGIN_H
#define RASTRIGIN_H

#include "Problem/Problem.h"
#include <vector>
#include <cmath>
#include <numbers>


class Rastrigin : public Problem {
private:
    static constexpr double LOWER = -30.0;
    static constexpr double UPPER = 30.0;
    static constexpr std::string_view NAME = "Rastrigin";

public:
    Rastrigin() : Problem(LOWER, UPPER, NAME) {}

    double evaluate(const std::vector<double>& x) const override {
        double sum = 0.0;

        for(double xi : x) // Calculate summation
            sum += xi * xi - 10 * std::cos(2 * std::numbers::pi * xi);

        return 10 * x.size() + sum;
    }
};

#endif
```

9.61 src/opti_py/cpp/include/opti_py/Problem/Rosenbrock.h File Reference

Implementation of the Rosenbrock function.

#include "Problem/Problem.h"
#include <vector>

Classes

- class Rosenbrock

  Implements the Rosenbrock benchmark function.

9.61.1 Detailed Description

Implementation of the Rosenbrock function.

Author

Alex Buckley

9.62 Rosenbrock.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef ROSENBROCK_H
00010 #define ROSENBROCK_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014
00015
00020 class Rosenbrock : public Problem {
00021 private:
00022     static constexpr double LOWER = -100.0;
00023     static constexpr double UPPER = 100.0;
00024     static constexpr std::string_view NAME = "Rosenbrock";
00025
00026 public:
00027     Rosenbrock() : Problem(LOWER, UPPER, NAME) {}
00028
00029     double evaluate(const std::vector<double>& x) const override {
00030         double sum = 0.0;
00031
00032         for(size_t i = 0; i < x.size() - 1; i++) {
00033             // Calculate inner terms
00034             const double term1 = x[i] * x[i] - x[i+1];
00035             const double term2 = 1 - x[i];
00036
00037             // Calculate value for summation
00038             sum += 100 * term1 * term1 + term2 * term2;
00039         }
00040
00041         return sum;
00042     }
00043 };
00044
00045 #endif
```

9.63 src/opti_py/cpp/include/opti_py/Problem/Schwefel.h File Reference

Implementation of the Schwefel function.

#include "Problem/Problem.h"
#include <vector>

Classes

- class Schwefel

  Implements the Schwefel benchmark function.

9.63.1 Detailed Description

Implementation of the Schwefel function.

Author

Alex Buckley

9.64 Schwefel.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef SCHWEFEL_H
00010 #define SCHWEFEL_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014
00015
00020 class Schwefel : public Problem {
00021 private:
00022     static constexpr double LOWER = -512.0;
00023     static constexpr double UPPER = 512.0;
00024     static constexpr std::string_view NAME = "Schwefel";
00025
00026 public:
00027     Schwefel() : Problem(LOWER, UPPER, NAME) {}
00028
00029     double evaluate(const std::vector<double>& x) const override {
00030         double sum = 0.0;
00031
00032         for(double xi : x)
00033             sum += xi * std::sin(std::sqrt(std::abs(xi)));
00034
00035         return 418.9829 * x.size() - sum;
00036     }
00037 };
00038
00039 #endif
```

9.65 src/opti_py/cpp/include/opti_py/Problem/SineEnvelope.h File Reference

Implementation of the Sine Envelope function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>

Classes

- class SineEnvelope

  Implements the Sine Envelope benchmark function.

9.65.1 Detailed Description

Implementation of the Sine Envelope function.

Author

Alex Buckley

9.66 SineEnvelope.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef SINE_ENVELOPE_H
00010 #define SINE_ENVELOPE_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014 #include <cmath>
00015
00016
00021 class SineEnvelope : public Problem {
00022 private:
00023     static constexpr double LOWER = -30.0;
00024     static constexpr double UPPER = 30.0;
00025     static constexpr std::string_view NAME = "SineEnvelope";
00026
00027 public:
00028     SineEnvelope() : Problem(LOWER, UPPER, NAME) {}
00029
00030     double evaluate(const std::vector<double>& x) const override {
00031         double sum = 0.0;
00032
00033         for(size_t i = 0; i < x.size() - 1; i++) {
00034             double sqrSum = x[i] * x[i] + x[i+1] * x[i+1];
00035             double numerator = std::sin(sqrSum - 0.5) * std::sin(sqrSum - 0.5);
00036             double denom = (1 + 0.001 * sqrSum) * (1 + 0.001 * sqrSum);
00037             sum += numerator / denom + 0.5;
00038         }
00039
00040         return sum * -1;
00041     }
00042 };
00043
00044 #endif
```

9.67 src/opti_py/cpp/include/opti_py/Problem/StretchedV.h File Reference

Implementation of the Stretched V function.

#include "Problem/Problem.h"
#include <vector>
#include <cmath>

Classes

- class StretchedV

  Implements the StretchedV benchmark function.

9.67.1 Detailed Description

Implementation of the Stretched V function.

Author

Alex Buckley

9.68 StretchedV.h

Go to the documentation of this file.

```
00001
00007
00008
00009 #ifndef STRETCHED_V_H
00010 #define STRETCHED_V_H
00011
00012 #include "Problem/Problem.h"
00013 #include <vector>
00014 #include <cmath>
00015
00016
00021 class StretchedV : public Problem {
00022 private:
00023     static constexpr double LOWER = -30.0;
00024     static constexpr double UPPER = 30.0;
00025     static constexpr std::string_view NAME = "StretchedV";
00026
00027 public:
00028     StretchedV() : Problem(LOWER, UPPER, NAME) {}
00029
00030     double evaluate(const std::vector<double>& x) const override {
00031         double sum = 0.0;
00032
00033         for(size_t i = 0; i < x.size() - 1; i++) {
00034             double sqrSum = x[i] * x[i] + x[i+1] * x[i+1];
00035             double factor1 = std::sqrt(std::sqrt(sqrSum));
00036             double factor2 = std::sin(50 * std::pow(sqrSum, 0.1));
00037
00038             sum += factor1 * factor2 * factor2 + 1;
00039         }
00040
00041         return sum;
00042     }
00043 };
00044
00045 #endif
```

9.69 src/opti_py/cpp/include/opti_py/ProblemFactory.h File Reference

Factory for instantiating benchmark problems by ID.

#include "Problem/Problem.h"
#include "Optimizer/DifferentialEvolution/DifferentialEvolution.h"
#include "Optimizer/OptResult.h"
#include "Problem/ProblemResult.h"
#include <array>
#include <memory>
#include <stdexcept>

Classes

- class ProblemFactory

  Utility to create problem instances dynamically.

9.69.1 Detailed Description

Factory for instantiating benchmark problems by ID.

Author

Alex Buckley

9.70 ProblemFactory.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef PROBLEM_FACTORY_H
00009 #define PROBLEM_FACTORY_H
00010
00011 #include "Problem/Problem.h"
00012 #include "Optimizer/DifferentialEvolution/DifferentialEvolution.h"
00013 #include "Optimizer/OptResult.h"
00014 #include "Problem/ProblemResult.h"
00015
00016 #include <array>
00017 #include <memory>
00018 #include <stdexcept>
00019
00020
00025 class ProblemFactory {
00026 public:
00033     static std::shared_ptr<Problem> create(int id);
00034
00035
00036     static inline OptResult optimizeDE(
00037         int id,
00038         double f,
00039         double cr,
00040         size_t maxGenerations,
00041         size_t popSize,
00042         unsigned long seed,
00043         size_t dimensions,
00044         std::string& mutationStrategy,
00045         std::string& crossoverStrategy
00046     ) {
00047         // Create problem
```

```
        std::shared_ptr<Problem> problem = create(id);
        problem->setSeed(seed);
        problem->setDimensions(dimensions);

        // Optimize problem
        return DifferentialEvolution::optimize(
            *problem,
            popSize,
            f,
            cr,
            maxGenerations,
            seed,
            mutationStrategy,
            crossoverStrategy
        );
    }
};


#endif
```

9.71 src/opti_py/cpp/include/opti_py/SolutionBuilder.h File Reference

Utility class for generating and manipulating candidate solutions.

#include <vector>
#include "Problem/Problem.h"
#include "External/mt.h"

Classes

- class SolutionBuilder

  Responsible for creating random solutions and neighborhood samples.

9.71.1 Detailed Description

Utility class for generating and manipulating candidate solutions.

Author

Alex Buckley

9.72 SolutionBuilder.h

Go to the documentation of this file.

```


#ifndef SOLUTION_BUILDER_H
#define SOLUTION_BUILDER_H

#include <vector>

#include "Problem/Problem.h"
#include "External/mt.h"


class SolutionBuilder {
```

```
private:
    const int dimensions;
    const int lower;
    const int upper;
    MersenneTwister mt;

public:
    SolutionBuilder(int dimensions, int lower, int upper, int seed)
        : dimensions(dimensions),
          lower(lower),
          upper(upper)
    {
        mt.init_genrand(seed);
    }


    std::vector<double> getRand();


    std::vector<std::vector<double» getNeighbors(
        const std::vector<double>& center,
        int numNeighbors,
        double maxDelta
    );


    double getDimensions() { return dimensions; }


    std::vector<int> getSubset(
        int populationSize,
        int subsetSize,
        int source
    );


    std::vector<double> getDifference(
        const std::vector<double> vec1,
        const std::vector<double> vec2
    );


    double checkBounds(double value);


    double randNum() { return mt.genrand_real1(); }
    int randNum(int x, int y) { return x + mt.genrand_int32() % (y - x); }
    std::vector<double> getInitialVelocity();
    double clampVelocity(double v);
};


#endif
```

Index

~MersenneTwister
  MersenneTwister, 72

AckleyOne, 19
  evaluate, 20
AckleyTwo, 21
  evaluate, 22

Best1, 22
  mutate, 26
Best2, 27
  mutate, 28
bestFitness
  OptResult, 87
bestFitnesses
  OptResult, 88
bestSolution
  OptResult, 90
BinCrossover, 28
  crossover, 32
Blind, 33
  Blind, 34
  optimize, 34

checkBounds
  SolutionBuilder, 122
completionTimes
  FlowShopResult, 64
Concept Definitions, 14
create
  ProblemFactory, 96
createCrossover
  DifferentialEvolution, 41
createMutation
  DifferentialEvolution, 41
Crossover, 35
  crossover, 36
crossover
  BinCrossover, 32
  Crossover, 36
  DifferentialEvolution, 41
  ExpCrossover, 50

data
  FlowShop, 55
DE Crossover Strategies, 12
DE Mutation Strategies, 13
DE Optimization Results, 14
DeJong, 38
DeJongOne, 38
  evaluate, 39
Differential Evolution Optimizer, 12
DifferentialEvolution, 40
  createCrossover, 41
  createMutation, 41
  crossover, 41
  optimize, 42
due_dates_data
  FlowShop, 55

EggHolder, 44
  evaluate, 45
Evaluable, 15
evaluate
  AckleyOne, 20
  AckleyTwo, 22
  DeJongOne, 39
  EggHolder, 45
  Griewangk, 69
  Problem, 95
  Rastrigin, 116
  Rosenbrock, 118
  Schwefel, 119
  SineEnvelope, 121
  StretchedV, 126
evaluateSolution
  FlowShop, 56
ExpCrossover, 45
  crossover, 50
ExperimentConfig, 51

fitnesses
  FlowShopResult, 64
Flow Shop Results, 11
Flow Shop Scheduling, 11
FlowShop, 51
  data, 55
  due_dates_data, 55
  evaluateSolution, 56
  FlowShop, 54
  getInitialSolutions, 56
  getLowerBounds, 57
  getUpperBounds, 57
  num_jobs, 58
  num_machines, 58
  runDE, 58
  runNEH, 59
  set_due_dates, 60
  set_jobs, 61
FlowShopResult, 61

completionTimes, 64
fitnesses, 64
makespan, 66
sequence, 66
tardiness, 67

genrand_int31
MersenneTwister, 73
genrand_int32
MersenneTwister, 73
genrand_real1
MersenneTwister, 73
genrand_real2
MersenneTwister, 73
genrand_real3
MersenneTwister, 73
genrand_res53
MersenneTwister, 74
getBestFitness
Optimizer, 82
getBestFitnesses
Optimizer, 82
getBestSolution
Optimizer, 83
getInitialSolutions
FlowShop, 56
getLowerBounds
FlowShop, 57
Problem, 95
getMaxIterations
Optimizer, 83
getName
Problem, 95
getNeighbors
SolutionBuilder, 122
getProblem
Optimizer, 83
getRand
SolutionBuilder, 124
getSolutionBuilder
Optimizer, 83
getSolutions
Optimizer, 83
getSubset
Mutation, 77
SolutionBuilder, 124
getUpperBounds
FlowShop, 57
Problem, 95
Griewangk, 68
evaluate, 69

init_by_array
MersenneTwister, 74
init_genrand
MersenneTwister, 74
initOptimizer
OptimizerFactory, 84

LocalSearch, 70
LocalSearch, 71
optimize, 71

makespan
FlowShopResult, 66
MersenneTwister, 72
~MersenneTwister, 72
genrand_int31, 73
genrand_int32, 73
genrand_real1, 73
genrand_real2, 73
genrand_real3, 73
genrand_res53, 74
init_by_array, 74
init_genrand, 74
MersenneTwister, 72
print, 74
mutate
Best1, 26
Best2, 28
Mutation, 78
Rand1, 101
Rand2, 107
RandBest1, 114
Mutation, 75
getSubset, 77
mutate, 78

num_jobs
FlowShop, 58
num_machines
FlowShop, 58

Optimization Algorithms, 11
Optimization Problems, 14
optimize
Blind, 34
DifferentialEvolution, 42
LocalSearch, 71
Optimizer, 83
ParticleSwarm, 93
Optimizer, 81
getBestFitness, 82
getBestFitnesses, 82
getBestSolution, 83
getMaxIterations, 83
getProblem, 83
getSolutionBuilder, 83
getSolutions, 83
optimize, 83
Optimizer, 82
OptimizerFactory, 84
initOptimizer, 84
OptResult, 85
bestFitness, 87
bestFitnesses, 88
bestSolution, 90

ParticleSwarm, 91
optimize, 93
print
MersenneTwister, 74
Problem, 93
evaluate, 95
getLowerBounds, 95
getName, 95
getUpperBounds, 95
Problem, 94
ProblemFactory, 96
create, 96
ProblemResult, 97

Rand1, 97
mutate, 101
Rand2, 102
mutate, 107
RandBest1, 107
mutate, 114
RandBest1, 113
Rastrigin, 115
evaluate, 116
Rosenbrock, 117
evaluate, 118
runDE
FlowShop, 58
runNEH
FlowShop, 59

Schwefel, 118
evaluate, 119
sequence
FlowShopResult, 66
set_due_dates
FlowShop, 60
set_jobs
FlowShop, 61
SineEnvelope, 120
evaluate, 121
SolutionBuilder, 121
checkBounds, 122
getNeighbors, 122
getRand, 124
getSubset, 124
SolutionBuilder, 122
src/opti_py/cpp/bindings.cpp, 127
src/opti_py/cpp/include/opti_py/debug.h, 128
src/opti_py/cpp/include/opti_py/ExperimentConfig.h, 128
src/opti_py/cpp/include/opti_py/External/mt.h, 129
src/opti_py/cpp/include/opti_py/FlowShop/FlowShop.h, 129, 130
src/opti_py/cpp/include/opti_py/FlowShop/FlowShopResult.h, 133, 134
src/opti_py/cpp/include/opti_py/Optimizer/Blind.h, 134, 135
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 135, 136
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 136, 137
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 137, 138
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 138, 139
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 139, 140
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 141, 142
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 142, 143
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 143, 144
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 144, 145
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 146
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 147
src/opti_py/cpp/include/opti_py/Optimizer/DifferentialEvolution, 148, 149
src/opti_py/cpp/include/opti_py/Optimizer/Evaluable.h, 149, 150
src/opti_py/cpp/include/opti_py/Optimizer/LocalSearch.h, 150, 151
src/opti_py/cpp/include/opti_py/Optimizer/Optimizer.h, 151, 152
src/opti_py/cpp/include/opti_py/Optimizer/OptimizerFactory.h, 152, 153
src/opti_py/cpp/include/opti_py/Optimizer/OptResult.h, 154
src/opti_py/cpp/include/opti_py/Optimizer/ParticleSwarm.h, 155
src/opti_py/cpp/include/opti_py/Problem/AckleyOne.h, 156
src/opti_py/cpp/include/opti_py/Problem/AckleyTwo.h, 157
src/opti_py/cpp/include/opti_py/Problem/DeJongOne.h, 158
src/opti_py/cpp/include/opti_py/Problem/EggHolder.h, 158, 159
src/opti_py/cpp/include/opti_py/Problem/Griewangk.h, 159, 160
src/opti_py/cpp/include/opti_py/Problem/Problem.h, 160, 161
src/opti_py/cpp/include/opti_py/Problem/ProblemResult.h, 161
src/opti_py/cpp/include/opti_py/Problem/Rastrigin.h, 162
src/opti_py/cpp/include/opti_py/Problem/Rosenbrock.h, 163
src/opti_py/cpp/include/opti_py/Problem/Schwefel.h, 164
src/opti_py/cpp/include/opti_py/Problem/SineEnvelope.h, 165

src/opti_py/cpp/include/opti_py/Problem/StretchedV.h, 166
src/opti_py/cpp/include/opti_py/ProblemFactory.h, 167
src/opti_py/cpp/include/opti_py/SolutionBuilder.h, 168
StretchedV, 125
    evaluate, 126

tardiness
    FlowShopResult, 67